DNSR: A Security-Hardened Iterative DNS Resolver Built from Scratch in Go

BCSE308P – COURSE PROJECT

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Science and Engineering (Cyber Security)

in

Department of Cybersecurity and Intelligent Systems

by

Student Name(s)

Register Number(s)



School of Computer Science Engineering and Information Systems [SCORE]

VIT, Vellore

November, 2026

November, 2026

DECLARATION

I hereby declare that the BCSE308P – Course Project entitled "DNSR: A Security-Hardened Iterative DNS Resolver Built from Scratch in Go" submitted by me, for the award of the degree of *Bachelor of Computer Science and Engineering (Cyber Security)*, School of Computer Science Engineering and Information Systems to VIT is a record of bonafide work carried out by me under the supervision of Faculty Guide Name, Designation, SCORE, VIT, Vellore.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the **BCSE308P – Course Project** entitled "DNSR: A Security-Hardened Iterative DNS Resolver Built from Scratch in Go" submitted by Student Name & Reg. No., SCORE, VIT, for the award of the degree of *Bachelor of Computer Science and Engineering (Cyber Security)*, School of Computer Science Engineering and Information Systems, is a record of bonafide work carried out under my supervision during the period dd.mm.yyyy to dd.mm.yyyy, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE – Guide

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Faculty Guide Name, Designation, School of Computer Science Engineering and Information Systems (SCORE), VIT, Vellore, for the guidance, encouragement and valuable suggestions given throughout this project.

I thank the Dean of SCORE, the Head of the Department of Cybersecurity and Intelligent Systems and all the faculty members of the department for providing the knowledge, facilities and environment needed to carry out this work. I am grateful to the management of Vellore Institute of Technology for the opportunity to undertake this course project.

I also acknowledge the authors of the Internet standards and of the open-source tools on which this project builds, and the operators of the public DNS infrastructure whose servers answered the many queries sent during testing.

Finally, I thank my family and friends for their constant support and encouragement.

Student Name

EXECUTIVE SUMMARY

The Domain Name System (DNS) turns names into addresses for almost every Internet connection, which makes it a prime target for spoofing and cache poisoning. Yet the recursive resolver, the component that walks the DNS tree and decides which answers to trust, is usually hidden inside large production systems or libraries.

This project presents DNSR, an iterative DNS resolver written from scratch in Go using only the standard library. It encodes and decodes the RFC 1035 format by hand, resolves names from the root servers with QNAME minimisation, follows referrals, glue, glueless delegations and CNAMEs, and falls back to TCP. It rejects forged replies using random IDs, ports and letter case, rejects out-of-bailiwick data, validates DNSSEC signatures from the root trust anchors, and bounds its caches. It runs as a traced command-line tool or as an access-controlled, rate-limited server over UDP, TCP and DNS over TLS 1.3.

All 32 offline tests against a simulated hierarchy with signed and hostile servers pass. Live, DNSR matched Google Public DNS for 15 domains (median 557 ms cold, under 1 ms cached), validated seven signed domains as secure and rejected a mis-signed one, and survived 9.2 million fuzzing inputs.

CONTENTS

		Page No.
	Acknowledgement	iv
	Executive Summary	v
	Table of Contents	vi
	List of Figures	viii
	List of Tables	x
	Abbreviations	xii
	Symbols and Notations	xiv
1	**INTRODUCTION**	1
	1.1 Objective	2
	1.2 Motivation	3
	1.3 Background	4
2	**PROJECT DESCRIPTION AND GOALS**	8
	2.1 Literature Review	8
	2.2 Gap Identification	11
	2.3 Problem Statement	12
	2.4 Project Goals	12
	2.5 Scope of the Project	13
3	**TECHNICAL SPECIFICATION**	14
	3.1 Functional Requirements	14
	3.2 Security Requirements	15
	3.3 Non-Functional Requirements	15
	3.4 Hardware and Software Specification	16
	3.5 DNS Message Format	16
	3.6 Operating Limits and Constants	19
	3.7 Command-Line Interface Specification	20
4	**DESIGN APPROACH AND DETAILS**	21
	4.1 Design Approach / Materials and Methods	21
	4.2 Codes and Standards	41
	4.3 Constraints, Alternatives and Trade-offs	42

5 SCHEDULE, TASKS AND MILESTONES 45
5.1 Task Breakdown 45
5.2 Gantt Chart 46
5.3 Milestones 46
5.4 Risk Management 46

6 PROJECT DEMONSTRATION 48
6.1 Sample Codes 48
6.2 Sample Screen Shots 67

7 COST ANALYSIS / RESULT AND DISCUSSION 80
7.1 Experimental Setup 80
7.2 Functional Verification 80
7.3 Live Resolution: Correctness and Latency 81
7.4 Cost of QNAME Minimisation 83
7.5 Effect of Caching 84
7.6 Record Types and Special Cases 85
7.7 DNSSEC Validation of Real Domains 86
7.8 Server-Mode Evaluation 88
7.9 Security and Robustness Evaluation 89
7.10 Codec and Signature Performance 92
7.11 Security Review Findings and Their Status 92
7.12 Achievement of Goals 93
7.13 Cost Analysis 93
7.14 Discussion 94

8 SUMMARY 95
8.1 Findings and Results 95
8.2 Recommendations 96
8.3 Future Work 96

9 REFERENCES 98

Appendix A – Complete Source Code 101

Appendix B – User Manual (README) 146

LIST OF FIGURES

Figure No.	Title	Page No.
1.1	Part of the DNS Namespace Used in This Project	5
1.2	Recursive Query from a Stub and Iterative Queries from the Resolver	6
2.1	Kaminsky-Style Cache Poisoning by an Off-Path Attacker	9
3.1	Layout of a DNS Message and Its 12-Byte Header	17
3.2	Name Compression: a Pointer to the Question Name	18
4.1	System Architecture of DNSR	23
4.2	Iterative Resolution Flowchart	26
4.3	Sequence of Messages for a Cold-Cache Lookup of www.github.com	28
4.4	Bailiwick Decisions for Replies from a .com Server	32
4.5	Questions Seen by Each Server Without and With QNAME Minimisation	33
4.6	Chain of Trust from the Root Trust Anchor to an Answer	34
4.7	Structure of the Recursive Server Mode	37
4.8	Offline Test Hierarchy on Loopback Addresses	39
5.1	Gantt Chart of the Project Schedule	46
6.1	Building DNSR and Running the Offline Test Suite	68
6.2	Usage Message Printed When No Name Is Given	69
6.3	Resolving www.github.com with QNAME Minimisation	70
6.4	The Same Lookup with Minimisation Disabled (-nomin)	70
6.5	MX Lookup for gmail.com	71
6.6	AAAA Lookup for www.wikipedia.org Through a CNAME	71
6.7	NXDOMAIN and NODATA Responses	72
6.8	Glueless Delegation for github.io	73
6.9	Glueless Delegation and NS Lookup for vit.ac.in	74
6.10	Truncated UDP Reply Retried over TCP for apple.com TXT	75
6.11	Delegation Cache and Answer Cache in One Run	76
6.12	Quiet Mode Printing Only the Answers	76
6.13	DNSSEC Chain of Trust for cloudflare.com	77
6.14	Insecure, Bogus and Secure Results	78

6.15	DNSR Running as a Validating Server with DNS over TLS	78
6.16	An Independent Client Using UDP, TCP and DNS over TLS	78
6.17	Server Started with an Access List	79
6.18	Refused Client and TLS Version Enforcement	79
6.19	Random-Packet Test, Benchmarks and Coverage-Guided Fuzzing	79
7.1	Cold-Cache Resolution Time per Domain	83
7.2	Queries per Resolution With and Without QNAME Minimisation	84
7.3	Effect of the Delegation Cache and the Answer Cache	85
7.4	Resolution Time Without and With DNSSEC Validation	87
7.5	Server Mode: Latency of First and Repeated Queries	88
7.6	Probability of a Successful Blind Spoof as a Function of Forged Packets	90
7.7	Outcome of One Million Random and Mutated Packets	91

LIST OF TABLES

Table No.	Title	Page No.
1.1	Record Types Supported by DNSR	6
2.1	Comparison of Existing DNS Resolution Approaches	11
2.2	Project Goals and How They Are Measured	12
3.1	Functional Requirements	14
3.2	Security Requirements	15
3.3	Development and Test Environment	16
3.4	Header Fields and Their Use in DNSR	17
3.5	Resolver Constants	19
3.6	Command-Line Usage and Flags	20
4.1	Module Description	23
4.2	Main Data Structures	24
4.3	Threat Model	29
4.4	DNSSEC Algorithms and Digests Supported	35
4.5	Test Cases of the Offline Suite	39
4.6	Standards Followed	41
4.7	Design Alternatives and Decisions	42
5.1	Task Breakdown and Schedule	45
5.2	Project Milestones	46
5.3	Risks and Mitigations	47
7.1	Offline Test Results	81
7.2	Live Cold-Cache Resolution of 15 Domains (Three Runs Each)	81
7.3	Latency with Empty, Delegation and Answer Caches (ms, hops in brackets)	84
7.4	Live Results for Each Supported Record Type	85
7.5	DNSSEC Validation Results (Median of Three Cold-Cache Runs)	86
7.6	Server Mode over UDP: First and Repeated Query	88
7.7	Server Mode over DNS over TLS	89
7.8	Fuzzing Results	91
7.9	Benchmark Results (Intel Core i7-13650HX)	92
7.10	Findings of the Security Review	92
7.11	Goals and Results	93

7.12 Cost of Resources 94

LIST OF ABBREVIATIONS

AA	Authoritative Answer (header flag)
AAAA	IPv6 address record
AD	Authenticated Data (header flag)
API	Application Programming Interface
CD	Checking Disabled (header flag)
CDN	Content Delivery Network
CLI	Command-Line Interface
CNAME	Canonical Name record
DNS	Domain Name System
DNSKEY	DNS public key record
DNSSEC	DNS Security Extensions
DO	DNSSEC OK (EDNS flag)
DoH	DNS over HTTPS
DoT	DNS over TLS
DS	Delegation Signer record
ECDSA	Elliptic Curve Digital Signature Algorithm
EDNS(0)	Extension Mechanisms for DNS, version 0
FQDN	Fully Qualified Domain Name
IANA	Internet Assigned Numbers Authority
ID	Transaction Identifier
IETF	Internet Engineering Task Force
IP	Internet Protocol
ISP	Internet Service Provider
KSK	Key-Signing Key
mDNS	Multicast DNS
MTU	Maximum Transmission Unit
MX	Mail Exchanger record
NODATA	Name exists but has no records of the requested type
NS	Name Server record
NSEC, NSEC3	Next Secure records (authenticated denial of existence)
NXDOMAIN	Non-Existent Domain (response code 3)
OPT	EDNS(0) option pseudo-record
OS	Operating System
PTR	Pointer record (reverse lookup)

QNAME	Query Name
QR	Query/Response (header flag)
RA	Recursion Available (header flag)
RCODE	Response Code
RD	Recursion Desired (header flag)
RDATA	Resource Record Data
RFC	Request for Comments
RR	Resource Record
RRset	Resource Record Set
RRSIG	Resource Record Signature
RSA	Rivest–Shamir–Adleman public-key algorithm
RTT	Round-Trip Time
SERVFAIL	Server Failure (response code 2)
SLD	Second-Level Domain
SOA	Start of Authority record
TC	Truncation (header flag)
TCP	Transmission Control Protocol
TLD	Top-Level Domain
TLS	Transport Layer Security
TTL	Time To Live
TXT	Text record
UDP	User Datagram Protocol
ZSK	Zone-Signing Key

SYMBOLS AND NOTATIONS

P	Probability that a blind spoofing attempt succeeds
N	Number of equally likely (ID, port) combinations
k	Number of forged replies that arrive before the genuine reply
Rport	Size of the ephemeral source-port range
$t$exp	Absolute expiry time of a cached RRset
$t$store	Time at which an RRset was stored in the cache
$t$now	Current time
TTLi	Time to live of the i-th record of an RRset (seconds)
TTLrem	Remaining time to live returned on a cache hit
Li	Latency of the i-th resolution (ms)
$\bar{L}$	Mean latency over n resolutions (ms)
Lcold, Lcached	Latency with an empty cache and with a warm cache
ρ	Relative latency reduction due to caching (%)
n	Number of measured resolutions
Ldnssec, Lplain	Latency with and without DNSSEC validation (ms)
Δ	Extra latency caused by validation (ms)
$\lfloor x \rfloor$	Floor of x (largest integer not greater than x)

CHAPTER 1

INTRODUCTION

The Domain Name System (DNS) is the naming service of the Internet. Every time a browser opens a web page, a mail server delivers a message or a phone application contacts its back-end, a DNS lookup first turns a human-readable name such as `www.github.com` into the numeric IP address that the network actually routes on. Because almost every connection starts with such a lookup, DNS is one of the most heavily used and most security-critical protocols in existence. If an attacker can change the answer to a DNS question, the attacker can silently redirect users to a server of their choice, without breaking any encryption and without touching the victim's computer.

Despite its importance, the inner working of DNS is invisible to most programmers and students. Applications call a library function such as `getaddrinfo()` or Go's `net.LookupHost()`, the operating system forwards the question to a resolver run by an Internet Service Provider (ISP) or a public service such as 8.8.8.8, and an answer comes back a few milliseconds later. The part that does the real work, the *recursive resolver* that walks the DNS tree from the root servers down to the authoritative server of the domain, is hidden inside large production systems such as BIND or Unbound. The rules by which a resolver decides which data to trust, which are exactly the rules that attackers try to break, are therefore rarely seen, tested or understood.

This project, **DNSR**, opens that black box. DNSR is an iterative (recursive) DNS resolver written from scratch in the Go programming language, using only Go's standard library. No DNS library is used: every byte of the binary message format defined in RFC 1035 is encoded and decoded by hand. Starting from the thirteen root server addresses published by the Internet Assigned Numbers Authority (IANA), DNSR sends non-recursive queries, follows referrals level by level, chases aliases, handles glueless delegations and falls back to TCP when a reply is too large for UDP. Along the way it applies the defensive checks that protect a resolver against spoofing and cache poisoning, validates signed answers with DNSSEC from the root trust anchor, reveals to each server only as much of the queried name as that server needs (QNAME minimisation), and prints a readable trace of every step so that the resolution process can be observed and studied.

DNSR can be used in two ways: as a command-line tool that resolves one or more names and shows the full trace, or as a recursive DNS server that standard DNS clients can query over UDP, TCP or encrypted DNS over TLS. Its behaviour is verified by an offline test suite that builds a complete fake DNS hierarchy on loopback addresses, by live measurements against the real Internet, and by fuzz testing of the packet parser. This report describes the motivation, design, implementation, testing and results of the project.

1.1 Objective

The primary objective of this project is to design, implement and evaluate a small, transparent and security-hardened recursive DNS resolver that resolves names directly from the root servers, without relying on any upstream resolver or DNS library. The specific objectives are listed below.

- ✓ To implement an RFC 1035 compliant encoder and decoder for DNS messages, including the header, question and resource-record sections, domain-name compression and the EDNS(0) OPT pseudo-record, and to support the A, AAAA, NS, CNAME, SOA, PTR, MX and TXT record types.
- ✓ To implement iterative resolution starting from the IANA root hints, including referral following, use of glue records, resolution of glueless delegations, CNAME chasing, and correct handling of NXDOMAIN and NODATA responses.
- ✓ To harden the resolver against spoofing and cache poisoning by using random transaction IDs, strict matching of each reply against its query, bailiwick checks on both referrals and glue records, and loop protection in the packet parser.
- ✓ To validate answers with DNSSEC, building a chain of trust from the IANA root trust anchors through DS and DNSKEY records down to the signature on each answer, and to reject forged or tampered data.
- ✓ To improve performance with a TTL-aware answer cache and a delegation (zone to name-server) cache, together with automatic fallback from UDP to TCP for truncated responses, while bounding the size and lifetime of every cache.

- ✓ To protect user privacy and the service itself: QNAME minimisation towards authoritative servers, DNS over TLS 1.3 towards clients, and an access list, rate limiting and resource limits in server mode.
- ✓ To provide two modes of operation: a command-line interface that prints a human-readable resolution trace, and a recursive DNS server mode that can be queried by standard DNS clients.
- ✓ To verify correctness and robustness through an automated offline test suite, live measurements against Internet domains, and fuzz testing of the message parser, and to analyse the remaining security gaps.

1.2 Motivation

DNS cache poisoning is one of the oldest and most damaging attacks on Internet infrastructure. In 2008 Dan Kaminsky showed that an off-path attacker could poison the cache of almost any recursive resolver within seconds by asking it for random, non-existent names and racing forged replies against the genuine ones [9]. The forged replies did not carry a fake address for the queried name; instead they carried a forged *referral* and forged *glue* records that redirected the entire domain to the attacker. The attack forced an emergency, industry-wide patch of resolver software. The lessons learned, randomise everything the attacker must guess and accept only records that the replying server is entitled to give, are now codified in RFC 5452 [8] and in the data-ranking rules of RFC 2181 [4].

These defences are simple to state but easy to get wrong, and research continues to find new ways around them. Son and Shmatikov [10] showed that weak bailiwick rules in deployed resolvers still allowed poisoning, and the SAD DNS attack of 2020 [11] used a side channel in the operating system to infer randomised source ports, bringing Kaminsky-style poisoning back to modern resolvers. The NXNSAttack [24] showed that the way a resolver follows referrals can be abused to amplify traffic towards victim servers. For a student of cyber security, the best way to understand these attacks is to build a resolver that must defend against them and to see exactly where each check sits in the code.

A second motivation is education and transparency. Tools such as `dig +trace` show the referral chain but do not cache or validate anything, while production resolvers are hundreds of thousands of lines of C code. A compact resolver of about

one thousand lines, which prints each referral, each glue decision and each cache hit, makes the protocol concrete. It can be used in class to demonstrate how DNS works, in the laboratory to reproduce attacks against a fake DNS hierarchy, and as a base for further research such as DNSSEC validation or privacy features.

Finally, the project is motivated by the value of writing protocol code from first principles. Parsing untrusted binary data received from the network is a classic source of vulnerabilities: buffer over-reads, infinite loops caused by compression pointers and crashes caused by inconsistent length fields. Implementing the parser by hand in a memory-safe language, and then attacking it with millions of random and mutated packets, provides practical experience in secure coding that a library-based approach would hide.

1.3 Background

This section introduces the DNS concepts that are needed to follow the rest of the report: the namespace, the different kinds of DNS servers, the difference between recursive and iterative queries, the message format and the most important record types.

1.3.1 The DNS namespace and zones

The DNS namespace is a tree. The root of the tree is written as a single dot. Below the root are the top-level domains (TLDs) such as `com.`, `org.`, `in.` and `io.`; below them are second-level domains such as `github.com.` and `ac.in.`, and so on. Each node is identified by its *fully qualified domain name* (FQDN), which lists the labels from the node up to the root, for example `www.github.com.`. Each label may be at most 63 bytes long and a complete name at most 255 bytes [2].

Administration of the tree is divided into *zones*. A zone is a part of the tree that is managed by one organisation and served by a set of authoritative name servers. The root zone delegates each TLD to the TLD operator, the `com.` zone delegates `github.com.` to GitHub's name servers, and so on. A delegation is expressed by NS (name server) records in the parent zone. Fig. 1.1 shows part of the tree with the zones that appear in the experiments of this project.

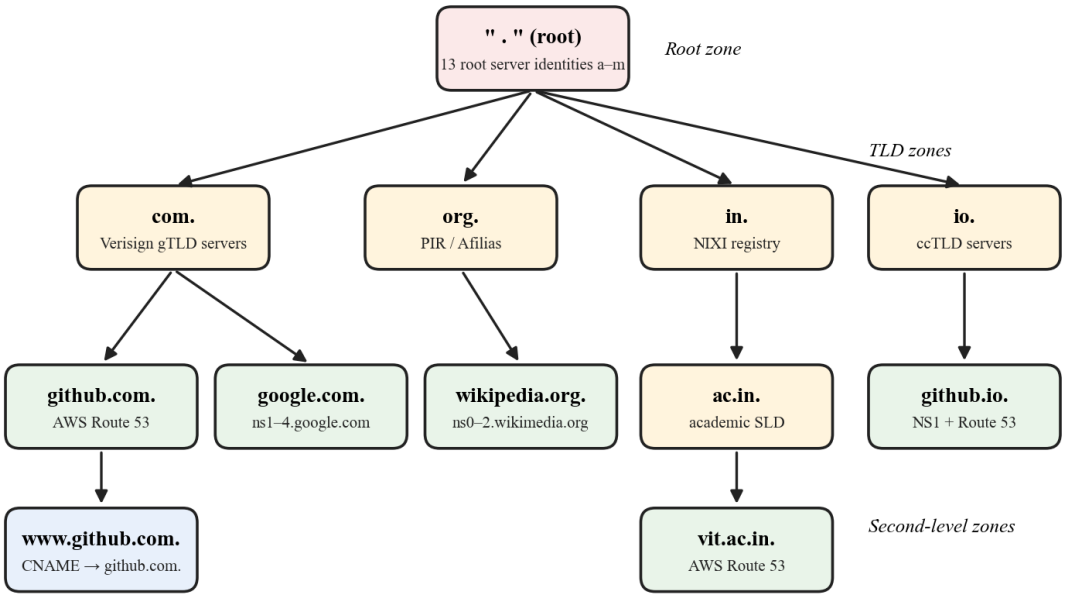


Fig. 1.1 Part of the DNS Namespace Used in This Project

1.3.2 Stub, recursive and authoritative servers

Three kinds of DNS software cooperate to answer a question. A *stub resolver* is the small client inside the operating system; it sends the question to a recursive resolver with the Recursion Desired (RD) flag set and waits for the final answer. A *recursive resolver* accepts such questions and does all the work: it asks the root servers, the TLD servers and finally the authoritative servers, following referrals until it obtains an answer, which it caches and returns to the stub. An *authoritative server* holds the actual data of a zone and answers only for that zone; for names outside its zone it either refers the resolver to a server lower in the tree or refuses.

The queries that a recursive resolver sends to authoritative servers are *iterative* (non-recursive): the RD flag is cleared, and each server answers with what it knows, either the final data or a referral to the next zone down. Fig. 1.2 shows the eight messages exchanged when a stub asks for `www.github.com` and the recursive resolver starts with an empty cache. DNSR plays the role of the recursive resolver in this figure.

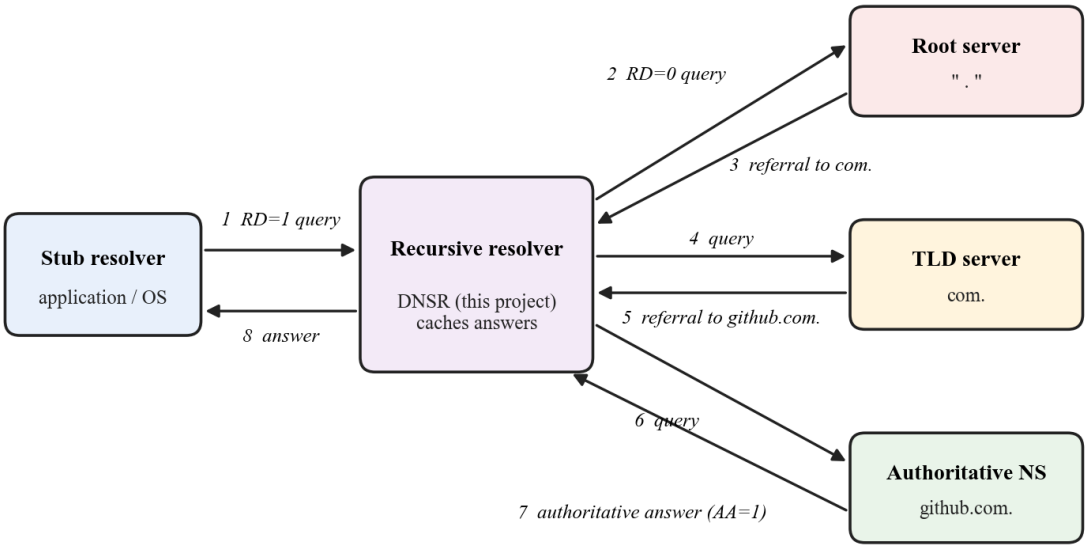


Fig. 1.2 Recursive Query from a Stub and Iterative Queries from the Resolver

1.3.3 Resource records and caching

All DNS data is stored as *resource records* (RRs). Every RR has an owner name, a type, a class (almost always IN for Internet), a time to live (TTL) in seconds and type-specific data called RDATA. Records with the same owner, type and class form an RRset. The TTL tells a resolver how long it may cache the RRset; a well-behaved resolver counts the TTL down while the record sits in the cache and returns the remaining TTL to its clients. Table 1.1 lists the record types handled by DNSR.

Table 1.1 Record Types Supported by DNSR

Type	Code	Meaning	RDATA handled by DNSR
A	1	IPv4 address of a host	4-byte address printed in dotted form
NS	2	Authoritative name server of a zone	Domain name (may be compressed)
CNAME	5	Canonical name: the owner is an alias	Domain name of the target
SOA	6	Start of authority: zone parameters	MNAME, RNAME and five 32-bit timers
PTR	12	Pointer, used for reverse lookups	Domain name
MX	15	Mail exchanger for a domain	16-bit preference and a domain name
TXT	16	Free-form text strings	One or more length-prefixed strings
AAAA	28	IPv6 address of a host	16-byte address printed in IPv6 form

Type	Code	Meaning	RDATA handled by DNSR
OPT	41	EDNS(0) pseudo-record (not data)	Encoded in queries (with the DO bit for DNSSEC); client buffer size read in server mode
DS	43	Delegation signer: hash of a child zone's key	Key tag, algorithm, digest type, digest
RRSIG	46	Signature over an RRset	All signature fields; used for validation
DNSKEY	48	Public key of a zone	Flags, protocol, algorithm, key; key tag computed

1.3.4 Why DNS is a target

Classic DNS runs over UDP and carries no authentication. A reply is accepted if it arrives at the right port, carries the right 16-bit transaction ID and answers the right question. An attacker who is not on the network path between the resolver and the server (an *off-path* attacker) cannot see the query, but can send many forged replies that guess these values. If one forged reply arrives before the genuine one and is accepted, its records enter the cache and are served to every client of the resolver until they expire. The defences against this are discussed in detail in Chapter 2 and their implementation in Chapter 4. Cryptographic protection is available through DNSSEC [13]: zones sign their data and a validating resolver checks the signatures. DNSR validates positive answers in this way, as described in Section 4.1.9; proofs that a name does not exist (NSEC/NSEC3) are left as future work.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

This chapter reviews the literature on which the project builds, identifies the gap that it addresses, states the problem formally and sets out the goals and scope of the work.

2.1 Literature Review

2.1.1 Foundations of the Domain Name System

The DNS was designed by Mockapetris and standardised in RFC 1034 and RFC 1035 [1], [2]. RFC 1034 describes the concepts: the hierarchical namespace, zones and delegation, the roles of resolvers and name servers, and the algorithm a resolver follows. RFC 1035 specifies the implementation, in particular the binary message format with its fixed 12-byte header and four variable sections, the encoding of names as sequences of length-prefixed labels and the compression scheme that replaces repeated name suffixes with 14-bit pointers. Mockapetris and Dunlap [3] later described the design goals behind the system, namely scalability, distributed administration and aggressive caching, and the lessons learned from its first years of operation.

RFC 2181 [4] clarified several ambiguous parts of the original specification. Most important for this project is its section on the *ranking of data*: data from the answer section of an authoritative reply is trusted more than data from the authority or additional section, and a resolver must not let lower-ranked data overwrite higher-ranked data. This ranking is the formal basis of the bailiwick rule used by modern resolvers. The current terminology of DNS, including the terms *in-bailiwick*, *glue* and *referral*, is collected in RFC 8499 [26].

2.1.2 Protocol extensions and transport

RFC 1035 limited UDP messages to 512 bytes. As DNS answers grew, especially with IPv6 addresses, long TXT records and DNSSEC signatures, this limit became too small. The Extension Mechanisms for DNS, EDNS(0), defined in RFC 6891 [6], allow a client to place an OPT pseudo-record in the additional section to advertise a larger UDP buffer. Large UDP packets, however, are fragmented by IP and fragments are often dropped by firewalls. The DNS Flag Day 2020 initiative [20] therefore recommended a buffer size of 1232 bytes, which fits in the minimum IPv6

path MTU without fragmentation. If a reply is still too large, the server sets the TC (truncation) bit and the client must repeat the query over TCP, where each message is preceded by a two-byte length field. RFC 7766 [7] made TCP support mandatory for all DNS implementations.

Negative answers come in two forms. NXDOMAIN (name error) means that the queried name does not exist at all; NODATA means that the name exists but has no records of the requested type. Both are recognised by the SOA record that the authoritative server places in the authority section, and RFC 2308 [5] specifies how they may be cached. RFC 8020 [27] adds that an NXDOMAIN for a name implies that no names exist below it.

2.1.3 Cache poisoning and its defences

Because classic DNS is unauthenticated, a resolver can be tricked into caching forged records. Early attacks simply guessed the transaction ID, which many implementations generated sequentially. Kaminsky [9] showed in 2008 that the attack becomes far more powerful when the attacker forces the resolver to ask for many different non-existent names (so that no cached answer stops the race) and places a forged NS record and forged glue in each fake reply, so that one success hijacks a whole domain. Fig. 2.1 illustrates the attack.

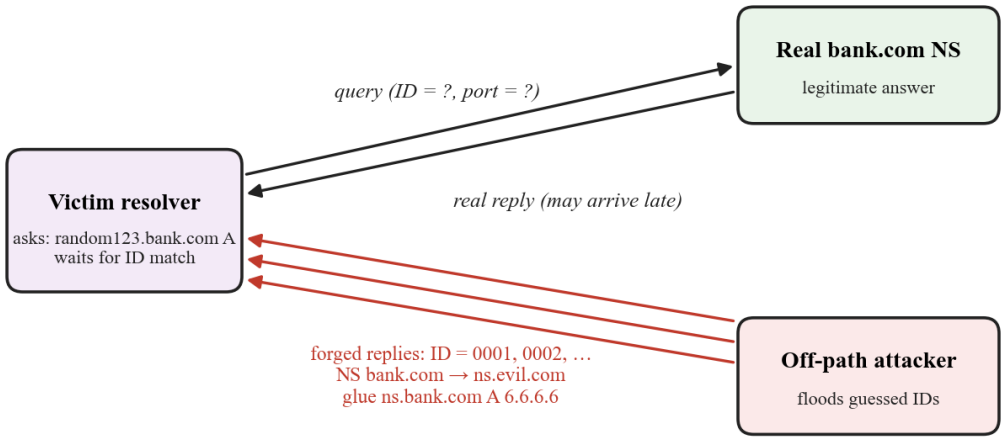


Fig. 2.1 Kaminsky-Style Cache Poisoning by an Off-Path Attacker

RFC 5452 [8] summarises the defences that followed. A resolver must use an unpredictable transaction ID and an unpredictable UDP source port, must accept a reply

only if its source address, ID and question exactly match an outstanding query, and should treat out-of-zone records with suspicion. Dagon et al. [28] proposed adding further entropy by randomising the upper and lower case of letters in the query name (the 0x20 technique), and DNS Cookies (RFC 7873) [29] add a lightweight shared secret between client and server. Son and Shmatikov [10] systematically examined how real resolvers accept records and showed that weak or missing bailiwick checks allow poisoning even when IDs and ports are random. Man et al. [11] presented SAD DNS, which uses an ICMP rate-limit side channel to discover the randomised source port, demonstrating that cache poisoning remains an active research area. The broader threat analysis of DNS is given in RFC 3833 [25].

Resolvers can also be abused as attack amplifiers. The NXNSAttack [24] uses referrals that list many name servers without glue; a resolver that eagerly resolves the address of every listed name server sends a large number of queries for each query the attacker makes. This shows that the referral-following logic must limit how much work a single upstream reply can trigger.

2.1.4 Performance, caching and privacy

Jung et al. [12] studied real DNS traces and found that caching of NS records, which lets a resolver skip the root and TLD servers on later queries, is what most reduces load and latency, while the benefit of caching A records levels off quickly because of short TTLs. This result directly supports the separate delegation cache used in DNSR. On the privacy side, QNAME minimisation [14] reduces how much of the query name each upstream server sees, and DNS over TLS [15] and DNS over HTTPS [16] encrypt the path between the stub and the recursive resolver. These measures protect privacy but do not replace the integrity checks a recursive resolver must perform on the path to the authoritative servers. DNSSEC [13] adds data-origin authentication through digital signatures, but deployment is still incomplete and a validating resolver must still perform all of the checks above for unsigned zones.

2.1.5 Existing implementations

Production resolvers such as BIND 9 [18] and Unbound [17] implement the full protocol, including DNSSEC validation, but they are large C code bases with hundreds of thousands of lines, which makes them hard to study for learning or research. In the Go ecosystem, the `miekg/dns` library [19] provides message parsing and is widely

used, but it hides the wire format behind abstractions, and Go's standard `net.Resolver` is only a stub that delegates recursion to an upstream server. Online tools and `dig +trace` show the referral chain but do not cache or validate anything. Table 2.1 compares these approaches with the proposed work.

Table 2.1 Comparison of Existing DNS Resolution Approaches

Approach	Iterative from root	Hand-written wire format	Explicit security checks	Suitability for learning / research
OS stub / Go net.Resolver	No (delegates)	Hidden	Relies on upstream	Low: a black box
miekg/dns library	Only if the user builds it	No (library)	Left to the user	Medium
BIND 9 / Unbound	Yes	Yes	Yes, including DNSSEC	Low: very large code base
dig +trace / online tools	Yes (trace only)	Hidden	No caching or validation	Medium: observation only
DNSR (this project)	Yes	Yes, about 2,000 lines	ID and question match, bailiwick, DNSSEC, QNAME minimisation	High: small, traced and tested

2.2 Gap Identification

The literature and the existing tools leave the following gaps, which this project sets out to close.

- ✓ **Opacity of existing tools.** Stub resolvers and libraries such as Go's `net.Resolver` hand recursion to an upstream server, so the referral chain, glue handling and caching decisions cannot be seen.
- ✓ **Complexity of production resolvers.** BIND and Unbound are complete but very large, which makes it impractical to trace one specific behaviour, for example glue acceptance, for learning or experiments.
- ✓ **Security checks are rarely made explicit.** Bailiwick rules for referrals and glue, together with reply matching, are the main defence against Kaminsky-style poisoning, but most teaching material and simple implementations leave them out.

- ✓ **Lack of offline, repeatable testing.** Resolution behaviour depends on live Internet servers that change over time, which makes edge cases such as glueless delegation, poisoned glue and truncation hard to reproduce.
- ✓ **Library dependence hides the wire format.** Relying on a DNS library prevents a hands-on understanding of name compression, EDNS(0) and the malformed-packet risks in parsing.

2.3 Problem Statement

DNS is a critical and frequently attacked layer of Internet infrastructure, yet the way a recursive resolver walks the hierarchy, judges which records to trust and caches results is usually hidden inside large production systems or high-level libraries. There is a need for a compact, transparent and verifiable recursive resolver that implements the protocol from first principles, shows every step of resolution, and builds in the security checks that prevent cache-poisoning attacks, so that DNS behaviour and its weaknesses can be studied, demonstrated and tested in a controlled environment.

2.4 Project Goals

The goals of the project follow from the objectives in Section 1.1 and are grouped below into functional, security, quality and usability goals. Each goal is later checked against the results in Chapter 7.

Table 2.2 Project Goals and How They Are Measured

ID	Goal	Measure of success
G1	Correct wire-format codec	All supported record types parsed from live replies; compressed names decoded; malformed packets rejected without crashing
G2	Complete iterative resolution	Names resolved from the root for 15 popular domains with the same response code as a public resolver
G3	Correct special cases	CNAME, NXDOMAIN, NODATA, glueless delegation and TC to TCP fallback demonstrated both offline and live
G4	Spoofing and poisoning defences	Forged or mismatched replies ignored (random ID, port and letter case); out-of-bailiwick referrals and glue rejected in tests
G5	Caching for performance	Measurable latency reduction for repeated and sibling names
G6	Two modes of operation	CLI trace output, and a server that answers standard DNS clients
G7	Robust parser	No panic or hang under large-scale random and mutation-based fuzzing

ID	Goal	Measure of success
G8	Small and dependency-free	Standard library only; single binary; about 2,000 lines of code
G9	DNSSEC validation	Signed domains reported secure, a deliberately broken domain rejected, unsigned domains reported insecure
G10	Privacy	Upstream servers see minimised names; clients can use DNS over TLS
G11	Resilient server	Access list, rate limit, concurrency cap, bounded caches and TTL cap verified by tests

2.5 Scope of the Project

The project covers the resolver side of DNS over IPv4 transport, for the record types listed in Table 1.1, with UDP and TCP transport to authoritative servers and QNAME minimisation. It covers positive caching of answers and caching of delegations, DNSSEC validation of positive answers, and a recursive server that listens on UDP, TCP and DNS over TLS. The following items are explicitly outside the scope of this version and are discussed as future work in Chapter 8: validation of negative answers (NSEC and NSEC3 proofs), IPv6 transport to name servers (AAAA glue is parsed but not used), negative caching, caching of CNAME records, DNS over HTTPS and round-trip-time based server selection.

CHAPTER 3

TECHNICAL SPECIFICATION

This chapter specifies the requirements that DNSR must satisfy, the hardware and software environment used to build and test it, and the protocol details, message layout, flags and limits, that the implementation follows.

3.1 Functional Requirements

Table 3.1 Functional Requirements

ID	Requirement
FR1	Accept a domain name and a record type (A, AAAA, NS, MX, TXT, CNAME, SOA or PTR) from the command line; the default type is A.
FR2	Resolve the name iteratively, starting from the 13 IANA root server addresses, without using any upstream recursive resolver.
FR3	Follow referrals using NS records from the authority section and glue address records from the additional section.
FR4	When a referral has no usable glue, resolve the address of one of the listed name servers with a nested lookup (glueless delegation).
FR5	Follow CNAME aliases by restarting resolution for the target name and return the CNAME together with the final records.
FR6	Distinguish NXDOMAIN (name does not exist) from NODATA (name exists, no records of the type) and return the SOA from the authority section.
FR7	Retry a query over TCP when a UDP reply has the TC bit set.
FR8	Cache positive answers with a TTL that counts down, and cache delegations so that later queries can start below the root.
FR9	Print a human-readable trace of every step, with an option to suppress it and print only the answer, and an option to disable colour.
FR10	Run as a recursive DNS server on a configurable address and answer standard DNS queries from clients over UDP and TCP.
FR11	With -dnssec, validate every positive answer from the root trust anchor and report it as secure, insecure or bogus; in server mode set AD on secure answers and return SERVFAIL for bogus ones.
FR12	Minimise query names by default (RFC 9156) and allow minimisation to be switched off with -nomin.
FR13	Optionally serve clients over DNS over TLS (RFC 7858), TLS 1.3 only, with a supplied or self-signed certificate.
FR14	Answer only clients whose address is in a configurable access list (loopback by default) and refuse all others.

3.2 Security Requirements

Table 3.2 Security Requirements

ID	Requirement	Threat addressed
SR1	Use a 16-bit transaction ID from the cryptographic random source for every query	Blind ID guessing
SR2	Accept a reply only if ID, QR bit, question name and question type match the query, over UDP and TCP; on UDP keep waiting until the deadline	Forged and stray replies
SR3	Accept a referral only if the child zone is strictly below the current zone and contains the queried name	Redirection to an unrelated zone
SR4	Accept a glue record only if it belongs to one of the referred name servers and lies inside the zone of the server that sent it	Kaminsky-style glue poisoning
SR5	Accept only answer records whose owner is the queried name	Injection of unrelated records into the cache
SR6	Bound every read in the parser and limit compression-pointer jumps	Crashes and hangs from malformed packets
SR7	Limit hops, nesting depth, servers tried per step and time per query	Resolution loops and resource exhaustion
SR8	Listen on the loopback address by default in server mode	Exposure as an open resolver
SR9	Cap every TTL at one day; bound each cache to 10,000 entries; let delegations expire	Long-lived poisoning, memory exhaustion
SR10	Resolve at most three name-server addresses for one glueless referral	Query amplification (NXNSAttack)
SR11	Rate-limit each client and cap queries in flight in server mode	Flooding and reflection abuse
SR12	Validate signatures from the IANA root anchors (KSK-2017 and KSK-2024)	Forged answers in signed zones, including on-path forgery
SR13	Randomise the letter case of every query name (0x20) and require the reply to repeat it exactly	Blind spoofing (more entropy)
SR14	Serve only clients in the access list; accept only TLS 1.3 for DNS over TLS	Open-resolver abuse; weak TLS versions

3.3 Non-Functional Requirements

- ✓ **Portability:** the resolver must build into a single static binary for Windows, Linux and macOS with the standard Go tool chain.
- ✓ **Dependencies:** only the Go standard library may be used (`net`, `encoding/binary`, `sync`, `flag`, `math/rand` and similar).

- ✓ **Performance:** a cold-cache resolution should complete within a few seconds; cached answers should be returned without network traffic.
- ✓ **Concurrency safety:** in server mode, many queries may be resolved at the same time, so the shared caches must be protected against data races.
- ✓ **Readability:** the code should stay small enough (about 1,000 lines) to be read in one sitting, with one file per layer.
- ✓ **Testability:** all resolution logic must be testable offline without access to the Internet.

3.4 Hardware and Software Specification

Table 3.3 Development and Test Environment

Item	Specification
Processor	13th Gen Intel Core i7-13650HX (14 cores, 20 threads)
Operating system	Windows 11 Home Single Language (64-bit)
Language and tool chain	Go (module declares go 1.25; built and tested with go1.27.0 windows/amd64)
Libraries used by DNSR	Go standard library only
Testing	Go `testing` package: unit and integration tests, benchmarks and native fuzzing
Live validation client	dnspython 2.7.0 [31] on Python 3.13 (independent DNS implementation)
Reference resolver for comparison	Google Public DNS (8.8.8.8)
Network	Outbound UDP and TCP port 53 to the Internet
Minimum to run	Any 64-bit machine able to run Go binaries; less than 10 MB of memory in normal use

3.5 DNS Message Format

Every DNS message, query or reply, has the same layout, shown in Fig. 3.1. The 12-byte header is followed by four sections. The question section normally holds exactly one question. The answer section holds records that answer the question, the authority section holds NS records (for a referral) or an SOA record (for a negative answer), and the additional section holds helpful extra records such as glue addresses and the EDNS(0) OPT pseudo-record.

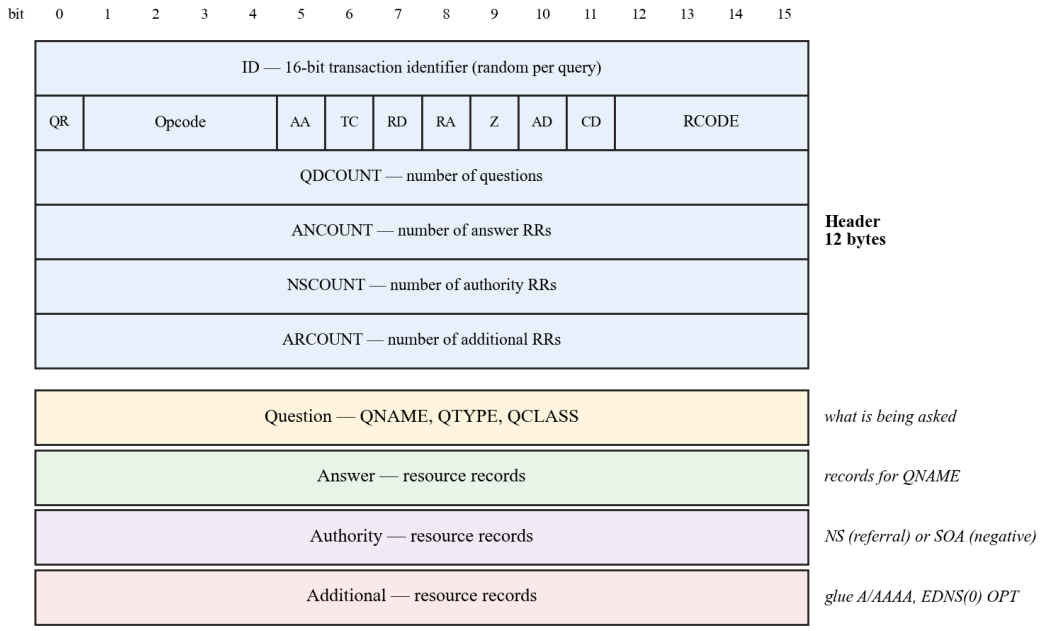


Fig. 3.1 Layout of a DNS Message and Its 12-Byte Header

The meaning of each header flag and how DNSR uses it is summarised in Table 3.4. All multi-byte integers in a DNS message are in network byte order (big-endian), which DNSR reads and writes with Go's `encoding/binary` package.

Table 3.4 Header Fields and Their Use in DNSR

Field	Bits	Meaning	Use in DNSR
ID	16	Transaction identifier copied into the reply	Random per query; must match in the reply
QR	1	0 = query, 1 = response	Replies must have QR = 1; server drops packets with QR = 1
Opcode	4	Kind of query (0 = standard)	Always 0
AA	1	Authoritative answer	AA = 1 with no answer records means NODATA
TC	1	Message was truncated	Triggers retry over TCP
RD	1	Recursion desired	0 in outgoing queries; copied back in server replies
RA	1	Recursion available	Set to 1 in server-mode replies
Z	1	Reserved	0
AD	1	Authenticated data (DNSSEC)	Set by the server on answers validated as secure
CD	1	Checking disabled (DNSSEC)	Parsed; not used
RCODE	4	Response code	NOERROR, NXDOMAIN, SERVFAIL and others handled

Field	Bits	Meaning	Use in DNSR
QDCOUNT ... ARCOUNT	4 x 16	Number of entries in each section	Drive the parsing loops; QDCOUNT must be 1

3.5.1 Domain-name encoding and compression

A name is encoded as a sequence of labels, each preceded by a one-byte length, and terminated by a zero byte that stands for the root. Because the same suffix appears many times in a reply (for example `github.com` in the question, the answer and the authority records), RFC 1035 allows the tail of a name to be replaced by a two-byte pointer whose two top bits are set to 1 and whose remaining 14 bits give the offset of an earlier occurrence. Fig. 3.2 shows a typical case: the owner name of the answer record is just the pointer `C0 0C`, which points back to offset 12 where the question name starts.

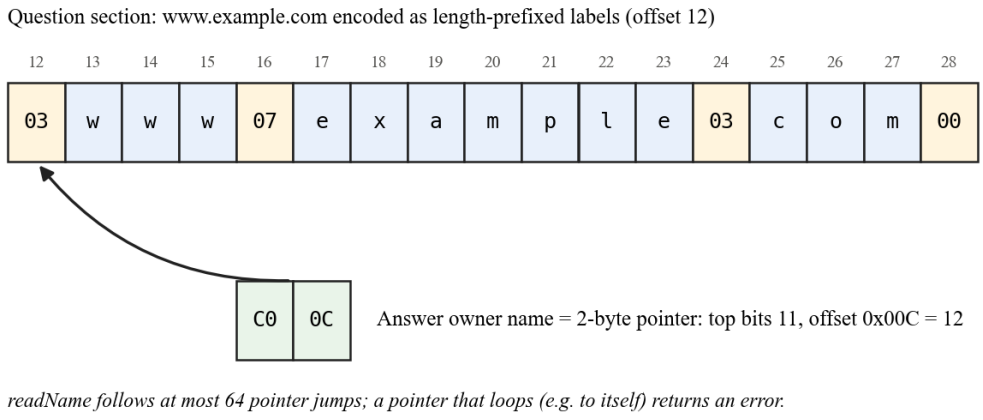


Fig. 3.2 Name Compression: a Pointer to the Question Name

Pointers are a well-known attack surface. A pointer that points to itself, or two pointers that point to each other, would make a naive parser loop forever. DNSR therefore counts the jumps and returns an error after 64 of them, and it checks every offset against the length of the message before reading it.

3.5.2 EDNS(0) and transport

Every query that DNSR sends carries an OPT pseudo-record in the additional section. Its owner name is the root, its type is 41, the CLASS field is reused to advertise the UDP payload size (1232 bytes) and the TTL field carries extended flags. When validation is enabled the DO (DNSSEC OK) bit is set, which asks servers to include

RRSIG records in their replies. If a reply still does not fit, the server sets TC and DNSR repeats the identical query over TCP. Over TCP each message is preceded by its length as a 16-bit integer, as defined in RFC 1035 Section 4.2.2.

3.6 Operating Limits and Constants

Table 3.5 lists the constants that bound the work DNSR does for one query. They protect against loops in misconfigured or malicious delegations and keep the worst-case time of a query predictable.

Table 3.5 Resolver Constants

Constant	Value	Purpose
Timeout	2 s per exchange	Deadline for each UDP or TCP exchange with one server
MaxHops	40	Maximum number of referral steps in one resolution
Nesting depth	8	Maximum depth of CNAME and glueless sub-resolutions
Servers per step	4	At most four randomly chosen servers are tried per step
Compression jumps	64	Maximum pointer jumps when decoding one name
EDNS(0) buffer	1232 bytes	Advertised UDP payload size (DNS Flag Day 2020)
UDP receive buffer	4096 bytes	Buffer used to read replies
Label length	1 to 63 bytes	Enforced when encoding names
Name length	255 bytes	Longer names in a reply are rejected
Maximum TTL	86,400 s (1 day)	Upper bound for every cached or returned record
Cache size	10,000 entries	Per cache: answers, delegations, DNSSEC keys
Glueless lookups	3 per referral	Name-server addresses resolved for one delegation
Minimisation	up to 10 labels	Longer names are sent in full
Server listen address	127.0.0.1:8053 (default)	UDP and TCP; loopback only unless changed with -addr
Server limits	256 in flight; 20 q/s, burst 40	Concurrent queries; per-client token bucket
Access list	127.0.0.0/8 and ::1 (default)	Clients outside the list receive REFUSED
0x20 fall-back memory	1 hour	Servers that do not preserve case are asked without 0x20
TLS version	1.3 only	Minimum version for DNS over TLS
Server receive buffer	1500 bytes	Maximum UDP query size accepted

Constant	Value	Purpose
TCP / TLS idle timeout	10 s	Idle client connections are closed

3.7 Command-Line Interface Specification

DNSR is a single executable with two forms of invocation. Table 3.6 lists the flags. Several names may be given in one invocation; they are resolved in order and share the same caches, which makes the effect of caching easy to observe.

Table 3.6 Command-Line Usage and Flags

Command or flag	Description
dnsr [flags] <name> [name...]	Resolve each name from the root down and print the trace and the answer
-t TYPE	Record type: A, AAAA, NS, MX, TXT, CNAME, SOA, PTR, DS, DNSKEY or RRSIG (default A)
-dnssec	Validate answers with DNSSEC and print secure, insecure or bogus
-nomin	Send the full name to every server (disable QNAME minimisation)
-no0x20	Do not randomise the letter case of query names
-q	Quiet mode: print only the final answer, not the trace
-nocolor	Disable ANSI colours (colour is also disabled automatically when output is not a terminal)
dnsr serve [-addr host:port]	Run as a recursive DNS server on UDP and TCP (default 127.0.0.1:8053)
serve -dot host:port	Also serve DNS over TLS; with -cert and -key, or -selfsigned for a generated certificate
serve -allow list	Client networks allowed to query, e.g. 127.0.0.0/8,192.168.0.0/16 (default loopback only)
serve -rate N -burst N	Per-client rate limit (default 20 queries per second, burst 40)
serve -dnssec, -nomin, -trace	Validation, minimisation switch and full trace for every query
Exit status	0 if every name resolved without error, 1 if any resolution failed, 2 on a usage error

CHAPTER 4

DESIGN APPROACH AND DETAILS

This chapter describes how DNSR was designed: the method followed, the architecture and its modules, the data structures, the resolution algorithm, the security mechanisms, caching and transport, the server mode and the test environment. It then lists the codes and standards that the design follows and discusses the constraints, alternatives and trade-offs that shaped it.

4.1 Design Approach / Materials and Methods

4.1.1 Development methodology

The project followed an incremental, test-driven methodology. Each protocol feature was first specified from the relevant RFC, then implemented in isolation, then verified by a test before the next layer was built on top of it. Building from the bottom up meant that the wire-format codec was reliable before the resolver used it, and that each resolver feature (referrals, glue, CNAMEs, negative answers, truncation) had a dedicated test in the fake DNS hierarchy before it was tried against the live Internet. The steps were as follows.

- ✓ **Requirement analysis and protocol study:** RFC 1034/1035, RFC 2181 (data ranking), RFC 6891 (EDNS(0)), RFC 7766 (TCP) and RFC 5452 (forgery resilience) were studied to fix the functional and security requirements of Chapter 3.
- ✓ **Wire-format codec:** message encoding and decoding were written by hand, including compression-pointer handling with loop protection and bounds checks on every read, so that malformed packets return errors instead of crashing the parser.
- ✓ **Iterative resolution engine:** the referral-following loop was built from the root hints, followed by CNAME chasing, NXDOMAIN and NODATA detection, glueless delegation and the hop and depth limits.
- ✓ **Security hardening:** random query IDs, strict reply matching and the two bailiwick checks were added and each was covered by a test with a deliberately malicious server.

- ✓ **Caching and transport:** the answer cache, the delegation cache, randomised server selection with fail-over and the UDP to TCP fallback were added.
- ✓ **Server mode:** the resolver was exposed as a concurrent DNS server over UDP, TCP and DNS over TLS.
- ✓ **DNSSEC and privacy:** validation from the root trust anchors, QNAME minimisation, and the hardening found in a code review (TTL cap, bounded caches, glueless lookup limit, TCP reply check, rate limiting, cryptographic query IDs, 0x20 case randomisation, a client access list and TLS 1.3 only) were added, each with tests against signed and hostile zones.
- ✓ **Validation:** the resolver was tested offline, measured live against 15 popular domains, compared with a public resolver, fuzzed and benchmarked (Chapter 7).

4.1.2 System architecture

DNSR follows a layered architecture, shown in Fig. 4.1. The interface layer offers two entry points: the command-line client in `main.go` and the recursive server in `server.go`, which listens on UDP, TCP and optionally DNS over TLS. Both pass requests to the resolution engine in `resolver.go`, which holds the iterative algorithm with QNAME minimisation, the security checks and the caches. When validation is enabled, `dnssec.go` checks every answer against a chain of trust that starts at the built-in root trust anchors. The engine talks to the Internet through a small transport layer that sends UDP queries with an EDNS(0) buffer of 1232 bytes and switches to TCP when the TC bit is set. All messages are built and parsed by the wire codec in `wire.go`, which implements the RFC 1035 binary format by hand. Each layer depends only on the layer below it, which keeps the code easy to test and to read.

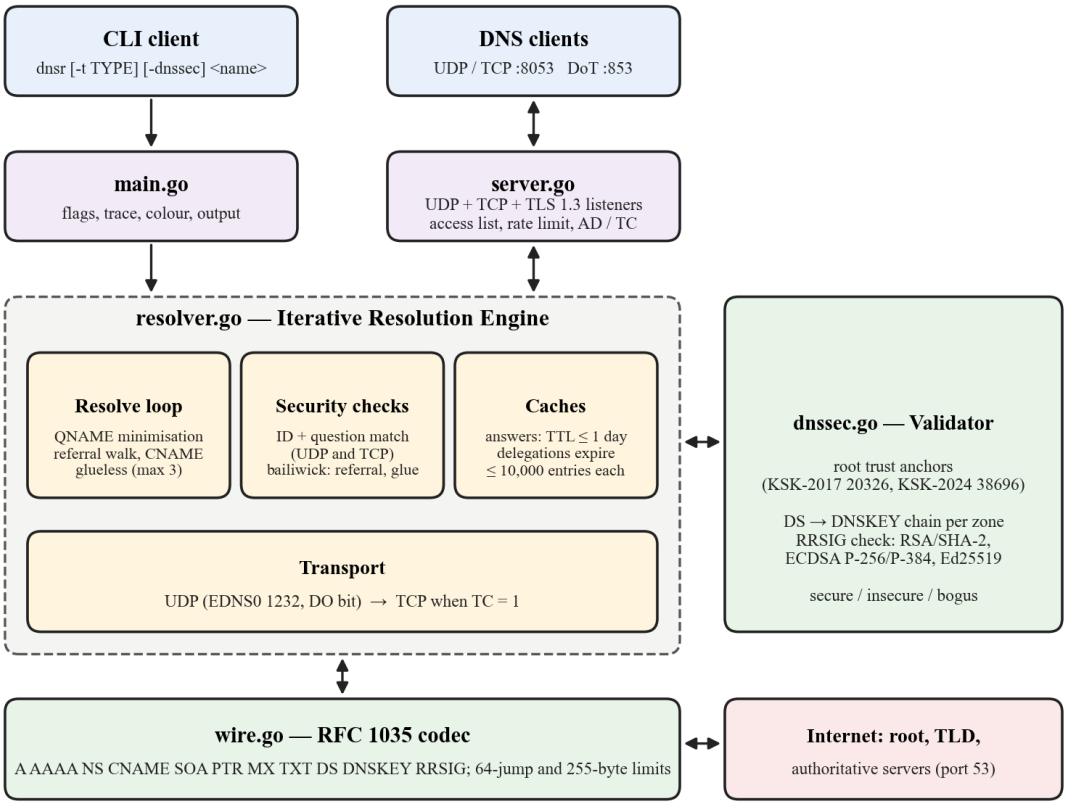


Fig. 4.1 System Architecture of DNSR

Table 4.1 describes the responsibility of each source file. The whole program, excluding tests, is 2,205 lines of Go; the two test files add 900 lines.

Table 4.1 Module Description

Module (lines)	Responsibility
wire.go (363)	Defines the Header, Question, RR and Message types; encodes names and queries (RD = 0, EDNS(0) OPT with the DO bit for DNSSEC); decodes compressed names with a 64-jump and 255-byte limit; parses A, AAAA, NS, CNAME, PTR, MX, SOA, TXT, DS, DNSKEY and RRSIG data; reads the client's EDNS settings.
resolver.go (719)	Root hints; iterative resolution loop with QNAME minimisation; 0x20 case randomisation with per-server fall-back; answer cache and delegation cache with TTL cap, expiry and size bound; referral and glue processing with bailiwick checks; glueless lookups (at most three); CNAME restart; random server selection; UDP and TCP exchange with reply matching; trace output with control characters escaped.
dnssec.go (527)	Root trust anchors; RRSIG, DNSKEY and DS parsing; key tags; canonical form; signature checks for RSA/SHA-256, RSA/SHA-512, ECDSA P-256 and P-384 and Ed25519; chain of trust per zone with a key cache; secure, insecure or bogus result for each answer.

Module (lines)	Responsibility
server.go (459)	UDP, TCP and DNS over TLS 1.3 listeners; client access list; concurrency cap; per-client token-bucket rate limit; response building with AD flag, SERVFAIL for bogus answers and truncation for small client buffers; self-signed certificate generation.
main.go (137)	Parses command-line flags, selects CLI or server mode, enables colour only on a terminal, prints the trace, the security status and the answer, and sets the exit status.
resolver_test.go (468) and security_test.go (432)	Fake DNS hierarchy on 127.0.0.10 to 127.0.0.15 with three DNSSEC-signed zones, and 32 deterministic tests including hostile servers, forged replies and tampered signatures.

4.1.3 Data structures

The design keeps the data model close to the RFC so that each structure can be checked against the specification. A parsed message is a `Message` holding a `Header`, a slice of `Question` values and three slices of `RR` values. Each `RR` keeps both a human-readable `Data` string, used for display and for the server's re-encoding, and the `Raw` RDATA bytes, used for types that DNSR does not interpret. Table 4.2 lists the main structures.

Table 4.2 Main Data Structures

Structure	Fields	Purpose
Header	ID, QR, Opcode, AA, TC, RD, RA, Rcode, QDCount, ANCount, NSCount, ARCount	Decoded 12-byte message header
Question	Name, Type, Class	One entry of the question section
RR	Name, Type, Class, TTL, Data, Raw	One resource record
Message	Header, Questions, Answers, Authority, Additional	Whole parsed message
nameserver	Name, IP	A server that can be queried
cacheEntry	rrs, expires	Cached RRset and its absolute expiry time
Resolver	Timeout, MaxHops, Trace, Port, Roots, cache, zoneNS, mutex	Resolver configuration and shared state
Result	Rcode, Answers, Authority, Sigs, Hops, Elapsed, Security	Outcome of one resolution, with its DNSSEC status
rrsig, dnskey, DSRecord	Signature fields; key flags, algorithm and tag; DS digest	Parsed DNSSEC records
keyEntry	keys, sec, why, expires	Cached result of validating one zone's keys

Structure	Fields	Purpose
Server, bucket	MaxInflight, Rate, Burst; tokens, last	Server limits and per-client token bucket

The caches are Go maps protected by one mutex. The answer cache maps the key `name|TYPE` (for example `github.com.|A`) to a `cacheEntry` holding the records, their signatures and an expiry time. The delegation cache `zoneNS` maps a zone name such as `github.com.` to the name servers (with addresses) that serve it and an expiry time taken from the NS records. The key cache maps a zone to its validated DNSSEC keys. Each cache is limited to 10,000 entries; when it is full, expired entries are removed first and then arbitrary ones. Root hints are not stored in the map; they are the fallback when no deeper zone is known.

4.1.4 Resolution algorithm

The heart of DNSR is the function `resolve`. Its basic control flow is shown in Fig. 4.2 and its full logic, including QNAME minimisation, in the pseudo-code that follows. Resolution starts by checking the answer cache. On a miss, the resolver picks the deepest zone it already knows from the delegation cache, or the root if it knows none, and enters a loop. In each iteration it sends a non-recursive query to up to four randomly chosen servers of the current zone and classifies the first valid reply. While the queried name is still more than one label below the current zone, the query is a minimised *probe*: an NS query for the name one label below the zone.

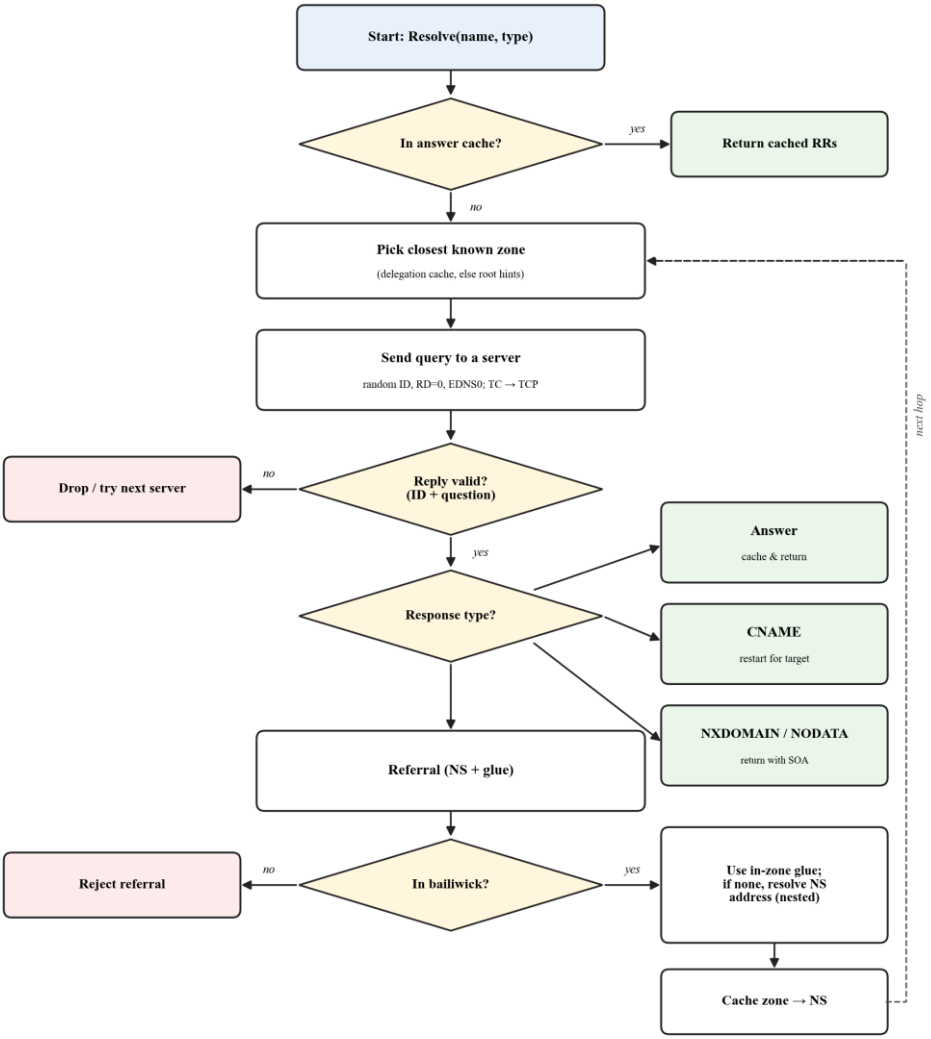


Fig. 4.2 Iterative Resolution Flowchart

```
resolve(name, type, depth, hops):
    if depth > 8:               return error "resolution loop / too deep"
    if cache has (name, type):  return cached records and signatures
    zone, servers := closest known zone (for DS: of the parent of name)
    probe := name one label below zone                # QNAME minimisation
    while hops < MaxHops (40):
        q := (probe, NS) if minimising and probe != name, else (name, type)
        reply := ask up to 4 random servers of zone     # UDP, TCP if TC=1
        if q is a probe and reply is an error or has other records:
            stop minimising; continue                   # ask the full name
        if q is the full name:
            if rcode == NXDOMAIN:  return NXDOMAIN + SOA
            if rcode != NOERROR:   drop server; continue (or return rcode)
            if answers of type for name:  cap TTLs; cache; return them
            if a CNAME for name:
                sub := resolve(target, type, depth+1, hops+1)
                return [cname] + sub.answers
            if reply.AA:           return NODATA + SOA
        ns, child := referral (authority NS, or answer NS for a probe)
        if ns is empty:
            if q is a probe and reply.AA: probe := one label deeper; continue
            return error "lame response"
        if child not strictly below zone or name not under child:
            return error "out-of-bailiwick referral"
```

```
        next := glue A records for ns that lie inside zone
        if next is empty:                              # glueless delegation
            for at most 3 ns targets in random order:
                addr := resolve(target, A, depth+1, 0)
                if addr found: next := addr; break
        cache delegation child -> next, expiring with NS TTL (max 1 day)
        zone, servers := child, next; probe := one label below child
    return error "resolution loop / too deep"
```

Five properties of the algorithm deserve attention. First, every exit from the loop is explicit, and two independent counters (hops and depth) guarantee termination even when servers misbehave. Second, a CNAME is never trusted together with records for its target that arrive in the same packet: resolution of the target restarts from the cache and the delegation tree, so a server can only vouch for names in its own zone. Third, a referral is only followed after the bailiwick check, so the delegation cache can only ever move *down* the tree towards the queried name. Fourth, the nested resolutions for CNAME targets and glueless name servers reuse the same function, so they inherit the same checks and limits. Fifth, minimisation never weakens correctness: any unexpected reply to a probe makes the resolver fall back to asking for the full name.

4.1.5 Example: resolving www.github.com

Fig. 4.3 shows the messages exchanged in a real resolution of `www.github.com` captured during testing. The root server is asked only for `com. NS` and refers the resolver to the 13 `.com` servers; one of them is asked only for `github.com. NS` and refers it to GitHub's eight name servers hosted by Amazon Route 53; the GitHub server receives the full question and answers with a CNAME to `github.com`. The resolver then restarts for `github.com`, but because the delegation for `github.com.` is now cached it goes straight to GitHub's name server, which returns the address. The complete resolution took four network round trips and 222 ms.

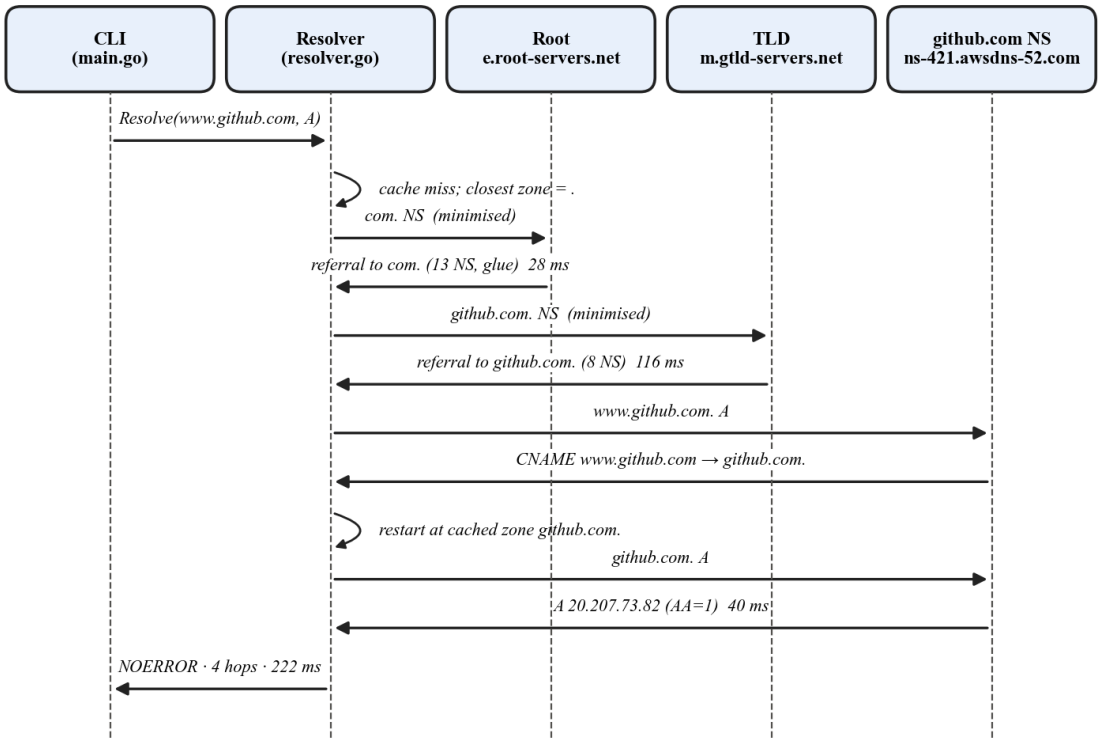


Fig. 4.3 Sequence of Messages for a Cold-Cache Lookup of www.github.com

4.1.6 Wire-format codec design

The codec is written as a set of small functions, each responsible for one element of the format. Encoding is simple because DNSR only encodes queries and, in server mode, responses without compression. `encodeName` splits a name at the dots, rejects empty labels and labels longer than 63 bytes, and writes each label with its length byte followed by the terminating zero. `BuildQuery` writes the header (random ID, all flags zero, QDCOUNT = 1, ARCOUNT = 1), the question and the OPT record, setting the DO bit when DNSSEC validation is on.

Decoding is where the risks lie, because the input comes from the network and may be hostile. The decoder follows three rules throughout. (1) Before any read, the offset plus the length of the read is compared with the length of the message; if it would run past the end the function returns `errShort`. (2) Lengths inside the packet (RDLENGTH, label lengths, TXT string lengths) are never trusted beyond the message boundary. (3) Loops are bounded: the number of records read is bounded by the header counts and the packet length, the number of compression jumps by 64, and the length of a decoded name by 255 bytes. `readName` additionally rejects the reserved label types

0x40 and 0x80. Names inside RDATA (NS, CNAME, PTR, MX and SOA) are decoded with the same `readName` function because they too may be compressed. DS, DNSKEY and RRSIG records are kept as raw bytes for validation and also decoded into their usual text form for display.

4.1.7 Security design

The security design starts from an explicit threat model, given in Table 4.3. Without DNSSEC, DNSR defends fully against the off-path attacker and against a malicious or compromised authoritative server trying to affect names outside its own zone. With DNSSEC validation enabled it also detects forged or altered answers in signed zones, even when the attacker is on the network path; for unsigned zones no cryptographic protection is possible.

Table 4.3 Threat Model

Adversary	Capabilities	Goal	DNSR defence
Off-path spoofer	Sends forged UDP packets with any source address; cannot see queries	Insert forged records into the cache	Cryptographic random ID, random source port and 0x20 letter case; full reply matching on UDP and TCP; answer filtering
Malicious authoritative server	Controls the replies for its own zone	Poison names in other zones	Bailiwick checks on referrals and glue; owner-name filtering; CNAME restart
On-path attacker	Sees and modifies traffic	Forge any answer	DNSSEC validation for signed zones (bogus → SERVFAIL); unsigned zones remain exposed
Malformed-packet sender	Sends arbitrary bytes	Crash or hang the resolver	Bounds checks; pointer-jump and name-length limits; Go memory safety
Hostile or misconfigured zone	Loops, many glueless NS, huge TTLs	Exhaust or amplify the resolver; persist poison	Hop, depth, server and time limits; at most 3 glueless lookups; TTL cap; bounded caches
Abusive client of the server	Floods queries, spoofs source addresses	Denial of service; reflection; open-resolver use	Access list (loopback by default); per-client rate limit; 256 queries in flight; TC for large UDP replies
Network observer	Reads queries in transit	Learn which names users visit	QNAME minimisation upstream; DNS over TLS from clients

Defence 1: unpredictable queries and strict reply matching

For each query DNSR draws a fresh 16-bit ID from the operating system's cryptographic random source (`crypto/rand`), so that no state of a pseudo-random generator can be learned and predicted, and sends it from a new UDP socket, for which the operating system chooses a random ephemeral source port. The socket is *connected* to the server's address, so the kernel discards datagrams from any other address or port. When a datagram arrives, DNSR accepts it only if (a) the ID equals the one sent, (b) the QR bit is set, (c) there is exactly one question and (d) its name (compared case-insensitively) and type equal those of the query. A datagram that fails any check is ignored and DNSR keeps reading until the 2-second deadline. Ignoring, rather than failing, matters: if a forged packet caused the exchange to fail, an attacker could block resolution simply by sending garbage. The same check (`matchesQuery`) is applied to replies received over TCP.

The benefit of this randomness can be quantified. If an attacker manages to deliver k forged replies before the genuine reply arrives, and each forged reply guesses uniformly among N equally likely combinations of ID and port, the probability that at least one forgery is accepted is given by Eqn. 4.1.

$$P = 1 - \left(1 - \frac{1}{N}\right)^{k} \approx \frac{k}{N} \tag{4.1}$$

With only a random ID, $N = 2^{16} = 65{,}536$, so about 45,000 forged packets give an even chance of success, which is achievable within one query timeout on a fast network. With a random source port in addition, N is multiplied by the size of the ephemeral port range (Eqn. 4.2). On Windows the dynamic range is 49152 to 65535 [32], that is $R = 16{,}384$ ports, which raises the effort by four orders of magnitude, to about 7.4×10^{8} packets for a 50% chance.

$$N = 2^{16} \times R_{port} = 65536 \times 16384 \approx 1.07 \times 10^{9} \tag{4.2}$$

DNSR adds a third source of entropy, the 0x20 technique of Dagon et al. [28]. DNS names are compared case-insensitively, but almost all servers copy the question back exactly as they received it. DNSR therefore sets every letter of the query name to upper or lower case at random (for example `wWw.ExAmPLe.cOm`) and accepts a reply only if its question repeats exactly that pattern. A name with ℓ letters multiplies the guess space by 2^{ℓ} (Eqn. 4.3); for `www.example.com`, with 13 letters, that is a further

factor of 8,192, which pushes the 50% point to about 6×10^{12} forged packets. Section 7.9 plots all three curves.

$$N = 2^{16} \times R_{port} \times 2^{\ell} \tag{4.3}$$

A few servers do not preserve case. When a reply matches in every respect except the letter case, DNSR records that server, asks it again without 0x20, and does not use 0x20 with it for an hour, so such domains still resolve. Because the attacker would already have had to guess the ID and port to trigger this, the fall-back gives no advantage. Accepted replies are converted to lower case, so the random case never reaches the caches or the output.

Defence 2: bailiwick checks on referrals and glue

Even a genuine reply from a real server must not be allowed to affect names that the server is not responsible for. DNSR applies two bailiwick rules, illustrated in Fig. 4.4. For a referral, the child zone named by the NS records must lie strictly below the zone of the server that sent it, and the queried name must lie at or below the child zone. A `.com` server can therefore delegate `example.com.` when asked for `www.example.com`, but it cannot delegate `net.` or re-delegate `com.` itself. For glue, an address record in the additional section is used only if its owner is one of the name servers listed in the referral *and* lies inside the zone of the server that sent it. A `.com` server may supply the address of `ns1.example.com`, but an address it supplies for `ns.dnshost.org` is ignored; DNSR resolves that name separately from the `.org` hierarchy. This is precisely the check that defeats Kaminsky's forged-glue technique.

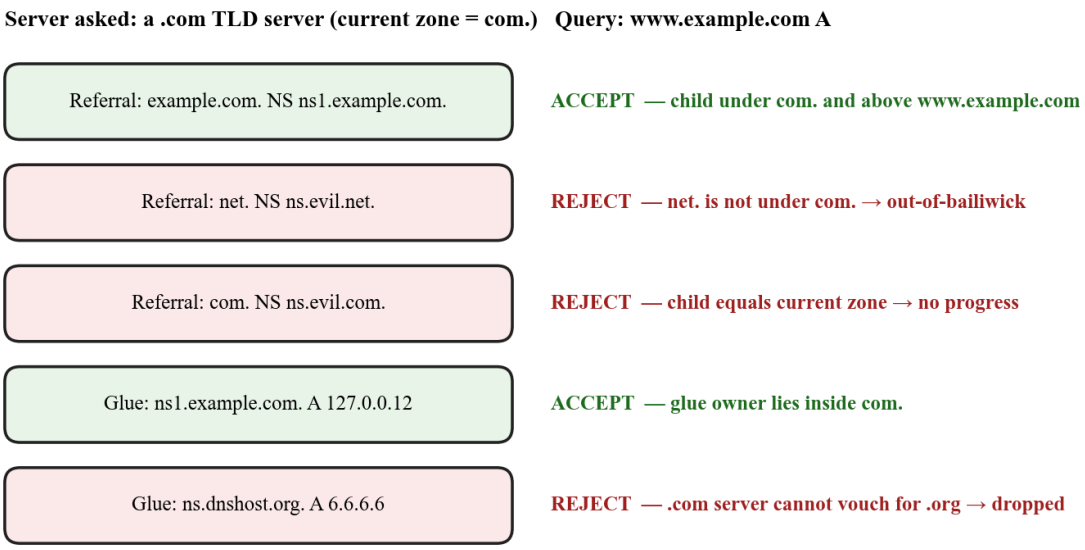


Fig. 4.4 Bailiwick Decisions for Replies from a .com Server

Defence 3: answer filtering and CNAME restart

Only records in the answer section whose owner name equals the queried name are considered; any other record is ignored and never cached. When the answer is a CNAME, DNSR follows it by starting a new resolution for the target rather than accepting records for the target from the same packet. Additional and authority records from a reply are never inserted into the answer cache.

Defence 4: safe parsing and resource limits

Go is memory-safe, so an out-of-range read causes a panic instead of silent memory corruption, but a panic would still crash the resolver. DNSR's parser therefore checks every length before reading, as described in Section 4.1.6, so that malformed input produces an error value. The hop limit (40), depth limit (8), four servers per step and the two-second deadline bound the amount of work and time that any reply can cause. A glueless referral can trigger at most three nested address lookups, which caps the amplification exploited by the NXNSAttack [24]. Every TTL is capped at one day and each cache is limited to 10,000 entries, so a successful poisoning cannot persist and a flood of unique names cannot exhaust memory. Names printed in the trace have control characters escaped, so a hostile name cannot inject terminal escape sequences. Chapter 7 reports the results of fuzzing the parser with more than nine million inputs.

Defence 5: DNSSEC validation

When validation is enabled, every positive answer must carry a valid signature from a key that is linked to the root trust anchor by an unbroken chain of DS and DNSKEY records. A forged answer, an altered record or a broken chain makes the answer bogus, and in server mode a bogus answer is replaced by SERVFAIL. The design is described in Section 4.1.9.

4.1.8 QNAME minimisation

A classic resolver sends the full query name to every server it asks, so the root and TLD servers learn every name a user visits even though they only need to know the next label. QNAME minimisation (RFC 9156 [14]) removes this leak: each server is asked an NS question for the name one label below its own zone, and only the server that is authoritative for the final zone receives the full name and type. Fig. 4.5 compares the two behaviours for `www.example.com`.

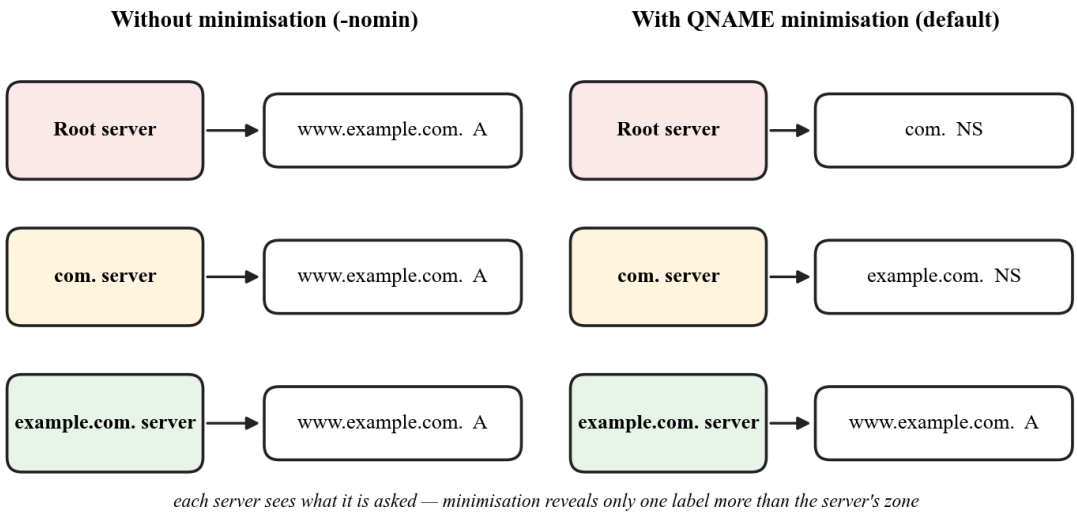


Fig. 4.5 Questions Seen by Each Server Without and With QNAME Minimisation

Three cases need care. If a probe reaches a name that is not a zone cut (for example `ac.in.` inside the `in.` zone), the server answers NODATA with AA = 1, and DNSR adds one more label and asks the same servers again. If the server holds both the parent and the child zone, it answers the NS probe with NS records in the answer section, which DNSR treats as a referral. If a probe receives any other reply (an error code, NXDOMAIN or a CNAME), DNSR stops minimising for that resolution and asks for the full name, because some servers handle empty non-terminals incorrectly. Names

with more than ten labels are not minimised, which bounds the number of extra queries. Minimisation can be switched off with `-nomin`.

4.1.9 DNSSEC validation design

DNSSEC (RFC 4033 to 4035 [13], [33], [34]) adds three record types. A zone publishes its public keys in a DNSKEY RRset and signs every RRset with RRSIG records. The parent zone publishes a DS record that contains a hash of the child's key-signing key and signs it with its own keys. A validating resolver therefore checks each answer against the keys of the zone that signed it, checks those keys against the DS in the parent, and repeats the process up to the root, whose key is trusted because its hash is built into the resolver as a trust anchor. Fig. 4.6 shows the chain for `www.cloudflare.com`.

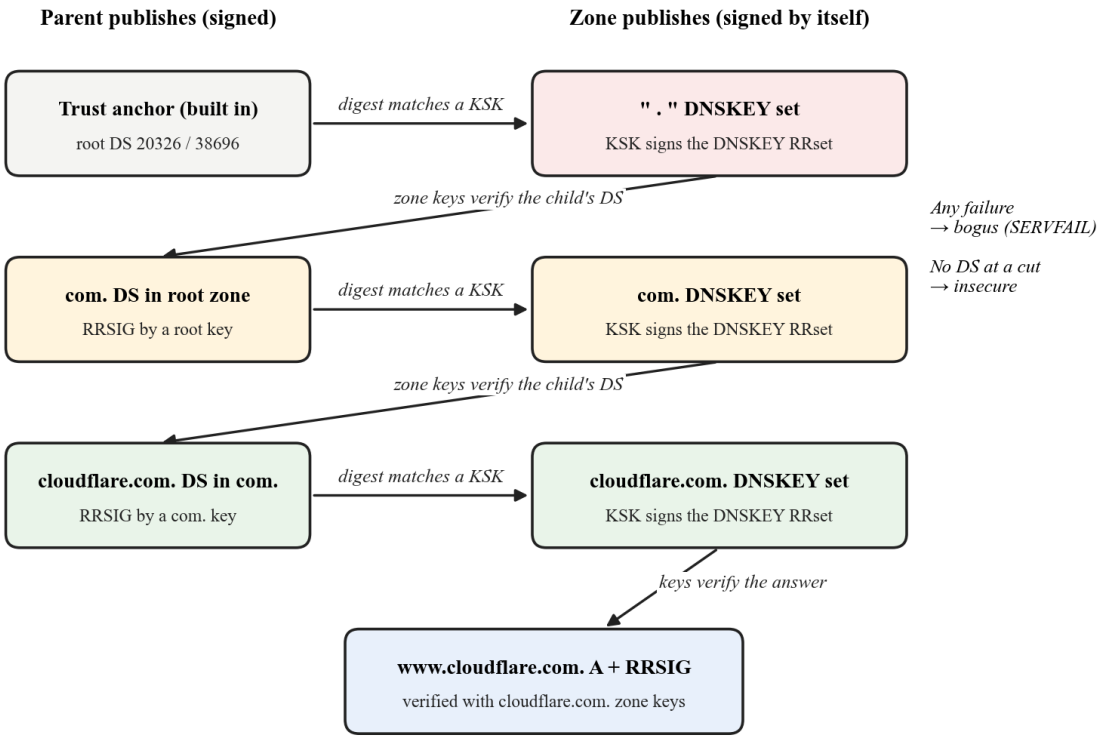


Fig. 4.6 Chain of Trust from the Root Trust Anchor to an Answer

DNSR implements this in `dnssec.go` as follows. For each RRset of a positive answer, it finds the RRSIG that covers the RRset and reads the signer's zone. `zoneKeys` returns the validated keys of that zone, computing them recursively: it fetches the zone's DS RRset from the parent (DS queries start at the parent zone), validates that RRset with the parent's keys, fetches the zone's DNSKEY RRset, finds a key-signing key

whose digest matches a DS record, and checks that this key signed the DNSKEY RRset. For the root, the DS records are the two IANA trust anchors [35]: KSK-2017 (key tag 20326) and KSK-2024 (key tag 38696). The second anchor is essential because the root zone moved to the 2024 key during the project. Validated keys are cached per zone with the DNSKEY TTL, so a chain is built at most once per zone.

Signature verification follows RFC 4034 exactly. The signed data is the RRSIG's own fields (without the signature) followed by every record of the RRset in canonical form: owner name in lower case without compression, the original TTL from the RRSIG, and RDATA in which embedded names are also lower-cased and uncompressed, sorted in canonical order. Wildcard expansions are handled through the RRSIG labels field. The validity window is checked with serial-number arithmetic (RFC 1982). Table 4.4 lists the algorithms supported, which cover the overwhelming majority of signed zones.

Table 4.4 DNSSEC Algorithms and Digests Supported

Number	Algorithm	Standard	Use
8	RSA/SHA-256	RFC 5702 [36]	Root zone, .com, .org and many others
10	RSA/SHA-512	RFC 5702 [36]	Some zones
13	ECDSA P-256 with SHA-256	RFC 6605 [37]	Most modern zones (e.g. cloudflare.com)
14	ECDSA P-384 with SHA-384	RFC 6605 [37]	Some zones
15	Ed25519	RFC 8080 [38]	Newer zones
DS digest 1, 2, 4	SHA-1, SHA-256, SHA-384	RFC 4509, 6605	Hash of the child key in DS records

Each answer receives one of three results. *Secure* means that every RRset was verified along a complete chain. *Insecure* means that the chain stops at a delegation without a DS record, which is how unsigned zones such as `github.com` appear. *Bogus* means that a signature failed, expired or was missing in a signed zone, or that no key matched the DS. The weakest result over all RRsets of an answer is reported. Two limitations are deliberate and documented: the absence of a DS record is not proven with NSEC or NSEC3 records, so an on-path attacker could make a signed zone appear unsigned; and negative answers (NXDOMAIN and NODATA) are not validated. Both are listed as future work.

4.1.10 Caching design

DNSR keeps three caches. The *answer cache* stores RRsets that answered a question, together with their signatures. Because the records of an RRset may carry different TTLs, the entry expires when the shortest TTL expires, but never later than one day (Eqn. 4.4), where t_store is the time the RRset was stored.

$$t_{exp} = t_{store} + min(\, min_i \, TTL_i \, , 86400 \, s \,) \tag{4.4}$$

When an entry is returned, every record's TTL is rewritten to the time that remains (Eqn. 4.5), as a real resolver does, so that a client never caches a record for longer than its owner intended.

$$TTL_{rem} = \lfloor \, t_{exp} - t_{now} \, \rfloor \tag{4.5}$$

The *delegation cache* stores, for every zone the resolver has been referred to, the name servers and addresses it learned, with an expiry time taken from the NS records. Before a resolution, `closestKnownZone` strips labels from the left of the queried name (`api.github.com.`, `github.com.`, `com.`, `.`) until it finds an unexpired zone in the cache. A second lookup under the same domain therefore skips the root and TLD servers completely, which Section 7.5 shows is the most effective optimisation for real workloads, in agreement with Jung et al. [12]. The *key cache* stores the result of validating each zone's DNSKEY set. All three caches are bounded to 10,000 entries.

4.1.11 Transport design

Each exchange uses a new UDP socket with a deadline equal to the timeout. The reply is read into a 4096-byte buffer and parsed. If the TC bit is set, the same query bytes are sent over a new TCP connection to the same server, framed with a two-byte length prefix; the reply length is read first and then exactly that many bytes are read with `io.ReadFull`. The maximum message size over TCP is therefore 65,535 bytes, which also bounds the memory a TCP reply can consume. DNSSEC responses are larger because of the signatures, so TCP fallback is used more often when validation is enabled. Servers in a zone are tried in random order, which spreads load and makes the next server harder to predict, and a server that returns an error code is removed from the candidate list for the rest of that resolution.

4.1.12 Server-mode design

In server mode DNSR acts as a recursive resolver for other clients. Its structure is shown in Fig. 4.7. It listens on UDP and TCP on the same address and, optionally, on a separate address for DNS over TLS (RFC 7858 [15]). The UDP loop copies each datagram and hands it to a goroutine; when 256 queries are already in flight, new datagrams are dropped rather than queued, so a flood cannot create unbounded goroutines. TCP and TLS connections share one stream handler that reads two-byte length-prefixed messages and closes idle connections after ten seconds. All transports call the same `handle` function, which parses the query, drops it silently if it cannot be parsed, has the QR bit set or does not have exactly one question, and applies a per-client token-bucket rate limit (20 queries per second with a burst of 40 by default). Rate-limit drops are logged at most once per client per minute, so an attacker cannot flood the log either.

Before any work is done for a query, the client's address is checked against an access list given with `-allow` as CIDR ranges or single addresses, for example `-allow 127.0.0.0/8,192.168.0.0/16`. Without the flag only loopback clients are served, so even a server bound to a public address does not become an open resolver that strangers could use for reflection attacks. A client outside the list receives REFUSED, a reply of the same size as its query, so the refusal cannot be used for amplification; refusals are logged at most once per client per minute. DNS over TLS accepts only TLS 1.3 (RFC 8446 [39]), which removes older protocol versions and cipher suites.

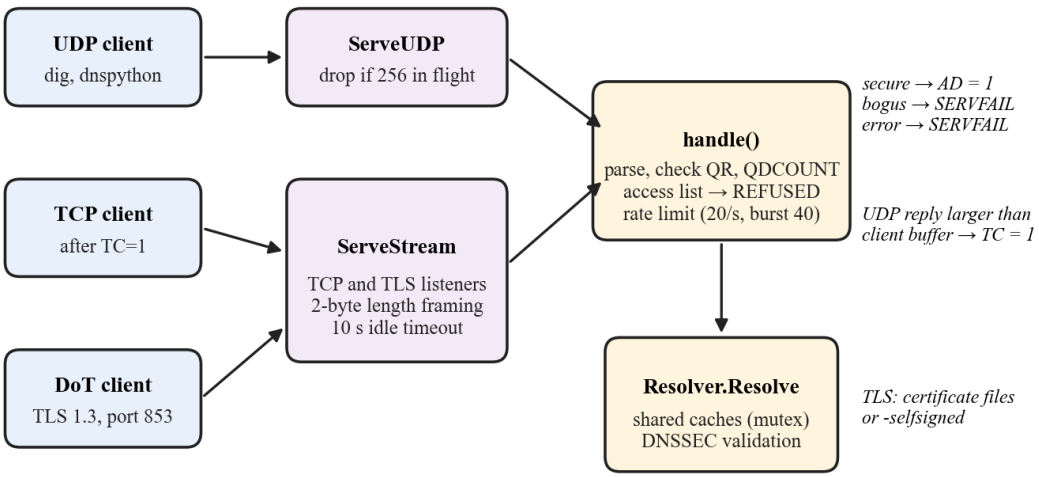


Fig. 4.7 Structure of the Recursive Server Mode

The response carries QR = 1, RA = 1, the client's RD bit, the original ID and question, and the answer and authority records. With validation enabled, secure answers get the AD (authenticated data) flag and bogus answers are replaced by SERVFAIL, as a validating resolver should. RRSIG, NSEC and NSEC3 records are returned only to clients that set the DO bit; for other clients they are removed, which also keeps negative answers small. If a UDP response is larger than the client accepts (512 bytes, or its EDNS buffer size up to 1232), the server returns an empty response with TC = 1 so that the client retries over TCP. For DNS over TLS the server loads a certificate and key from files, or generates an in-memory self-signed ECDSA certificate for testing and prints its SHA-256 fingerprint so that clients can pin it.

A subtle design point is the encoding of the response. Records parsed from upstream replies may contain compressed names whose pointers refer to offsets in the *upstream* packet. Copying the raw RDATA into a new packet would leave those pointers pointing at random bytes. `encodeRData` therefore re-encodes the names inside NS, CNAME, PTR, MX and SOA records from their decoded form; other types, including the DNSSEC types whose names are never compressed, are copied as raw bytes.

4.1.13 Test environment design

Testing a resolver against the live Internet is not repeatable: servers, addresses and TTLs change, and edge cases such as poisoned glue cannot be produced on demand. DNSR's test suite therefore builds a complete fake DNS hierarchy inside the test process, shown in Fig. 4.8. Six small authoritative servers listen on loopback addresses 127.0.0.10 to 127.0.0.15, port 15353, over both UDP and TCP. The resolver under test is configured with the fake root as its only root hint and port 15353 as its destination port; no other change is needed, which means the tests exercise exactly the same code that runs in production.

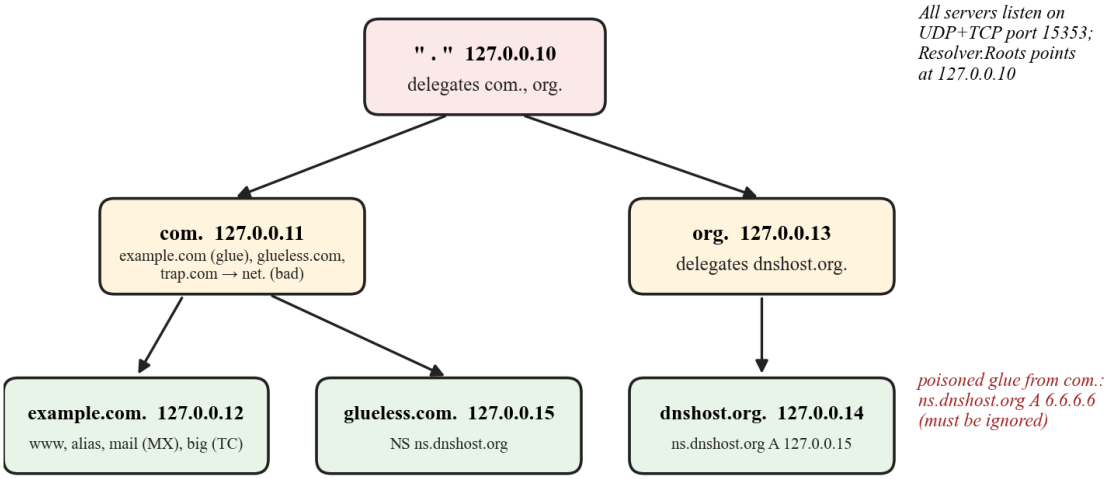


Fig. 4.8 Offline Test Hierarchy on Loopback Addresses

The root, `com.` and `example.com.` zones of the fake hierarchy are signed with freshly generated ECDSA P-256 keys, with DS records in their parents and the fake root key as the trust anchor; `glueless.com.` is deliberately unsigned. The hierarchy also contains hostile behaviour: the `com.` server delegates `trap.com.` with NS records for `net.` and attaches poisoned glue (`ns.dnshost.org A 6.6.6.6`) to the delegation of `glueless.com.`; it delegates `fanout.com.` to eight glueless name servers that do not exist; the `example.com.` server sends a forged reply with a wrong ID before the real one for one name, returns a reply to a different question over TCP for another, serves a record that was altered after signing, a record with a TTL of four billion seconds and an answer too large for 512 bytes. Every fake server logs the questions it receives, so the tests can check what each server saw. Table 4.5 lists the 32 tests.

Table 4.5 Test Cases of the Offline Suite

Test	What it verifies
TestReferralChain	Full root to TLD to authoritative walk returns the correct A record
TestDelegationCache	A second lookup in the same zone skips the root and TLD servers
TestCNAME	The alias is followed and the CNAME is placed before the final answer
TestNXDomainAndNoData	Non-existent names and missing record types are told apart
TestMX	MX RDATA (preference and exchange) is parsed correctly

Test	What it verifies
TestGluelessDelegationIgnoresPoisonedGlue	Out-of-bailiwick glue is discarded and the name server's address is resolved independently
TestOutOfBailiwickReferralRejected	A server cannot redirect resolution to an unrelated zone
TestTruncationFallsBackToTCP	A TC = 1 UDP reply triggers a successful retry over TCP
TestNameCompression	Compressed names decode correctly and a self-referencing pointer is rejected
TestTTLIsCapped	A TTL of four billion seconds is returned and cached as at most one day
TestCachesAreBounded	Answer and delegation caches never exceed their size limit
TestExpiredDelegationIsIgnored	An expired delegation is not used
TestGluelessLookupsAreLimited	Only 1 to 3 of 8 glueless name servers are looked up
TestForgedReplyIsIgnored	A reply with the wrong ID arriving first is ignored
TestTCPReplyMustMatchQuestion	A TCP reply for a different question is rejected
TestQNAMEMinimisation	The root sees only com. NS and the com. server only example.com. NS
TestQNAMEMinimisationCanBeDisabled	With -nomin the root sees the full name
TestDNSSECSecure	A, CNAME and MX answers from the signed zone validate as secure
TestDNSSECBogusWhenRecordIsTampered	A record altered after signing is bogus
TestDNSSECBogusWithWrongTrustAnchor	A chain that does not end at the trust anchor is bogus
TestDNSSECInsecureDelegation	A zone without a DS record is insecure
TestRRSIGParsesAndPrints	RRSIG fields are parsed correctly
TestServerSetsADForSecureAnswers	Server sets AD on a secure answer and returns SERVFAIL for a bogus one
TestServerTruncatesLargeUDPResponses	A response larger than 512 bytes is returned with TC = 1
TestServerStripsDNSSECRecordsForPlainClients	Signatures are sent only to clients that set DO
TestServerRateLimit	A client is limited to its burst; other clients are unaffected
TestDNSOverTLS	A TLS client verifying the server certificate receives a correct answer
TestDNSOverTLSRequiresTLS13	A TLS 1.2 client is rejected; a TLS 1.3 client connects
TestQueryIDsAndCaseAreRandom	Query IDs and letter-case patterns are drawn at random
TestQueriesUseRandomCaseAndAnswersAreLowercase	Servers receive mixed-case names; answers are returned in lower case

Test	What it verifies
TestCaseRandomisationFallsBackForServersThatDoNotPreserveCase	Resolution still succeeds against a server that ignores case, which is remembered
TestServerAccessList	Clients outside the list get REFUSED; the default list is loopback only; bad lists are rejected

4.2 Codes and Standards

DNSR implements a protocol defined entirely by Internet standards, so conformance to the relevant RFCs is the main standard the design follows. Table 4.6 lists each standard and the parts that DNSR implements. In addition, the code follows the conventions of the Go language.

Table 4.6 Standards Followed

Standard	Title	Used for
RFC 1034 [1]	Domain Names: Concepts and Facilities	Namespace, zones, resolver algorithm
RFC 1035 [2]	Domain Names: Implementation and Specification	Message format, name encoding and compression, RR types, TCP framing
RFC 2181 [4]	Clarifications to the DNS Specification	Data ranking and trust (basis of the bailiwick rules); RRset TTLs
RFC 2308 [5]	Negative Caching of DNS Queries	Recognising NXDOMAIN and NODATA by the SOA in authority
RFC 3596 [30]	DNS Extensions to Support IPv6	AAAA record format
RFC 4033, 4034, 4035 [13], [33], [34]	DNS Security Extensions	DS, DNSKEY, RRSIG; canonical form; key tags; validation
RFC 5702, 6605, 8080 [36]–[38]	DNSSEC algorithms	RSA/SHA-2, ECDSA and Ed25519 signatures
RFC 5452 [8]	Measures for Making DNS More Resilient against Forged Answers	Random ID and port, reply matching
RFC 6891 [6]	Extension Mechanisms for DNS (EDNS(0))	OPT record, UDP payload size, DO bit
RFC 7766 [7]	DNS Transport over TCP	TCP fallback; TCP service in server mode
RFC 7858 [15]	DNS over TLS	Encrypted client transport in server mode
RFC 8446 [39]	TLS 1.3	Only TLS version accepted for DNS over TLS
RFC 9156 [14]	QNAME Minimisation	Minimised NS probes towards authoritative servers
RFC 8499 [26]	DNS Terminology	Terms such as referral, glue, bailiwick
DNS Flag Day 2020 [20]	Recommended EDNS buffer size	1232-byte buffer
IANA named.root, root-anchors [21], [35]	Root hints and trust anchors	Root server addresses; KSK-2017 and KSK-2024

The coding standards applied are those of the Go project: all code is formatted with `gofmt`, checked with `go vet` and the race detector, and organised as one package with one file per layer. Errors are returned as values rather than through panics, exported identifiers start with a capital letter, and comments are limited to file headers and function descriptions. The references in this report follow the IEEE citation style.

4.3 Constraints, Alternatives and Trade-offs

4.3.1 Constraints

- ✓ **No DNS library:** the project brief required the protocol to be implemented from first principles, so every encoding, decoding and signature-verification step had to be written and tested by hand on top of the Go standard library.
- ✓ **Network access:** live resolution needs outbound UDP and TCP port 53. Some campus and corporate networks block this and force all DNS through their own resolver; the offline test hierarchy removes this dependency for testing.
- ✓ **Windows port conflicts:** on Windows port 5353 is used by the system's multicast DNS (mDNS) service, so the server's default port is 8053.
- ✓ **IPv4 only:** the test machine and many networks lack reliable IPv6 connectivity, so name servers are contacted only over IPv4.
- ✓ **Time:** the project had to be completed within one semester, which ruled out the authenticated denial of existence (NSEC and NSEC3 proofs) part of DNSSEC.

4.3.2 Alternatives considered

Several design decisions had reasonable alternatives. Table 4.7 records the options considered and the reason for each choice.

Table 4.7 Design Alternatives and Decisions

Decision	Alternatives	Choice and reason
Language	C, Rust, Python, Go	Go [22]: memory-safe, fast, built-in concurrency (goroutines), excellent networking and cryptography standard library, single static binary
Message handling	miekg/dns library; hand-written codec	Hand-written: required by the brief and needed to study and harden the parser
Resolution model	Forward to an upstream resolver; iterate from the root	Iterate from the root: the object of study and independent of any third party

Decision	Alternatives	Choice and reason
Server selection	Fixed order; random; RTT-based (as in Unbound)	Random: simple and spreads load; RTT-based selection left as future work
Parallelism	Query all servers in parallel; sequential fail-over	Sequential with a 2 s timeout: easier to trace and polite to servers, at the cost of slower recovery from a dead server
Cache structure	Single map; separate answer and delegation caches	Separate caches: delegations are the most valuable to keep (Jung et al. [12])
Glueless handling	Fail; resolve all NS names; resolve until one works	Resolve in random order until one address is found, at most three names
DNSSEC scope	None; positive answers only; full including NSEC/NSEC3	Positive answers: detects forged and altered data in signed zones within the time available
Minimisation mode	Strict; relaxed with fall-back	Relaxed (RFC 9156): fall back to the full name on unexpected replies, so no real domain fails
Client encryption	DNS over TLS; DNS over HTTPS	DoT: reuses the TCP framing already implemented and needs only the standard TLS library
Refused clients	Drop silently; answer REFUSED	REFUSED: informs legitimate clients, and is no larger than the query, so it cannot amplify
0x20 failures	Fail; accept anyway; retry without 0x20	Retry without 0x20 and remember the server: no domain breaks and no protection is lost elsewhere
Testing	Live Internet only; mocks of the network layer; real servers on loopback	Real servers on loopback: tests the real code path, deterministic and offline

4.3.3 Trade-offs

The main trade-off is between simplicity and completeness. Keeping DNSR to about two thousand lines makes every security decision visible, but it means that features of production resolvers, such as negative caching, prefetching, serve-stale, RTT tracking and validated denial of existence, are absent. A second trade-off is between latency and politeness: querying one server at a time with a two-second timeout means that an unreachable server adds up to two seconds to a resolution, but it avoids sending duplicate traffic to the DNS infrastructure. A third trade-off is between privacy and speed: QNAME minimisation hides names from the root and TLD servers but costs an extra query whenever a probe reaches a name that is not a zone cut, as Section 7.4 measures. Likewise, DNSSEC validation adds DS and DNSKEY lookups the first time each zone is seen. A fourth trade-off concerns strictness. Rejecting a referral that does not move strictly down the tree prevents redirection attacks, but a resolver that is too strict can fail on unusual but legitimate configurations; the checks in DNSR follow RFC 2181 closely so that all 15 domains in the live evaluation resolved

correctly. Finally, the trace output favours readability (colours, arrows and indentation) over machine parsing; quiet mode (-q) is provided for scripts.

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

The project was carried out over a fourteen-week semester. The work was divided into fourteen tasks grouped into four phases: study and design, core implementation, security and privacy features, and validation and documentation. Testing was not a separate phase at the end; the offline test hierarchy was built early and extended with every feature, as required by the test-driven method of Section 4.1.1.

5.1 Task Breakdown

Table 5.1 Task Breakdown and Schedule

No.	Task	Weeks	Output
1	Literature survey and study of RFC 1034/1035, 2181, 5452, 6891, 7766	1 to 2	Notes, list of requirements
2	Requirement analysis and design of the architecture	2	Requirements (Chapter 3), module plan
3	Wire-format codec: header, question, RR parsing, name compression, EDNS(0)	3 to 4	wire.go
4	Iterative resolver core: root hints, referral loop, glue	4 to 6	First end-to-end resolution
5	CNAME chasing, NXDOMAIN / NODATA, glueless delegations	6 to 7	Complete resolution logic
6	Security hardening: random IDs, reply matching, bailiwick checks, limits	7 to 8	Hardened resolver
7	Answer cache, delegation cache, TCP fallback	8 to 9	Caching and transport
8	Recursive server mode	9	server.go
9	QNAME minimisation, DNS over TLS 1.3, access list, server limits, 0x20, review fixes	10	Privacy and resilience features
10	DNSSEC validation from the root trust anchors	10 to 11	dnssec.go, signed test zones
11	Offline test hierarchy and test cases (incremental)	5 to 11	resolver_test.go, security_test.go, 32 tests
12	Live validation and measurements	12	Results of Chapter 7
13	Fuzzing and benchmarking	12	Robustness and performance results
14	Report, diagrams and documentation	12 to 14	This report, README

5.2 Gantt Chart

Fig. 5.1 shows the same schedule as a Gantt chart. Blue bars are design and implementation tasks; green bars are testing, validation and documentation tasks. The overlap of the test task with the implementation tasks reflects the test-driven approach.

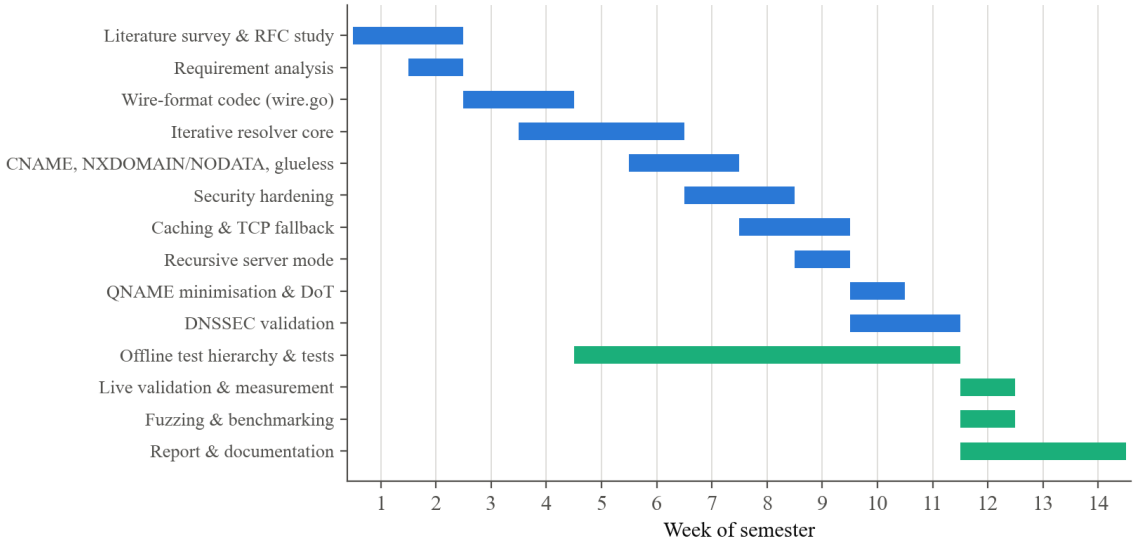


Fig. 5.1 Gantt Chart of the Project Schedule

5.3 Milestones

Table 5.2 Project Milestones

Milestone	Week	Completion criterion
M1: Design frozen	2	Requirements and architecture reviewed
M2: Codec complete	4	All record types parsed; compression test passes
M3: First live resolution	6	www.github.com resolved from the root servers
M4: Resolver feature-complete	8	CNAME, negative answers, glueless and bailiwick tests pass
M5: Caching, TCP and server	9	Delegation-cache, TCP and server tests pass
M6: Security and privacy features	11	DNSSEC, minimisation, DoT and hardening tests pass; live validation of signed domains
M7: Validation complete	12	Live comparison, cache and DNSSEC measurements and fuzzing done
M8: Final report	14	Report and documentation submitted

5.4 Risk Management

Table 5.3 lists the main risks identified during the project and how they were handled.

Table 5.3 Risks and Mitigations

Risk	Impact	Mitigation
Network blocks outbound port 53	Live tests impossible	Offline loopback hierarchy for all functional tests; live tests run from an unrestricted network
Live DNS data and network quality change between runs	Results not repeatable	Functional tests offline; live runs repeated three times and summarised by the median
Parser crashes on malformed input	Denial of service	Bounds checks everywhere; fuzzing with Go's native fuzzer
Root key rollover during the project	All DNSSEC validation fails	Both IANA trust anchors (KSK-2017 and KSK-2024) built in, taken from root-anchors.xml
Scope creep (NSEC proofs, IPv6)	Schedule slip	Explicit out-of-scope list (Section 2.5); items moved to future work
Port conflicts on Windows	Server mode cannot start	Default port 8053; configurable -addr flag

CHAPTER 6

PROJECT DEMONSTRATION

This chapter demonstrates the working system. Section 6.1 presents the most important parts of the source code with an explanation of each, and Section 6.2 shows the program in operation through screenshots of real runs against the live Internet and the test suite. The complete source code is given in Appendix A.

6.1 Sample Codes

The listings below are taken unchanged from the project source. They are grouped by layer, from the wire codec up to the server and the tests.

6.1.1 Building a query (wire.go)

`encodeName` turns a dotted name into length-prefixed labels and rejects invalid labels. `BuildQuery` writes a 12-byte header with a caller-supplied random ID and all flags cleared (so RD = 0), then the question, then the EDNS(0) OPT record that advertises a 1232-byte UDP buffer and, for DNSSEC, sets the DO bit.

```
func encodeName(buf []byte, name string) ([]byte, error) {
    name = strings.TrimSuffix(name, ".")
    if name != "" {
        for label := range strings.SplitSeq(name, ".") {
            if len(label) == 0 || len(label) > 63 {
                return nil, fmt.Errorf("bad label %q in %q", label, name)
            }
            buf = append(buf, byte(len(label)))
            buf = append(buf, label...)
        }
    }
    return append(buf, 0), nil
}

// BuildQuery makes a non-recursive query (RD=0) with an EDNS0 OPT record
// advertising a 1232-byte UDP buffer (the DNS Flag Day 2020 value).
// With dnssec set, the DO bit asks servers to include RRSIG records.
func BuildQuery(id uint16, name string, qtype RRType, dnssec bool) ([]byte, error) {
    b := make([]byte, 12, 512)
    binary.BigEndian.PutUint16(b[0:], id)
    binary.BigEndian.PutUint16(b[2:], 0)
    binary.BigEndian.PutUint16(b[4:], 1)
    binary.BigEndian.PutUint16(b[10:], 1)

    var err error
    if b, err = encodeName(b, name); err != nil {
        return nil, err
    }
    b = binary.BigEndian.AppendUint16(b, uint16(qtype))
    b = binary.BigEndian.AppendUint16(b, 1)

    b = append(b, 0)
```

```
    b = binary.BigEndian.AppendUint16(b, uint16(TypeOPT))
    b = binary.BigEndian.AppendUint16(b, 1232)
    var ext uint32
    if dnssec {
        ext = 0x8000
    }
    b = binary.BigEndian.AppendUint32(b, ext)
    b = binary.BigEndian.AppendUint16(b, 0)
    return b, nil
}
```

6.1.2 Decoding compressed names safely (wire.go)

`readName` is the most security-sensitive function in the codec, because it follows offsets taken from untrusted input. Every read is bounds-checked, the end offset of the name in the original stream is remembered at the first pointer, the number of pointer jumps is limited to 64, names longer than 255 bytes are rejected, and the reserved label types are refused.

```
// readName decodes a possibly-compressed name starting at off.
// Returns the name and the offset just past it in the *original* stream.
func readName(msg []byte, off int) (string, int, error) {
    var labels []string
    end, total := -1, 1
    for jumps := 0; ; {
        if off >= len(msg) {
            return "", 0, errShort
        }
        l := int(msg[off])
        switch {
        case l == 0:
            if end < 0 {
                end = off + 1
            }
            return strings.Join(labels, ".") + ".", end, nil
        case l&0xC0 == 0xC0:
            if off+1 >= len(msg) {
                return "", 0, errShort
            }
            if end < 0 {
                end = off + 2
            }
            jumps++
            if jumps > 64 {
                return "", 0, errors.New("compression pointer loop")
            }
            off = int(binary.BigEndian.Uint16(msg[off:]) & 0x3FFF)
        case l&0xC0 != 0:
            return "", 0, fmt.Errorf("unsupported label type 0x%x", l)
        default:
            off++
            if off+l > len(msg) {
                return "", 0, errShort
            }
            if total += l + 1; total > 255 {
                return "", 0, errors.New("name longer than 255 bytes")
            }
            labels = append(labels, string(msg[off:off+l]))
            off += l
        }
    }
```

```
}
```

6.1.3 Parsing resource records (wire.go)

`readRR` reads the fixed part of a record, checks that RDLENGTH stays within the message, and then decodes the RDATA according to the type, including the three DNSSEC types. Names inside RDATA are decoded with `readName` on the whole message because they may be compressed. Unknown types are shown in the generic RFC 3597 `\#` hexadecimal form.

```
func readRR(msg []byte, off int) (RR, int, error) {
    var rr RR
    var err error
    if rr.Name, off, err = readName(msg, off); err != nil {
        return rr, 0, err
    }
    if off+10 > len(msg) {
        return rr, 0, errShort
    }
    rr.Type = RRType(binary.BigEndian.Uint16(msg[off:]))
    rr.Class = binary.BigEndian.Uint16(msg[off+2:])
    rr.TTL = binary.BigEndian.Uint32(msg[off+4:])
    rdlen := int(binary.BigEndian.Uint16(msg[off+8:]))
    off += 10
    if off+rdlen > len(msg) {
        return rr, 0, errShort
    }
    rd := msg[off : off+rdlen]
    rr.Raw = rd

    switch rr.Type {
    case TypeA:
        if rdlen == 4 {
            rr.Data = net.IP(rd).String()
        }
    case TypeAAAA:
        if rdlen == 16 {
            rr.Data = net.IP(rd).String()
        }
    case TypeNS, TypeCNAME, TypePTR:
        rr.Data, _, err = readName(msg, off)
    case TypeMX:
        if rdlen >= 3 {
            var host string
            host, _, err = readName(msg, off+2)
            rr.Data = fmt.Sprintf("%d %s", binary.BigEndian.Uint16(rd), host)
        }
    case TypeSOA:
        var mname, rname string
        var p int
        if mname, p, err = readName(msg, off); err == nil {
            if rname, p, err = readName(msg, p); err == nil && p+20 <= off+rdlen {
                rr.Data = fmt.Sprintf("%s %s %d %d %d %d %d", mname, rname,
                    binary.BigEndian.Uint32(msg[p:]), binary.BigEndian.Uint32(msg[p+4:]),
                    binary.BigEndian.Uint32(msg[p+8:]),
binary.BigEndian.Uint32(msg[p+12:]),
                    binary.BigEndian.Uint32(msg[p+16:]))
            }
        }
    case TypeTXT:
        var parts []string
        for i := 0; i < len(rd); {
            n := int(rd[i])
```

```
            i++
            if i+n > len(rd) {
                break
            }
            parts = append(parts, fmt.Sprintf("%q", rd[i:i+n]))
            i += n
        }
        rr.Data = strings.Join(parts, " ")
    case TypeDS:
        if rdlen > 4 {
            rr.Data = fmt.Sprintf("%d %d %d %X", binary.BigEndian.Uint16(rd), rd[2], rd[3],
rd[4:])
        }
    case TypeDNSKEY:
        if rdlen > 4 {
            rr.Data = fmt.Sprintf("%d %d %d %s", binary.BigEndian.Uint16(rd), rd[2], rd[3],
base64.StdEncoding.EncodeToString(rd[4:]))
        }
    case TypeRRSIG:
        if s, perr := parseRRSIG(rd); perr == nil {
            ts := func(v uint32) string { return time.Unix(int64(v),
0).UTC().Format("20060102150405") }
            rr.Data = fmt.Sprintf("%s %d %d %d %s %s %d %s %s", s.TypeCovered, s.Algorithm,
s.Labels, s.OrigTTL,
                ts(s.Expiration), ts(s.Inception), s.KeyTag, s.Signer,
base64.StdEncoding.EncodeToString(s.Signature))
        }
    default:
        rr.Data = fmt.Sprintf("\\# %d %x", rdlen, rd)
    }
    if err != nil {
        return rr, 0, err
    }
    return rr, off + rdlen, nil
}
```

6.1.4 The iterative resolution loop (resolver.go)

The `resolve` function implements the algorithm of Section 4.1.4. It checks the cache, starts at the closest known zone (the parent zone for DS queries) and then loops: send a minimised probe or the full query, fall back to the full name on an unexpected probe reply, handle NXDOMAIN and error codes, look for direct answers and CNAMEs, detect NODATA, and otherwise validate and follow the referral, resolving at most three name-server addresses itself when there is no usable glue.

```
func (r *Resolver) resolve(name string, qtype RRType, depth, hops int) (*Result, error) {
    if depth > 8 {
        return nil, ErrLoop
    }
    if rrs, sigs, ok := r.cacheGet(name, qtype); ok {
        r.tracef(depth, "%s %s %s", r.c(cGreen, "✓ cache hit"), name, qtype)
        return &Result{Rcode: RcodeNoError, Answers: rrs, Sigs: sigs, Hops: hops}, nil
    }

    start := name
    if qtype == TypeDS && name != "." {
        start = parentName(name)
    }
    zone, servers := r.closestKnownZone(start)
    if zone != "." {
        r.tracef(depth, "%s starting at cached zone %s", r.c(cDim, "↷"), r.c(cBold, zone))
    }
    minimise := r.Minimise && len(splitLabels(name)) <= maxMinimiseLabels
```

```
    probe := childToward(name, zone)

    for ; hops < r.MaxHops; hops++ {
        qname, qt := name, qtype
        if minimise && probe != name {
            qname, qt = probe, TypeNS
        }
        probing := qname != name
        msg, ns, rtt, err := r.queryAny(servers, qname, qt, depth)
        if err != nil {
            return nil, fmt.Errorf("querying %s servers for %s: %w", zone, qname, err)
        }

        if probing && (msg.Header.Rcode != RcodeNoError || unexpectedAnswer(msg, qname)) {
            r.tracef(depth, "%s %s gave %s to the minimised query, asking for the full name",
r.c(cYellow, "↺"), ns.Name, rcodeString(msg.Header.Rcode))
            minimise = false
            continue
        }

        if !probing {
            if msg.Header.Rcode == RcodeNXDomain {
                r.tracef(depth, "%s %s says %s (%s)", r.c(cRed, "✗"), ns.Name, r.c(cRed,
"NXDOMAIN"), rtt)
                return &Result{Rcode: RcodeNXDomain, Authority: r.clampTTLs(msg.Authority), Hops:
hops + 1}, nil
            }
            if msg.Header.Rcode != RcodeNoError {
                r.tracef(depth, "%s %s returned %s, trying next server", r.c(cYellow, "!"),
ns.Name, rcodeString(msg.Header.Rcode))
                servers = without(servers, ns)
                if len(servers) == 0 {
                    return &Result{Rcode: msg.Header.Rcode, Hops: hops + 1}, nil
                }
                continue
            }

            var direct, cnames, sigs []RR
            for _, rr := range msg.Answers {
                if !strings.EqualFold(rr.Name, name) {
                    continue
                }
                rr.Zone = zone
                switch {
                case rr.Type == qtype:
                    direct = append(direct, rr)
                case rr.Type == TypeCNAME:
                    cnames = append(cnames, rr)
                case rr.Type == TypeRRSIG:
                    sigs = append(sigs, rr)
                }
            }
            if len(direct) > 0 {
                r.tracef(depth, "%s %s answered %s (%s)", r.c(cGreen, "✓"), ns.Name,
                    r.c(cGreen, fmt.Sprintf("%d %s record(s)", len(direct), qtype)), rtt)
                direct, sigs = r.clampTTLs(direct), coveredSigs(sigs, qtype)
                r.cachePut(name, qtype, direct, sigs)
                return &Result{Rcode: RcodeNoError, Answers: direct, Sigs: sigs, Hops: hops + 1},
nil
            }
            if len(cnames) > 0 {
                target := fqdn(cnames[0].Data)
                r.tracef(depth, "%s %s is an alias → %s, restarting from the top", r.c(cCyan, "↪
CNAME"), name, r.c(cBold, target))
                sub, err := r.resolve(target, qtype, depth+1, hops+1)
                if err != nil {
                    return nil, err
                }
                sub.Answers = append(r.clampTTLs(cnames[:1]), sub.Answers...)
                sub.Sigs = append(coveredSigs(sigs, TypeCNAME), sub.Sigs...)
                return sub, nil
            }
            if msg.Header.AA {
                r.tracef(depth, "%s %s: name exists but has no %s records (NODATA)", r.c(cYellow,
"∅"), ns.Name, qtype)
```

```
                return &Result{Rcode: RcodeNoError, Authority: r.clampTTLs(msg.Authority), Hops:
hops + 1}, nil
            }
        }

        nsRRs, child := referralNS(msg, qname, probing)
        if len(nsRRs) == 0 {
            if probing && msg.Header.AA {
                r.tracef(depth, "%s %s: no zone cut at %s (%s)", r.c(cDim, "·"), ns.Name, qname,
rtt)
                probe = childToward(name, probe)
                continue
            }
            return nil, fmt.Errorf("lame response from %s: no answer, no referral", ns.Name)
        }

        if !isSubdomain(name, child) || !isSubdomain(child, zone) || child == zone {
            return nil, fmt.Errorf("out-of-bailiwick referral from %s: %s is not under %s",
ns.Name, child, zone)
        }

        next := r.glueFor(nsRRs, msg.Additional, zone)
        names := make([]string, 0, len(nsRRs))
        for _, rr := range nsRRs {
            names = append(names, strings.TrimSuffix(rr.Data, "."))
        }
        r.tracef(depth, "%s %s → referral to %s %s (%s)", r.c(cCyan, "↓"), ns.Name, r.c(cBold,
child),
            r.c(cDim, fmt.Sprintf("[%s]", summarize(names, 3))), rtt)

        if len(next) == 0 {
            r.tracef(depth, "%s no glue for %s — resolving a nameserver's address first",
r.c(cYellow, "…"), child)
            for i, nsrr := range shuffled(nsRRs) {
                if i >= maxGluelessLookups {
                    r.tracef(depth, "%s giving up after %d nameserver lookups for %s",
r.c(cYellow, "!"), maxGluelessLookups, child)
                    break
                }
                sub, err := r.resolve(fqdn(nsrr.Data), TypeA, depth+1, 0)
                if err != nil || sub.Rcode != RcodeNoError {
                    continue
                }
                for _, a := range sub.Answers {
                    if a.Type == TypeA {
                        next = append(next, nameserver{fqdn(nsrr.Data), a.Data})
                    }
                }
                if len(next) > 0 {
                    break
                }
            }
            if len(next) == 0 {
                return nil, fmt.Errorf("could not find an address for any %s nameserver", child)
            }
        }

        r.putZone(child, next, minTTL(nsRRs))
        zone, servers = child, next
        probe = childToward(name, zone)
    }
    return nil, ErrLoop
}
```

6.1.5 QNAME minimisation helpers (resolver.go)

`childToward` computes the next probe name; `referralNS` extracts a delegation either from the authority section or, for a probe, from NS records in the answer; `unexpectedAnswer` detects replies that require falling back to the full name.

```
// childToward returns the name one label below zone on the way to name
// ("www.example.com." from "com." → "example.com.").
func childToward(name, zone string) string {
    if name == zone || !isSubdomain(name, zone) {
        return name
    }
    nl, zl := splitLabels(name), len(splitLabels(zone))
    return strings.Join(nl[len(nl)-zl-1:], ".") + "."
}

// referralNS extracts the delegation from a reply: NS records in the authority
// section, or, for a minimised NS query, NS records for qname in the answer.
func referralNS(msg *Message, qname string, probing bool) ([]RR, string) {
    var out []RR
    var child string
    if probing {
        for _, rr := range msg.Answers {
            if rr.Type == TypeNS && strings.EqualFold(rr.Name, qname) {
                out, child = append(out, rr), strings.ToLower(rr.Name)
            }
        }
        if len(out) > 0 {
            return out, child
        }
    }
    for _, rr := range msg.Authority {
        if rr.Type == TypeNS {
            out, child = append(out, rr), strings.ToLower(rr.Name)
        }
    }
    return out, child
}

// unexpectedAnswer reports whether a reply to a minimised NS query contains
// anything other than NS records for the probed name (a CNAME, for example).
func unexpectedAnswer(msg *Message, qname string) bool {
    for _, rr := range msg.Answers {
        if rr.Type == TypeRRSIG {
            continue
        }
        if rr.Type != TypeNS || !strings.EqualFold(rr.Name, qname) {
            return true
        }
    }
    return false
}
```

6.1.6 Bailiwick helpers (resolver.go)

`isSubdomain` compares names on label boundaries (so `evilgithub.com.` is not under `github.com.`). `glueFor` keeps only address records that belong to one of the referred name servers and lie inside the zone of the server that sent them.

```
// isSubdomain reports whether child is at or below parent ("a.b.c." under "b.c.").
func isSubdomain(child, parent string) bool {
    if parent == "." {
        return true
    }
    return child == parent || strings.HasSuffix(child, "."+parent)
}

// glueFor pairs NS names with A records from the Additional section.
// Glue is only trusted if it lies inside the zone of the server that sent it
// (askedZone) — a .com server may vouch for ns1.example.com, not for ns.evil.org.
func (r *Resolver) glueFor(nsRRs, additional []RR, askedZone string) []nameserver {
    want := map[string]bool{}
    for _, rr := range nsRRs {
        want[fqdn(rr.Data)] = true
    }
    var out []nameserver
    for _, rr := range additional {
        n := fqdn(rr.Name)
        if rr.Type == TypeA && want[n] && isSubdomain(n, askedZone) {
            out = append(out, nameserver{n, rr.Data})
        }
    }
    return out
}
```

6.1.7 Anti-spoofing exchange and TCP fallback (resolver.go)

`matchesQuery` is the reply check shared by both transports. `randomID` and `randomCase` draw the query ID and the 0x20 letter pattern from the cryptographic random source. `exchange` applies 0x20 unless the server is known not to preserve case and retries without it if the case does not come back. `exchangeOnce` sends one query from a new connected UDP socket with a deadline and keeps reading until a datagram passes the check; forged or stray packets are skipped. A truncated reply causes the same query to be sent over TCP, where a mismatching reply is an error.

```
// matchesQuery is the anti-spoofing check: a reply is only accepted if it
// carries our ID and repeats exactly the question we asked.
func matchesQuery(m *Message, id uint16, name string, qtype RRType) bool {
    return m.Header.ID == id && m.Header.QR && len(m.Questions) == 1 &&
        strings.EqualFold(m.Questions[0].Name, name) && m.Questions[0].Type == qtype
}

// randomID draws a query ID from the operating system's cryptographic random source.
func randomID() uint16 {
    var b [2]byte
    crand.Read(b[:])
    return binary.BigEndian.Uint16(b[:])
}

// randomCase flips each letter of name to upper or lower case at random
// (the 0x20 technique), adding one bit of entropy per letter to the query.
func randomCase(name string) string {
    bits := make([]byte, len(name))
    crand.Read(bits)
    b := []byte(name)
    for i, c := range b {
        if 'a' <= c|0x20 && c|0x20 <= 'z' {
```

```
            b[i] = c&^0x20 | (bits[i]&1)<<5
        }
    }
    return string(b)
}

// exchange sends one query, with 0x20 case randomisation unless the server is
// known not to preserve case. A server that answers with different case is
// remembered for an hour and asked again without 0x20.
func (r *Resolver) exchange(ip, name string, qtype RRType) (*Message, error) {
    mixed := r.CaseRandomisation && !r.caseUnsafe(ip)
    m, err := r.exchangeOnce(ip, name, qtype, mixed)
    if errors.Is(err, errCaseMismatch) {
        r.tracef(0, "%s %s does not preserve letter case, retrying without 0x20",
r.c(cYellow, "!"), ip)
        r.markCaseUnsafe(ip)
        m, err = r.exchangeOnce(ip, name, qtype, false)
    }
    return m, err
}

// exchangeOnce sends one query over UDP, retrying over TCP if the answer is truncated.
func (r *Resolver) exchangeOnce(ip, name string, qtype RRType, mixed bool) (*Message,
error) {
    sent := name
    if mixed {
        sent = randomCase(name)
    }
    id := randomID()
    q, err := BuildQuery(id, sent, qtype, r.DNSSEC)
    if err != nil {
        return nil, err
    }
    addr := net.JoinHostPort(ip, r.Port)

    conn, err := net.DialTimeout("udp", addr, r.Timeout)
    if err != nil {
        return nil, err
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(r.Timeout))
    if _, err := conn.Write(q); err != nil {
        return nil, err
    }
    buf := make([]byte, 4096)
    for {
        n, err := conn.Read(buf)
        if err != nil {
            return nil, err
        }
        m, err := ParseMessage(buf[:n])
        if err != nil || !matchesQuery(m, id, sent, qtype) {
            continue
        }
        if mixed && m.Questions[0].Name != sent {
            return nil, errCaseMismatch
        }
        if m.Header.TC {
            r.tracef(0, "%s response truncated, retrying over TCP", r.c(cYellow, "⇄"))
            return exchangeTCP(addr, q, id, sent, qtype, r.Timeout, mixed)
        }
        lowerNames(m)
        return m, nil
    }
}

// DNS over TCP: same message, prefixed with a 2-byte length (RFC 1035 §4.2.2).
func exchangeTCP(addr string, q []byte, id uint16, name string, qtype RRType, timeout
time.Duration, mixed bool) (*Message, error) {
    conn, err := net.DialTimeout("tcp", addr, timeout)
    if err != nil {
        return nil, err
```

```
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(timeout))
    frame := binary.BigEndian.AppendUint16(nil, uint16(len(q)))
    if _, err := conn.Write(append(frame, q...)); err != nil {
        return nil, err
    }
    var lb [2]byte
    if _, err := io.ReadFull(conn, lb[:]); err != nil {
        return nil, err
    }
    buf := make([]byte, binary.BigEndian.Uint16(lb[:]))
    if _, err := io.ReadFull(conn, buf); err != nil {
        return nil, err
    }
    m, err := ParseMessage(buf)
    if err != nil {
        return nil, err
    }
    if !matchesQuery(m, id, name, qtype) {
        return nil, errors.New("TCP reply does not match the query (ID or question)")
    }
    if mixed && m.Questions[0].Name != name {
        return nil, errCaseMismatch
    }
    lowerNames(m)
    return m, nil
}
```

6.1.8 Bounded caches (resolver.go)

`cachePut` caps the TTL at one day and makes room with `evict` before inserting. `closestKnownZone` walks up the queried name, label by label, to find the deepest unexpired delegation.

```
// evict makes room in a bounded cache: expired entries go first, then arbitrary ones.
func evict[V any](m map[string]V, max int, expired func(V) bool) {
    if len(m) < max {
        return
    }
    for k, v := range m {
        if expired(v) {
            delete(m, k)
        }
    }
    for k := range m {
        if len(m) < max {
            break
        }
        delete(m, k)
    }
}

func (r *Resolver) cachePut(name string, t RRType, rrs, sigs []RR) {
    if len(rrs) == 0 {
        return
    }
    ttl := min(minTTL(rrs), r.MaxTTL)
    now := time.Now()
    r.mu.Lock()
    defer r.mu.Unlock()
    evict(r.cache, r.MaxCacheEntries, func(e cacheEntry) bool { return
now.After(e.expires) })
```

```
    r.cache[name+"|"+t.String()] = cacheEntry{rrs, sigs, now.Add(time.Duration(ttl) *
time.Second)}
}

func (r *Resolver) closestKnownZone(name string) (string, []nameserver) {
    r.mu.Lock()
    defer r.mu.Unlock()
    now := time.Now()
    for z := name; ; {
        if e, ok := r.zoneNS[z]; ok && len(e.ns) > 0 && now.Before(e.expires) {
            return z, e.ns
        }
        if z == "." {
            return ".", r.Roots
        }
        z = parentName(z)
    }
}
```

6.1.9 Signature verification (dnssec.go)

`signedData` reconstructs exactly the bytes that the zone signed: the RRSIG fields followed by the RRset in canonical form and order. `verifySignature` checks one signature with one key for the supported algorithms, and `verifyRRset` tries each matching RRSIG and key within the signature's validity window.

```
// signedData builds the bytes an RRSIG signs: its own fields followed by the
// RRset in canonical form and order (RFC 4034 §3.1.8.1, §6.3).
func signedData(s *rrsig, set []RR) ([]byte, error) {
    owner := strings.ToLower(set[0].Name)
    if n := labelCount(owner); int(s.Labels) < n {
        ls := splitLabels(owner)
        owner = "*." + strings.Join(ls[len(ls)-int(s.Labels):], ".") + "."
        if s.Labels == 0 {
            owner = "*."
        }
    }
    ow, err := encodeName(nil, owner)
    if err != nil {
        return nil, err
    }
    var rds [][]byte
    for _, rr := range set {
        rd, err := canonicalRData(rr)
        if err != nil {
            return nil, err
        }
        rds = append(rds, rd)
    }
    sort.Slice(rds, func(i, j int) bool { return bytes.Compare(rds[i], rds[j]) < 0 })
    out := s.prefix()
    for i, rd := range rds {
        if i > 0 && bytes.Equal(rd, rds[i-1]) {
            continue
        }
        out = append(out, ow...)
        out = binary.BigEndian.AppendUint16(out, uint16(set[0].Type))
        out = binary.BigEndian.AppendUint16(out, set[0].Class)
        out = binary.BigEndian.AppendUint32(out, s.OrigTTL)
        out = binary.BigEndian.AppendUint16(out, uint16(len(rd)))
        out = append(out, rd...)
    }
    return out, nil
}
```

```
// verifySignature checks one RRSIG against one DNSKEY for the supported algorithms.
func verifySignature(s *rrsig, k *dnskey, data []byte) error {
    switch k.Algorithm {
    case 8, 10:
        pub, err := rsaPublicKey(k.Key)
        if err != nil {
            return err
        }
        h, sum := crypto.SHA256, sha256.Sum256(data)
        digest := sum[:]
        if k.Algorithm == 10 {
            s512 := sha512.Sum512(data)
            h, digest = crypto.SHA512, s512[:]
        }
        if rsa.VerifyPKCS1v15(pub, h, digest, s.Signature) != nil {
            return errBadSignature
        }
        return nil
    case 13, 14:
        curve, size := elliptic.P256(), 32
        var digest []byte
        if k.Algorithm == 13 {
            d := sha256.Sum256(data)
            digest = d[:]
        } else {
            d := sha512.Sum384(data)
            curve, size, digest = elliptic.P384(), 48, d[:]
        }
        if len(k.Key) != 2*size || len(s.Signature) != 2*size {
            return errors.New("bad ECDSA key or signature length")
        }
        pub, err := ecdsa.ParseUncompressedPublicKey(curve, append([]byte{4}, k.Key...))
        if err != nil {
            return err
        }
        r, ss := new(big.Int).SetBytes(s.Signature[:size]),
new(big.Int).SetBytes(s.Signature[size:])
        if !ecdsa.Verify(pub, digest, r, ss) {
            return errBadSignature
        }
        return nil
    case 15:
        if len(k.Key) != ed25519.PublicKeySize || !ed25519.Verify(ed25519.PublicKey(k.Key),
data, s.Signature) {
            return errBadSignature
        }
        return nil
    }
    return fmt.Errorf("unsupported algorithm %d", k.Algorithm)
}

// verifyRRset succeeds if any RRSIG made by zone with one of keys verifies the RRset.
func verifyRRset(set, sigs []RR, keys []*dnskey, zone string) error {
    t := set[0].Type
    lastErr := fmt.Errorf("no RRSIG from %s for %s %s", zone, set[0].Name, t)
    for _, sr := range sigs {
        s, err := parseRRSIG(sr.Raw)
        if err != nil {
            lastErr = err
            continue
        }
        if s.TypeCovered != t || !strings.EqualFold(sr.Name, set[0].Name) || s.Signer !=
zone ||
            !isSubdomain(strings.ToLower(set[0].Name), s.Signer) {
            continue
        }
        if int(s.Labels) > labelCount(set[0].Name) {
            lastErr = errors.New("RRSIG labels field larger than the owner name")
            continue
        }
        if err := checkValidity(s, time.Now()); err != nil {
            lastErr = err
```

```
            continue
        }
        data, err := signedData(s, set)
        if err != nil {
            return err
        }
        for _, k := range keys {
            if k.Tag != s.KeyTag || k.Algorithm != s.Algorithm {
                continue
            }
            if err := verifySignature(s, k, data); err != nil {
                lastErr = fmt.Errorf("%s %s: %w", set[0].Name, t, err)
                continue
            }
            return nil
        }
    }
    return lastErr
}
```

6.1.10 Chain of trust (dnssec.go)

`fetchZoneKeys` builds one link of the chain: it fetches and verifies the zone's DS RRset with the parent's keys (or uses the root trust anchors), fetches the DNSKEY RRset, finds a key-signing key that matches a DS record and checks that it signed the key set. `validateResult` applies this to every RRset of an answer and reports the weakest result.

```
func (r *Resolver) fetchZoneKeys(zone string, depth int) ([]*dnskey, Security, string, uint32) {
    var dss []DSRecord
    if zone == "." {
        dss = r.Anchors
    } else {
        res, err := r.resolve(zone, TypeDS, 1, 0)
        if err != nil {
            return nil, SecBogus, fmt.Sprintf("cannot fetch DS for %s: %v", zone, err), 60
        }
        dsSet := filterType(res.Answers, TypeDS)
        if res.Rcode != RcodeNoError || len(dsSet) == 0 {
            r.tracef(1, "%s no DS for %s at its parent — unsigned delegation", r.c(cYellow, "»"),
zone)
            return nil, SecInsecure, fmt.Sprintf("no DS record for %s (unsigned delegation)",
zone), 300
        }
        parent := dsSet[0].Zone
        for _, sr := range coveredSigs(res.Sigs, TypeDS) {
            if s, err := parseRRSIG(sr.Raw); err == nil && s.Signer != zone && isSubdomain(zone,
s.Signer) {
                parent = s.Signer
                break
            }
        }
        pkeys, psec, pwhy := r.zoneKeys(parent, depth+1)
        if psec != SecSecure {
            return nil, psec, pwhy, 300
        }
        if err := verifyRRset(dsSet, res.Sigs, pkeys, parent); err != nil {
            return nil, SecBogus, fmt.Sprintf("DS for %s: %v", zone, err), 60
        }
        for _, rr := range dsSet {
            if ds, err := parseDS(rr); err == nil && supportedAlgorithm(ds.Algorithm) &&
digester(ds.DigestType) != nil {
                dss = append(dss, ds)
            }
        }
        if len(dss) == 0 {
```

```
            return nil, SecInsecure, fmt.Sprintf("%s uses only unsupported DNSSEC algorithms",
zone), 300
        }
        r.tracef(1, "%s DS for %s verified with the keys of %s", r.c(cGreen, "»"), zone, parent)
    }

    res, err := r.resolve(zone, TypeDNSKEY, 1, 0)
    if err != nil {
        return nil, SecBogus, fmt.Sprintf("cannot fetch DNSKEY for %s: %v", zone, err), 60
    }
    keyRRs := filterType(res.Answers, TypeDNSKEY)
    if len(keyRRs) == 0 {
        return nil, SecBogus, fmt.Sprintf("%s has a DS record but no DNSKEY", zone), 60
    }
    var all, trusted, zsks []*dnskey
    for _, rr := range keyRRs {
        k, err := parseDNSKEY(rr)
        if err != nil || k.Flags&0x0100 == 0 {
            continue
        }
        all = append(all, k)
        for _, ds := range dss {
            if dsMatches(ds, k) {
                trusted = append(trusted, k)
                break
            }
        }
    }
    if len(trusted) == 0 {
        return nil, SecBogus, fmt.Sprintf("no DNSKEY of %s matches its DS / trust anchor", zone),
60
    }
    if err := verifyRRset(keyRRs, res.Sigs, trusted, zone); err != nil {
        return nil, SecBogus, fmt.Sprintf("DNSKEY set of %s: %v", zone, err), 60
    }
    zsks = all
    r.tracef(1, "%s DNSKEY set of %s verified (%d keys, signed by key %d)", r.c(cGreen, "»"),
zone, len(zsks), trusted[0].Tag)
    return zsks, SecSecure, "", minTTL(keyRRs)
}

// validateResult checks every RRset of a positive answer; the weakest result wins.
func (r *Resolver) validateResult(res *Result) (Security, string) {
    if res.Rcode != RcodeNoError || len(res.Answers) == 0 {
        return SecUnchecked, "negative answers are not validated (NSEC/NSEC3 not implemented)"
    }
    type key struct {
        owner string
        t     RRType
    }
    var order []key
    sets := map[key][]RR{}
    for _, rr := range res.Answers {
        k := key{strings.ToLower(rr.Name), rr.Type}
        if _, ok := sets[k]; !ok {
            order = append(order, k)
        }
        sets[k] = append(sets[k], rr)
    }
    overall, note := SecSecure, "chain of trust verified from the root key"
    for _, k := range order {
        set := sets[k]
        var sigs []RR
        for _, s := range coveredSigs(res.Sigs, k.t) {
            if strings.EqualFold(s.Name, k.owner) {
                sigs = append(sigs, s)
            }
        }
        zone := set[0].Zone
        for _, sr := range sigs {
            if s, err := parseRRSIG(sr.Raw); err == nil && isSubdomain(k.owner, s.Signer) {
                zone = s.Signer
                break
            }
        }
        if zone == "" {
            zone = "."
        }
```

```
        keys, sec, why := r.zoneKeys(zone, 1)
        switch sec {
        case SecBogus:
            return SecBogus, why
        case SecInsecure:
            overall, note = SecInsecure, why
        case SecSecure:
            if err := verifyRRset(set, sigs, keys, zone); err != nil {
                return SecBogus, err.Error()
            }
        }
    }
    return overall, note
}
```

6.1.11 Server: rate limiting and query handling (server.go)

`ParseAllow` and `permitted` implement the client access list. `allow` is a per-client token bucket that also limits how often drops are logged. `handle` is shared by UDP, TCP and DNS over TLS: it validates the query, refuses clients outside the access list, applies the rate limit, resolves, maps DNSSEC results to the AD flag or SERVFAIL, strips DNSSEC records for clients that did not ask for them and truncates oversized UDP responses. `ServeStream` serves length-prefixed messages on TCP and TLS connections.

```
// ParseAllow reads a comma-separated list of CIDR ranges or single addresses.
func ParseAllow(list string) ([]netip.Prefix, error) {
    var out []netip.Prefix
    for _, item := range strings.Split(list, ",") {
        item = strings.TrimSpace(item)
        if item == "" {
            continue
        }
        if strings.Contains(item, "/") {
            p, err := netip.ParsePrefix(item)
            if err != nil {
                return nil, err
            }
            out = append(out, p.Masked())
            continue
        }
        a, err := netip.ParseAddr(item)
        if err != nil {
            return nil, err
        }
        out = append(out, netip.PrefixFrom(a.Unmap(), a.Unmap().BitLen()))
    }
    return out, nil
}

// permitted reports whether a client address falls inside the access list.
func (s *Server) permitted(ip string) bool {
    a, err := netip.ParseAddr(ip)
    if err != nil {
        return false
    }
    a = a.Unmap().WithZone("")
    allow := s.Allow
    if len(allow) == 0 {
        allow = loopbackOnly
    }
    for _, p := range allow {
```

```
        if p.Contains(a) {
            return true
        }
    }
    return false
}

// allow is a per-client token bucket: Rate queries per second, bursts up to Burst.
func (s *Server) allow(ip string) bool {
    now := time.Now()
    s.mu.Lock()
    defer s.mu.Unlock()
    if len(s.buckets) > 10000 {
        for k, b := range s.buckets {
            if now.Sub(b.last) > time.Minute {
                delete(s.buckets, k)
            }
        }
    }
    b, ok := s.buckets[ip]
    if !ok {
        b = &bucket{tokens: s.Burst, last: now}
        s.buckets[ip] = b
    }
    b.tokens = min(s.Burst, b.tokens+now.Sub(b.last).Seconds()*s.Rate)
    b.last = now
    if b.tokens < 1 {
        if b.dropped++; now.Sub(b.logged) > time.Minute {
            log.Printf("rate limit: dropping queries from %s (%d dropped)", ip, b.dropped)
            b.logged = now
        }
        return false
    }
    b.tokens--
    return true
}

// handle turns one query into one response; nil means drop the packet.
// For UDP, a response larger than the client's buffer is truncated (TC=1).
func (s *Server) handle(pkt []byte, client string, udp bool) []byte {
    m, err := ParseMessage(pkt)
    if err != nil || len(m.Questions) != 1 || m.Header.QR {
        return nil
    }
    q := m.Questions[0]
    if !s.permitted(client) {
        s.noteRefused(client)
        return buildResponse(m.Header.ID, q, m.Header.RD, RcodeRefused, nil, nil, nil)
    }
    if !s.allow(client) {
        return nil
    }
    res, err := s.R.Resolve(q.Name, q.Type)
    var out []byte
    switch {
    case err != nil:
        log.Printf("%-30s %-5s SERVFAIL (%v)", q.Name, q.Type, err)
        out = buildResponse(m.Header.ID, q, m.Header.RD, RcodeServFail, nil, nil, nil)
    case res.Security == SecBogus:
        log.Printf("%-30s %-5s SERVFAIL (DNSSEC bogus: %s)", q.Name, q.Type,
res.SecurityNote)
        out = buildResponse(m.Header.ID, q, m.Header.RD, RcodeServFail, nil, nil, nil)
    default:
        log.Printf("%-30s %-5s %-8s %d answer(s), %d hops, %s%s", q.Name, q.Type,
rcodeString(res.Rcode),
            len(res.Answers), res.Hops, res.Elapsed.Round(1e6), securityTag(res))
        ans, auth := res.Answers, res.Authority
        if m.EDNS != nil && m.EDNS.DO {
            ans = append(append([]RR(nil), ans...), res.Sigs...)
        } else {
            auth = withoutDNSSEC(auth)
        }
        out = buildResponse(m.Header.ID, q, m.Header.RD, res.Rcode, ans, auth, nil)
```

```
        if res.Security == SecSecure {
            out[3] |= 0x20
        }
    }
    if limit := udpLimit(m); udp && len(out) > limit {
        out = buildResponse(m.Header.ID, q, m.Header.RD, int(out[3]&0xF), nil, nil, nil)
        out[2] |= 0x02
    }
    return out
}

// ServeStream answers length-prefixed queries on TCP or TLS connections (RFC 7766, RFC
7858).
func (s *Server) ServeStream(ln net.Listener) error {
    s.init()
    for {
        c, err := ln.Accept()
        if err != nil {
            return err
        }
        select {
        case s.sem <- struct{}{}:
        default:
            c.Close()
            continue
        }
        go func() {
            defer func() { <-s.sem }()
            defer c.Close()
            client := hostOf(c.RemoteAddr())
            for {
                c.SetDeadline(time.Now().Add(10 * time.Second))
                var lb [2]byte
                if _, err := io.ReadFull(c, lb[:]); err != nil {
                    return
                }
                pkt := make([]byte, binary.BigEndian.Uint16(lb[:]))
                if _, err := io.ReadFull(c, pkt); err != nil {
                    return
                }
                out := s.handle(pkt, client, false)
                if out == nil {
                    return
                }
                if _, err := c.Write(append(binary.BigEndian.AppendUint16(nil,
uint16(len(out))), out...)); err != nil {
                    return
                }
            }
        }()
    }
}

// dotConfig is the TLS configuration for DNS over TLS: TLS 1.3 only.
func dotConfig(cert tls.Certificate) *tls.Config {
    return &tls.Config{Certificates: []tls.Certificate{cert}, MinVersion: tls.VersionTLS13}
}
```

6.1.12 Adversarial tests (resolver_test.go, security_test.go)

Several tests check security properties directly against the hostile servers of the fake hierarchy. Two of them are shown: poisoned glue for `ns.dnshost.org` must be ignored, and a record altered after it was signed must be reported as bogus.

```
func TestGluelessDelegationIgnoresPoisonedGlue(t *testing.T) {
```

```
    r, trace := newTestResolver(t)
    res, err := r.Resolve("glueless.com", TypeA)
    if err != nil || len(res.Answers) != 1 || res.Answers[0].Data != "10.0.0.1" {
        t.Fatalf("%+v %v\n%s", res, err, trace)
    }
    if strings.Contains(trace.String(), "6.6.6.6") {
        t.Errorf("used out-of-bailiwick glue:\n%s", trace)
    }
    if !strings.Contains(trace.String(), "no glue") {
        t.Errorf("expected glueless path:\n%s", trace)
    }
}

func TestDNSSECBogusWhenRecordIsTampered(t *testing.T) {
    n := newTestNet(t)
    n.r.DNSSEC = true
    res, err := n.r.Resolve("tampered.example.com", TypeA)
    if err != nil || res.Security != SecBogus {
        t.Fatalf("security %v (%s) err %v", res.Security, res.SecurityNote, err)
    }
}
```

6.1.13 Fuzzing and benchmark harness

For the robustness evaluation in Chapter 7 an additional test file was used. `FuzzParseMessage` feeds inputs generated by Go's coverage-guided fuzzer to `ParseMessage` and the DNSSEC record parsers; `TestRandomAndMutatedPackets` parses one million random and mutated packets with a fixed seed so that the run can be repeated; the benchmarks measure parsing, query building and verification of an ECDSA signature.

```
import (
    "math/rand"
    "testing"
)

func seedPackets() [][]byte {
    q, _ := BuildQuery(0x1234, "www.example.com.", TypeA, true)
    resp := []byte{
        0x12, 0x34, 0x81, 0x80, 0, 1, 0, 1, 0, 0, 0, 0,
        3, 'w', 'w', 'w', 7, 'e', 'x', 'a', 'm', 'p', 'l', 'e', 3, 'c', 'o', 'm', 0,
        0, 1, 0, 1,
        0xC0, 0x0C, 0, 1, 0, 1, 0, 0, 0x0E, 0x10, 0, 4, 1, 2, 3, 4,
    }
    s := newSigner("example.com.")
    signed := buildResponse(0x1234, Question{"www.example.com.", TypeA, 1}, false, 0,
        []RR{a("www.example.com.", "1.2.3.4"), s.sign([]RR{a("www.example.com.",
"1.2.3.4")}), s.rr, s.ds()}, nil, nil)
    return [][]byte{q, resp, signed}
}

func FuzzParseMessage(f *testing.F) {
    for _, s := range seedPackets() {
        f.Add(s)
    }
    f.Fuzz(func(t *testing.T, b []byte) {
        if m, err := ParseMessage(b); err == nil {
            for _, rr := range m.Answers {
```

```
                if rr.Type == TypeRRSIG {
                    parseRRSIG(rr.Raw)
                }
                if rr.Type == TypeDNSKEY {
                    parseDNSKEY(rr)
                }
            }
        }
    })
}

func TestRandomAndMutatedPackets(t *testing.T) {
    rng := rand.New(rand.NewSource(42))
    seeds := seedPackets()
    errs, oks := 0, 0
    for i := range 1_000_000 {
        var b []byte
        if i%2 == 0 {
            b = make([]byte, rng.Intn(600))
            rng.Read(b)
        } else {
            s := seeds[rng.Intn(len(seeds))]
            b = append([]byte(nil), s...)
            for range 1 + rng.Intn(6) {
                b[rng.Intn(len(b))] = byte(rng.Intn(256))
            }
            if rng.Intn(4) == 0 {
                b = b[:rng.Intn(len(b))]
            }
        }
        if _, err := ParseMessage(b); err != nil {
            errs++
        } else {
            oks++
        }
    }
    t.Logf("1,000,000 packets: %d rejected with error, %d parsed, 0 panics", errs,
oks)
}

func BenchmarkParseMessage(b *testing.B) {
    p := seedPackets()[1]
    for b.Loop() {
        ParseMessage(p)
    }
}

func BenchmarkBuildQuery(b *testing.B) {
    for i := 0; b.Loop(); i++ {
        BuildQuery(uint16(i), "www.example.com.", TypeA, false)
    }
}

func BenchmarkVerifyECDSAP256(b *testing.B) {
    s := newSigner("example.com.")
    k, _ := parseDNSKEY(s.rr)
    set := []RR{a("www.example.com.", "1.2.3.4")}
    sig := s.sign(set)
    for b.Loop() {
        if err := verifyRRset(set, []RR{sig}, []*dnskey{k}, "example.com."); err !=
nil {
            b.Fatal(err)
        }
    }
}
```

6.2 Sample Screen Shots

The screenshots below were captured on the Windows 11 test machine in PowerShell. Live runs query the real root, TLD and authoritative servers, so the exact servers chosen and the timings differ from run to run. In the trace, → marks a query sent, ↓ a referral, · a probe that found no zone cut, ↪ a CNAME, ⋯ a glueless delegation, ✓ an answer, ✗ NXDOMAIN, ∅ NODATA, ⇄ a TCP retry and » a DNSSEC step; nested work is indented with │.

6.2.1 Build and offline test suite

Fig. 6.1 shows the program being built with the standard Go tool chain and all 32 offline tests passing against the fake DNS hierarchy on loopback.

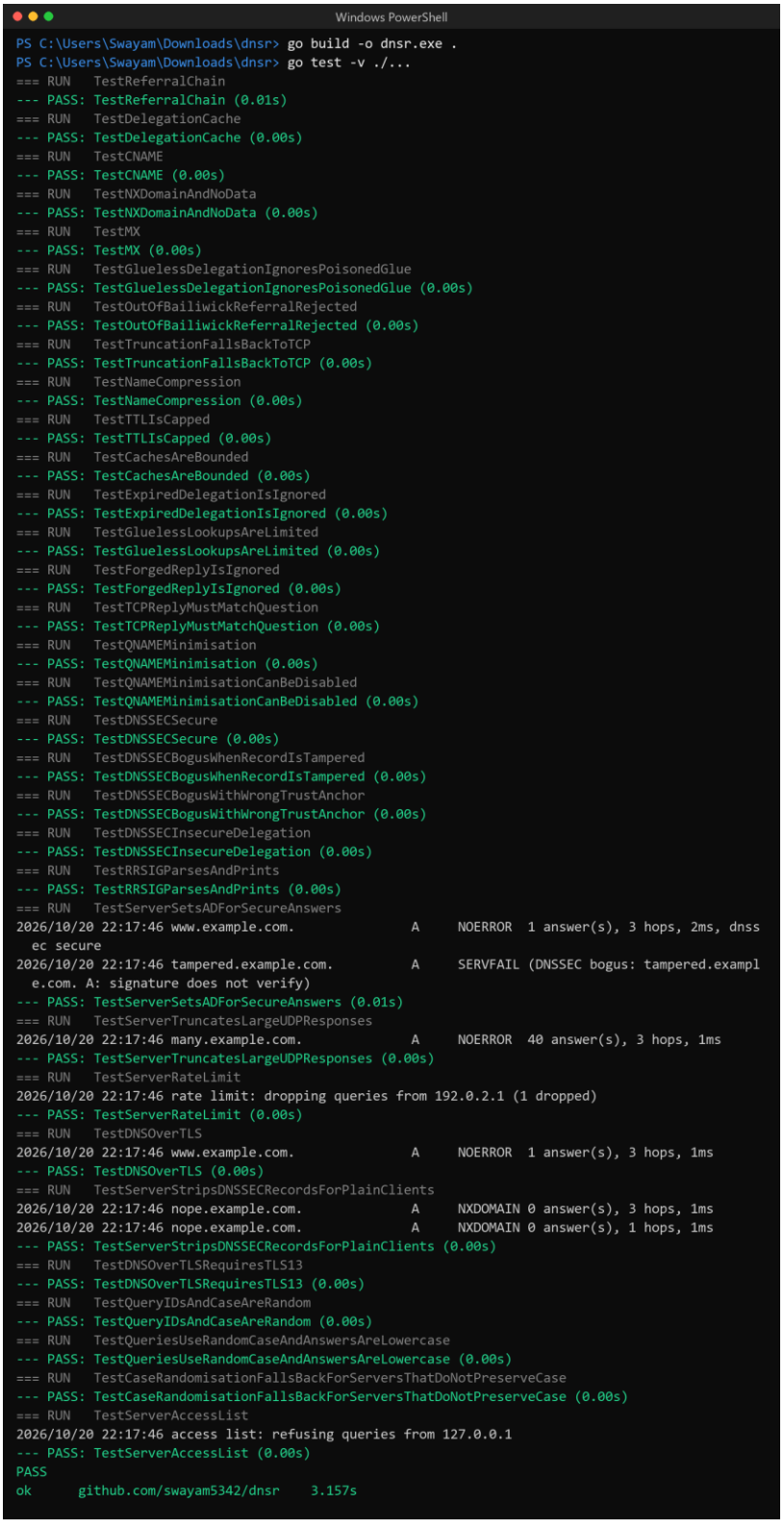


Fig. 6.1 Building DNSR and Running the Offline Test Suite

6.2.2 Usage

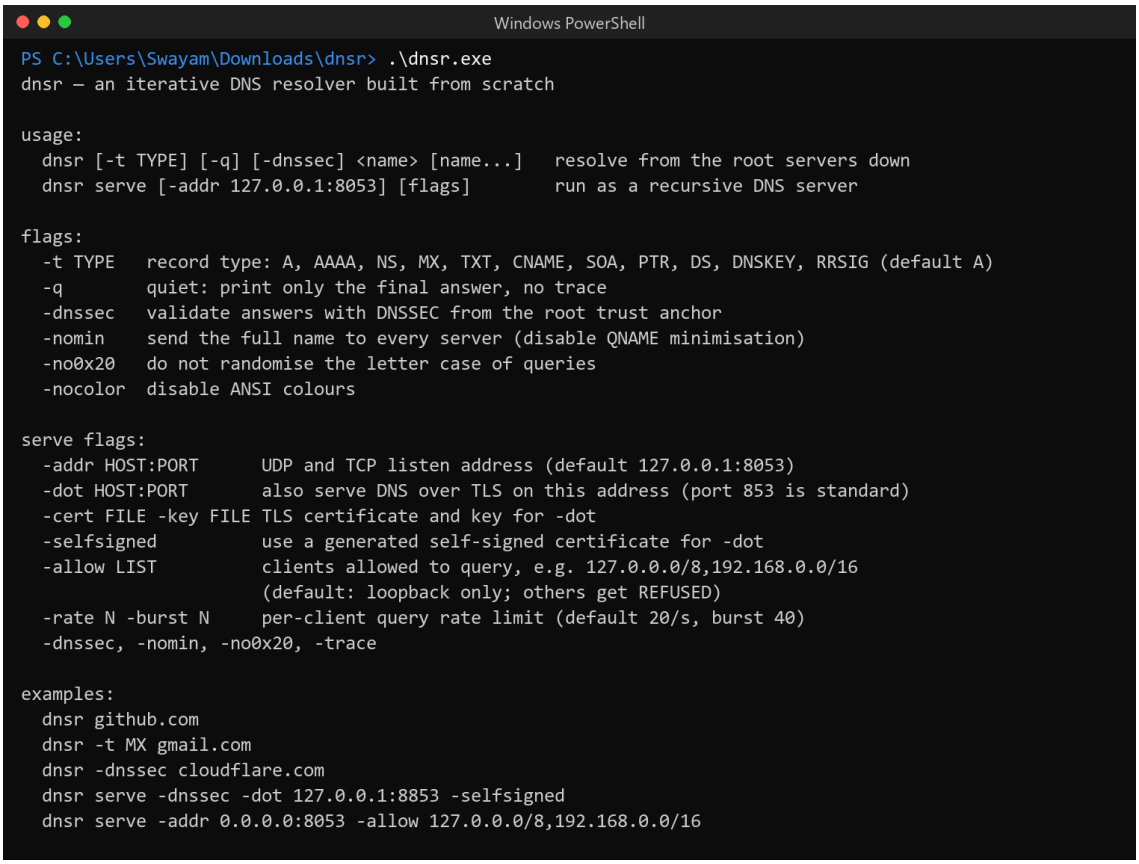


Fig. 6.2 Usage Message Printed When No Name Is Given

6.2.3 Iterative resolution with QNAME minimisation

Fig. 6.3 shows a cold-cache resolution of `www.github.com`. The root is asked only for `com. NS`, a `.com` server only for `github.com. NS`, and GitHub's name server receives the full question and answers with a CNAME. DNSR restarts for `github.com` at the cached `github.com.` zone. Fig. 6.4 shows the same lookup with minimisation switched off: every server, including the root, receives the full name.

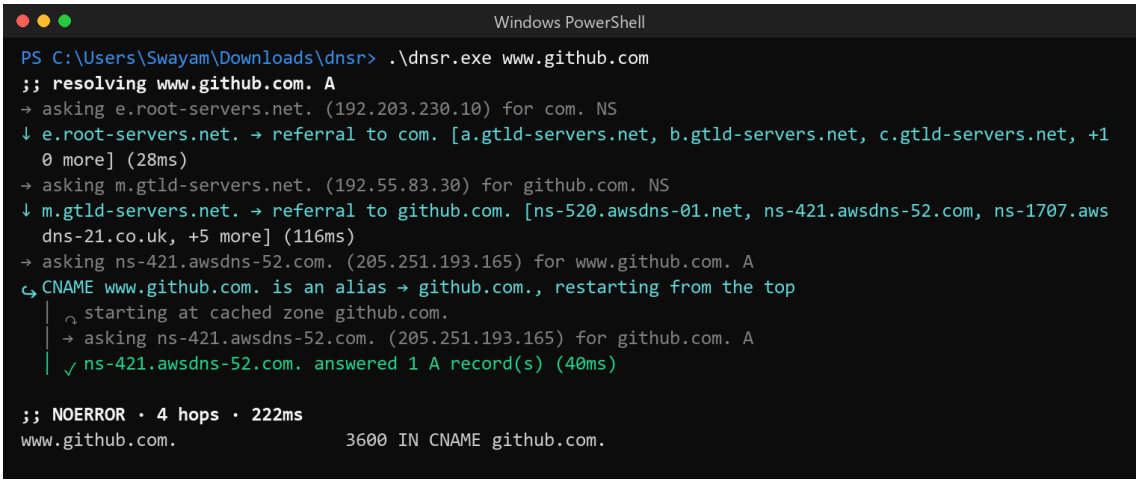


Fig. 6.3 Resolving www.github.com with QNAME Minimisation

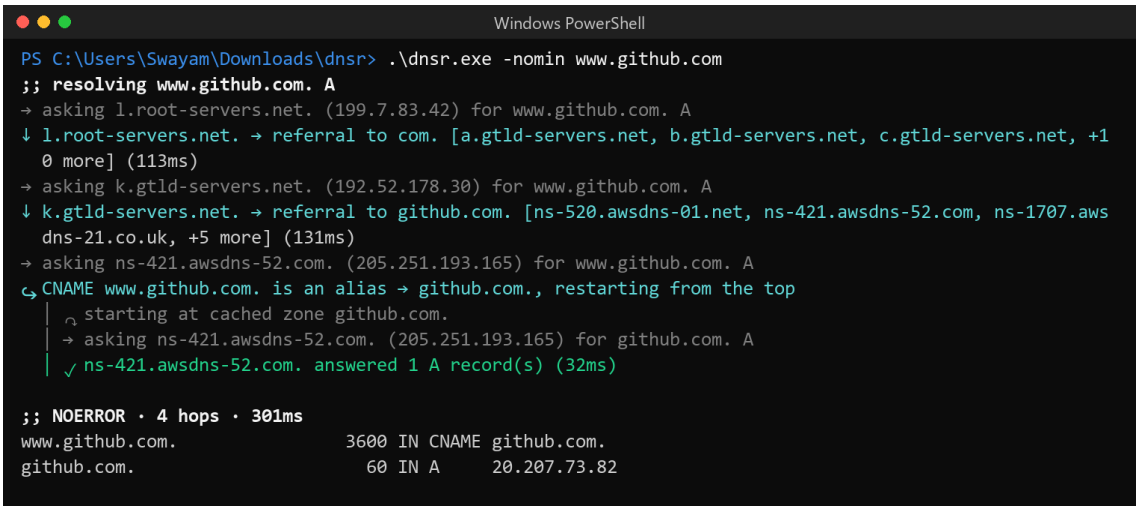


Fig. 6.4 The Same Lookup with Minimisation Disabled (-nomin)

6.2.4 MX and AAAA lookups

Fig. 6.5 resolves the mail exchangers of `gmail.com`, showing the MX preference values. Fig. 6.6 resolves the IPv6 address of `www.wikipedia.org`, which is a CNAME to `dyna.wikimedia.org`; because the `org.` delegation is already cached, the nested resolution for the target starts at `org.` rather than at the root.

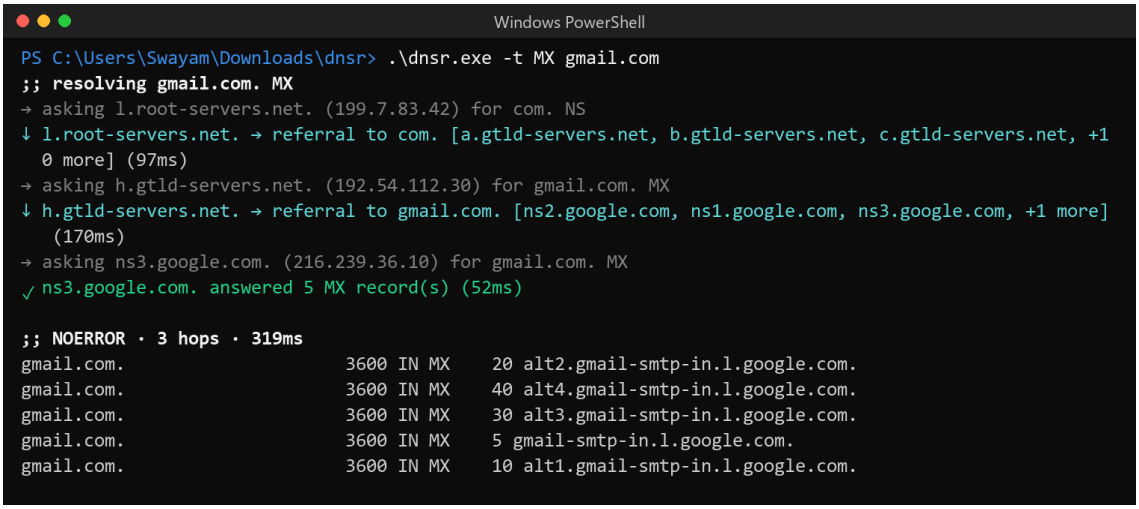


Fig. 6.5 MX Lookup for gmail.com

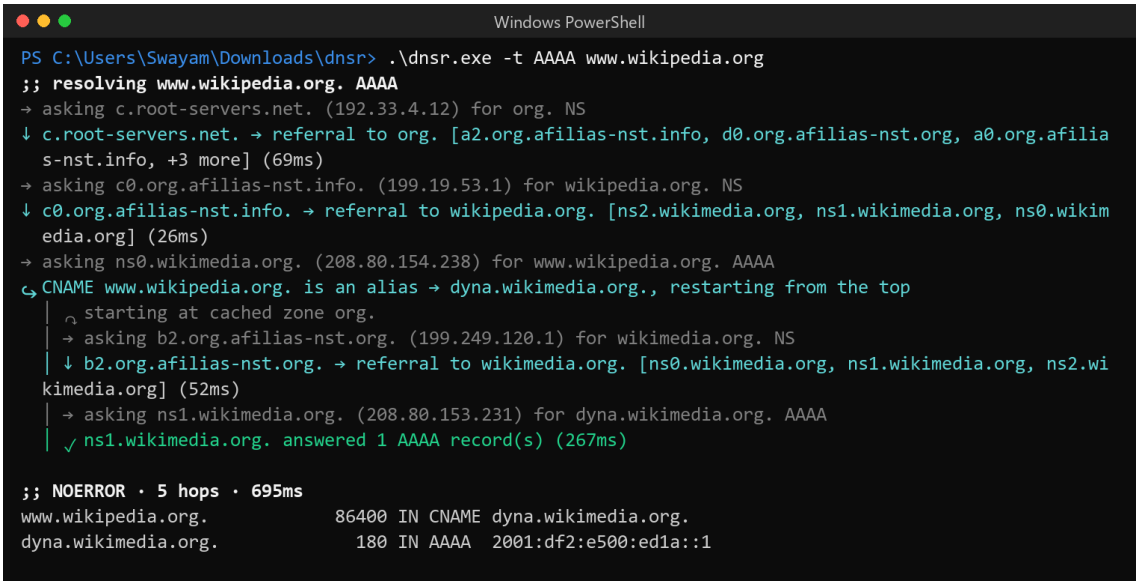


Fig. 6.6 AAAA Lookup for www.wikipedia.org Through a CNAME

6.2.5 Negative answers

Fig. 6.7 shows both kinds of negative answer. A name that does not exist under `.com` produces NXDOMAIN directly from the `.com` server, together with its SOA record. `github.com` exists but has no IPv6 address, so GitHub's authoritative server replies with AA = 1, no answer and an SOA record, which DNSR reports as NODATA with the NOERROR code.

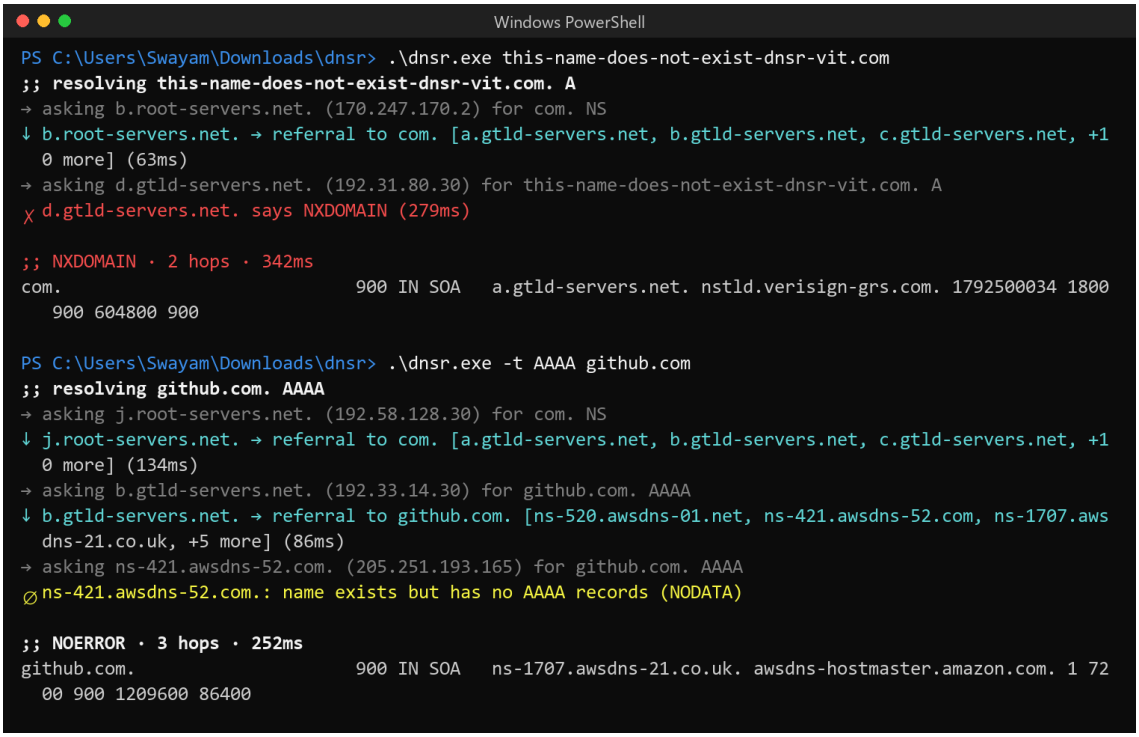


Fig. 6.7 NXDOMAIN and NODATA Responses

6.2.6 Glueless delegations

The name servers of `github.io` are in `.net` and `.co.uk`, so the `.io` server cannot provide glue for them. Fig. 6.8 shows DNSR pausing the main resolution and resolving the address of one name server through the `.net` hierarchy; the minimised probe for `p05.nsone.net.` finds no zone cut, so the next query uses the full name. Fig. 6.9 shows the same behaviour for the university's own domain `vit.ac.in`, whose name servers are hosted on Amazon Route 53; here the probe for `ac.in.` also finds no zone cut.

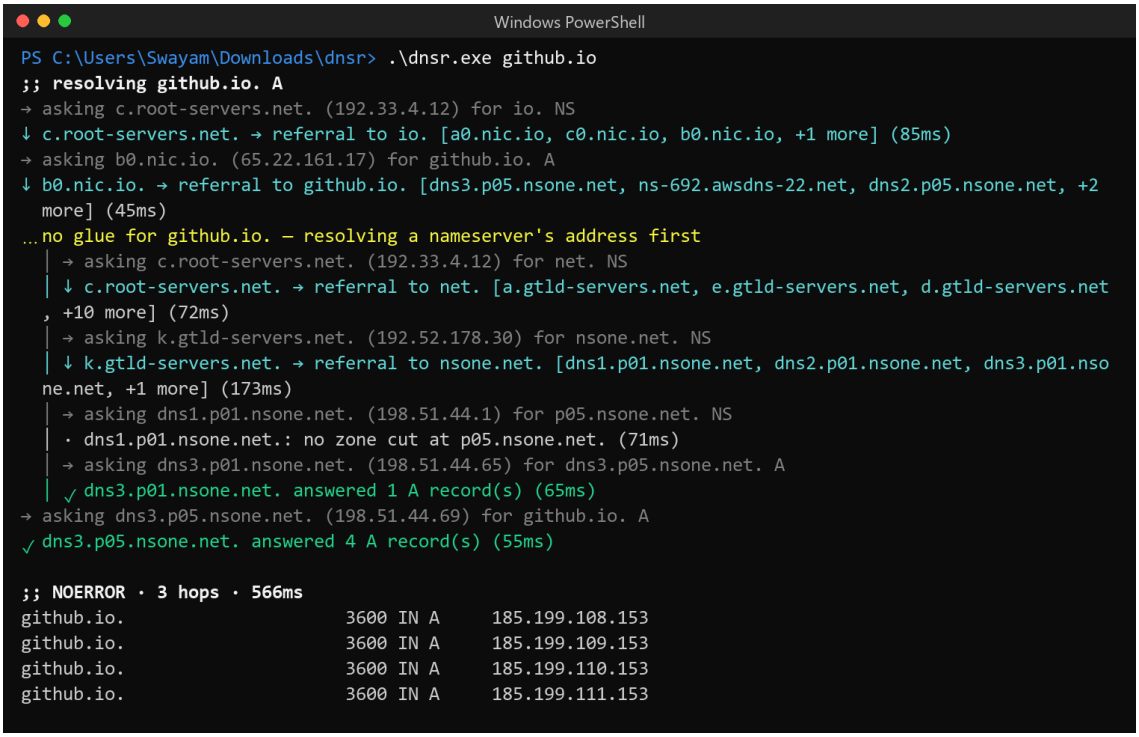


Fig. 6.8 Glueless Delegation for github.io

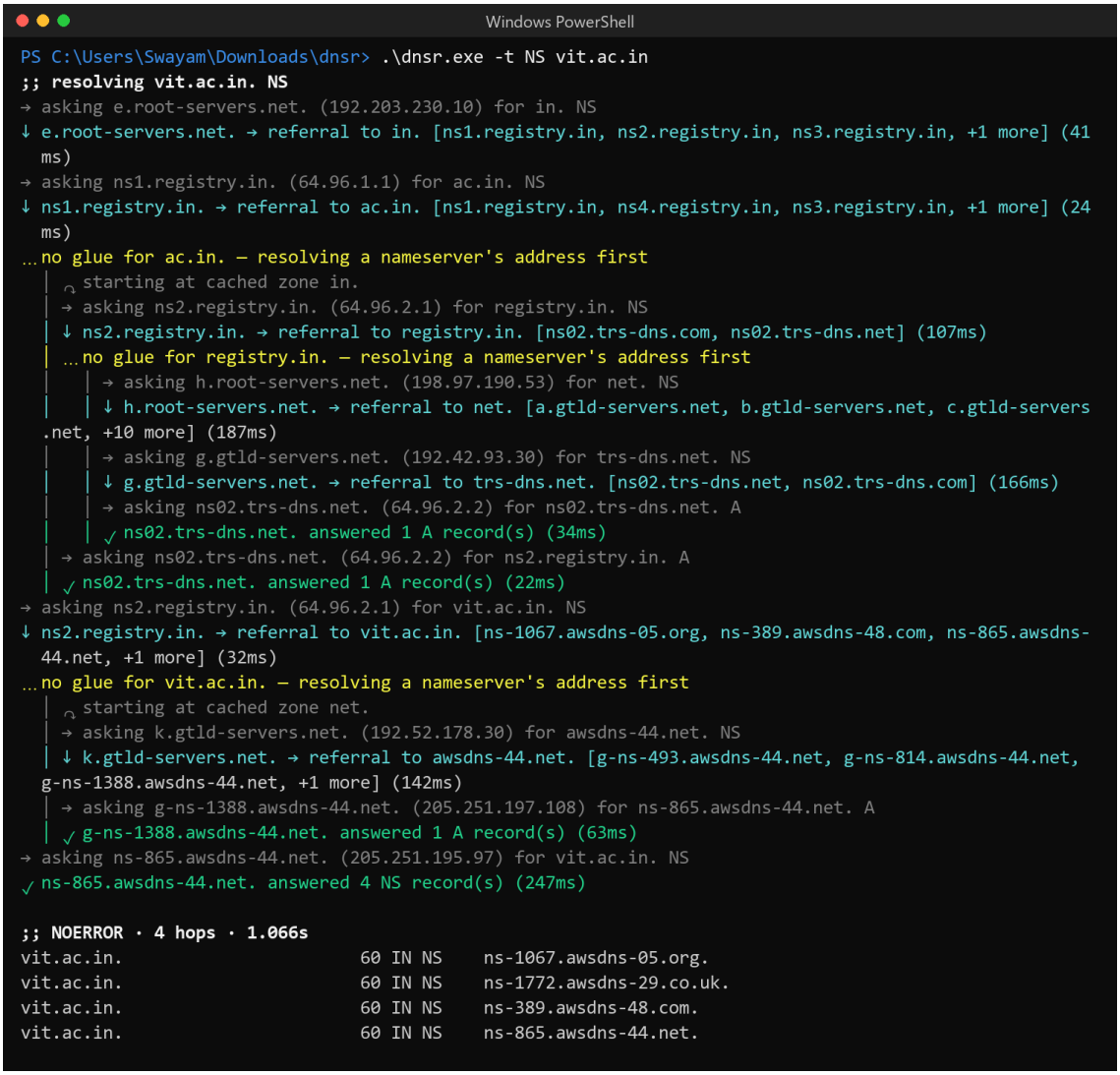


Fig. 6.9 Glueless Delegation and NS Lookup for vit.ac.in

6.2.7 Fallback from UDP to TCP

The TXT records of `apple.com` do not fit in a 1232-byte UDP reply. In Fig. 6.10 the authoritative server sets the TC bit and DNSR repeats the query over TCP (the screenshot shows the first lines of the answer).

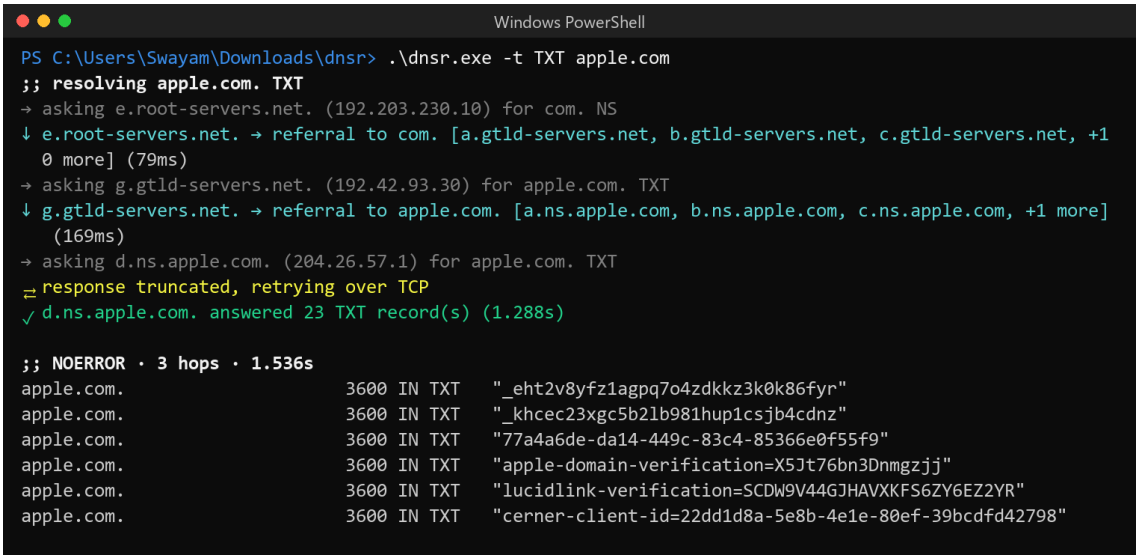


Fig. 6.10 Truncated UDP Reply Retried over TCP for apple.com TXT

6.2.8 Effect of the caches

In Fig. 6.11 three names are resolved in one run. The first lookup walks from the root. The second, `api.github.com`, starts at the cached `github.com.` zone and needs a single query. The third repeats the first name: the alias is asked once more, and the target `github.com` comes straight from the answer cache with its TTL reduced.

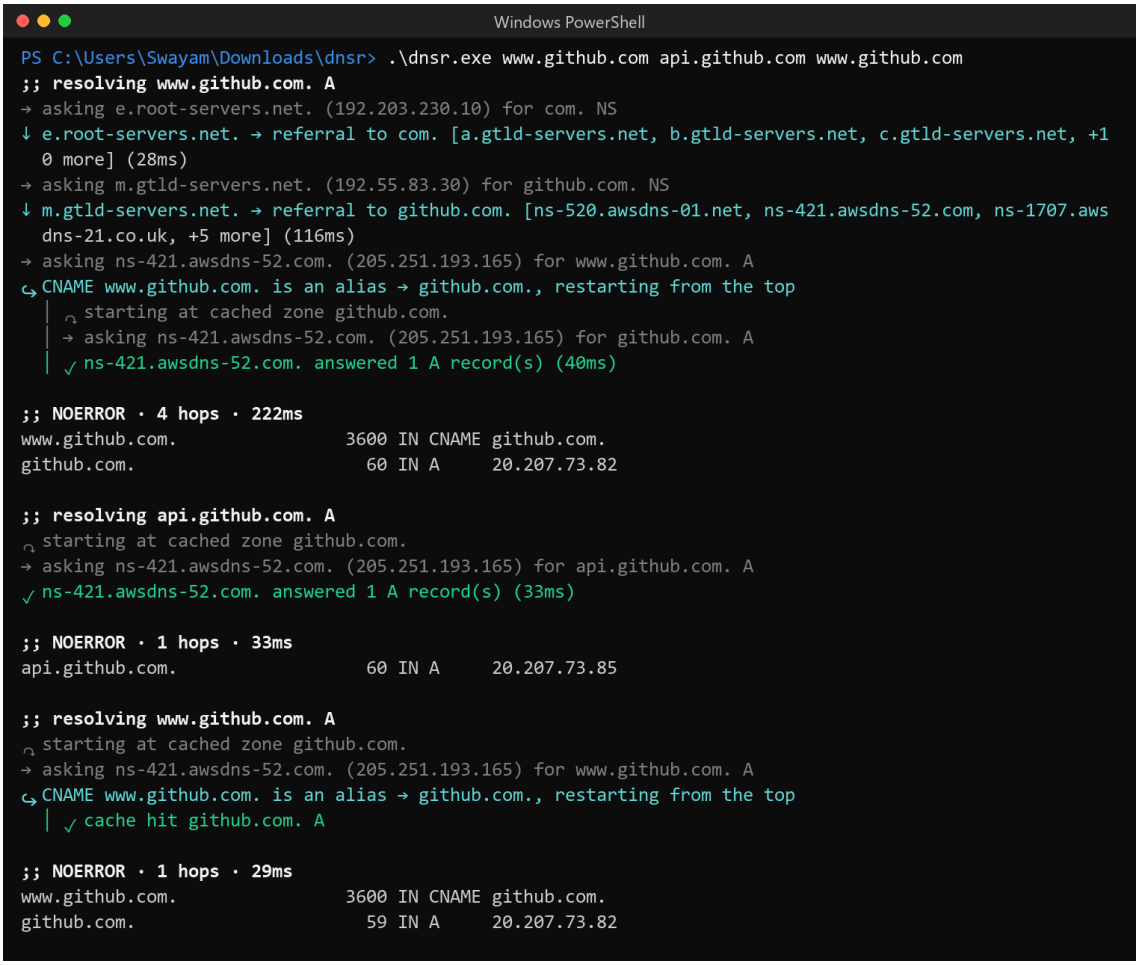


Fig. 6.11 Delegation Cache and Answer Cache in One Run

6.2.9 Quiet mode

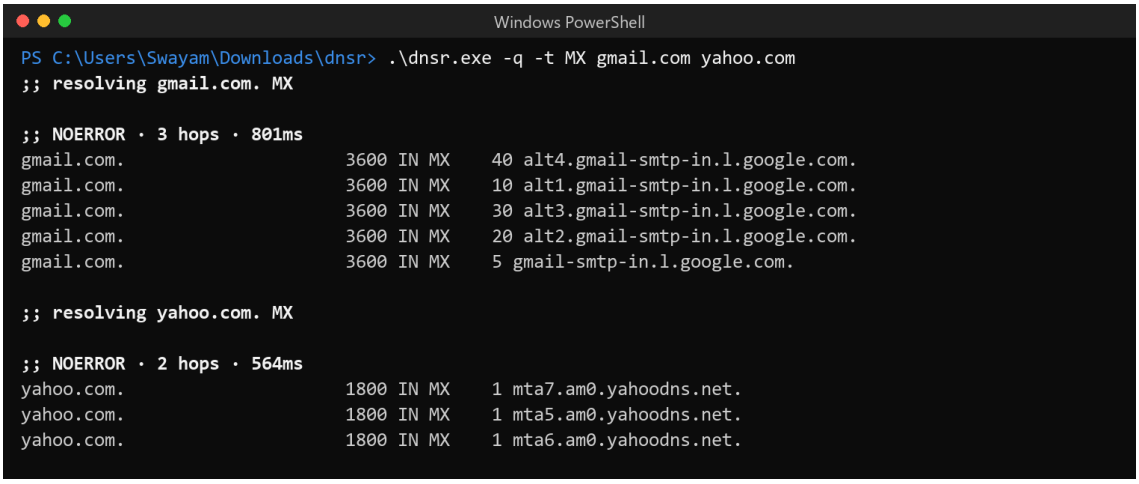


Fig. 6.12 Quiet Mode Printing Only the Answers

6.2.10 DNSSEC validation

Fig. 6.13 validates `cloudflare.com`. After the answer arrives, DNSR fetches the DS record of `cloudflare.com` from a `.com` server and the DS record of `com.` from a root server, then the root DNSKEY set, which it verifies against the trust anchor (the trace shows it was signed by KSK-2024, key 38696), then each zone's keys in turn, and finally the signature on the answer. Fig. 6.14 shows the three possible outcomes side by side: `github.com` is insecure because the `.com` zone has no DS record for it, `dnssec-failed.org` (a domain deliberately mis-signed for testing) is rejected as bogus, and `iana.org` is secure.

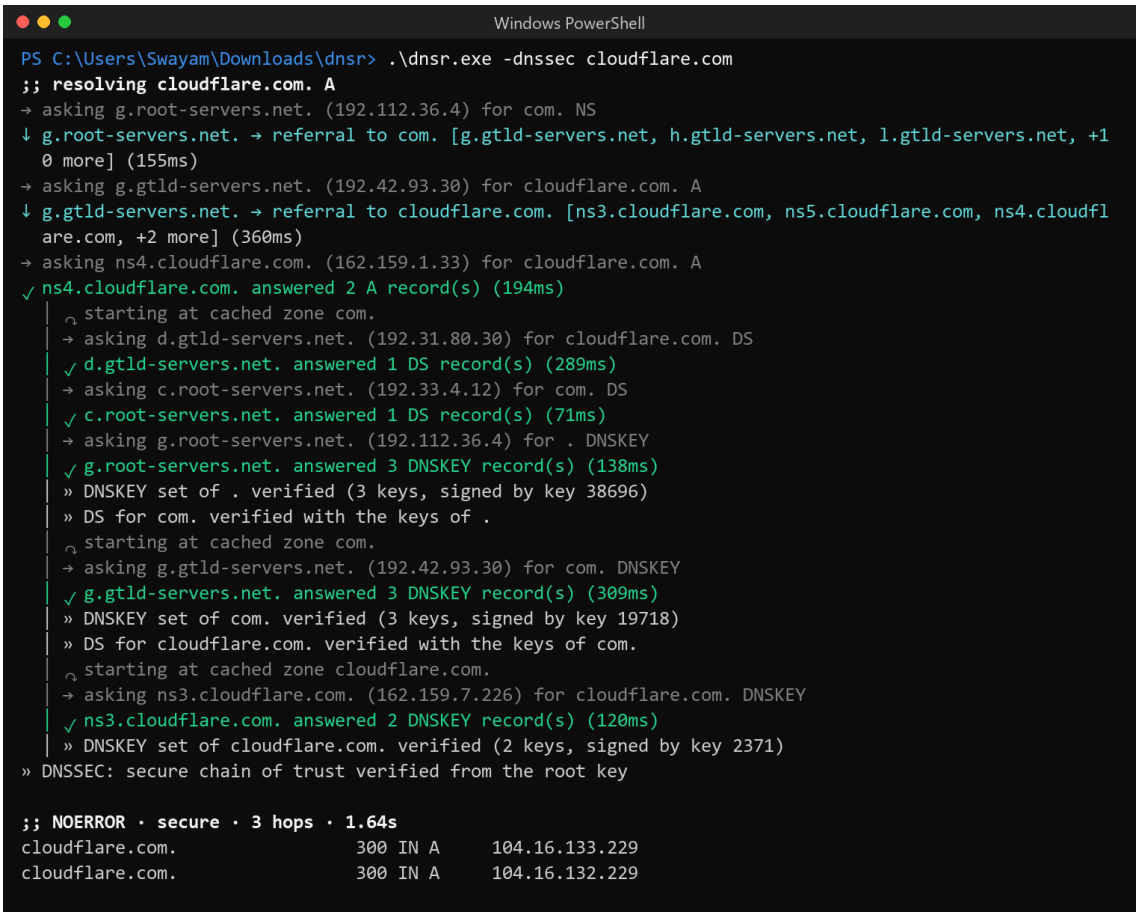


Fig. 6.13 DNSSEC Chain of Trust for cloudflare.com



Fig. 6.14 Insecure, Bogus and Secure Results

6.2.11 Recursive server with DNS over TLS

Fig. 6.15 shows DNSR running as a validating server on UDP and TCP port 8053 and on DNS over TLS port 8853 with a self-signed certificate, and Fig. 6.16 shows the client side. The client is an independent DNS implementation (dnspython [31]). It first reads the server certificate, whose SHA-256 fingerprint matches the one the server printed, then queries over UDP, TCP and TLS. Secure answers carry the AD flag, `github.com` is answered without it, the bogus domain receives SERVFAIL, and a burst of 60 back-to-back queries is partly dropped by the rate limiter.

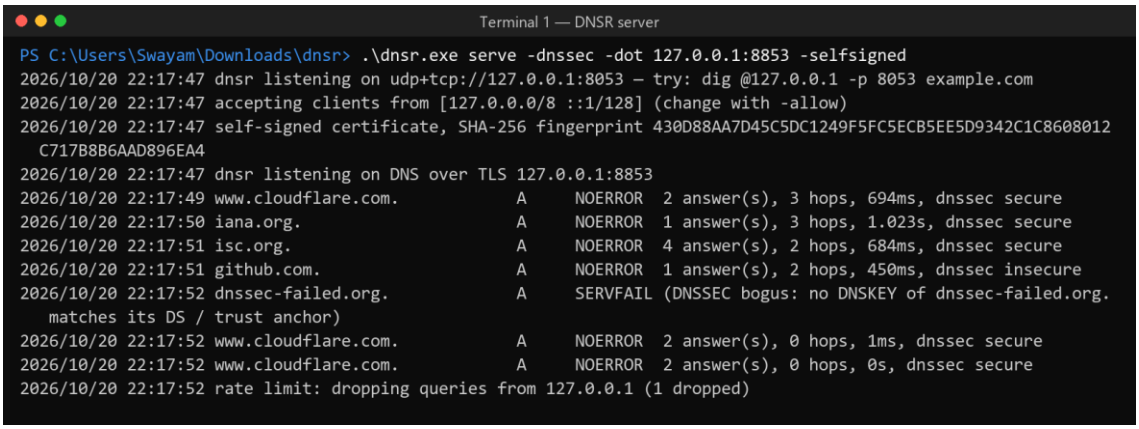


Fig. 6.15 DNSR Running as a Validating Server with DNS over TLS

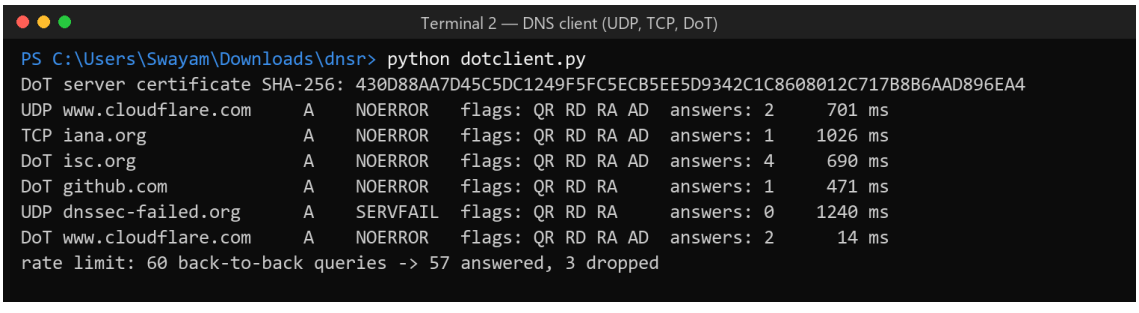


Fig. 6.16 An Independent Client Using UDP, TCP and DNS over TLS

Fig. 6.17 starts the server with `-allow 10.0.0.0/8`, so a query from 127.0.0.1 is outside the list. Fig. 6.18 shows the client side: the query is answered with REFUSED and the server logs the refusal once; a client limited to TLS 1.2 is rejected with a protocol-version alert, while a TLS 1.3 client connects.



Fig. 6.17 Server Started with an Access List

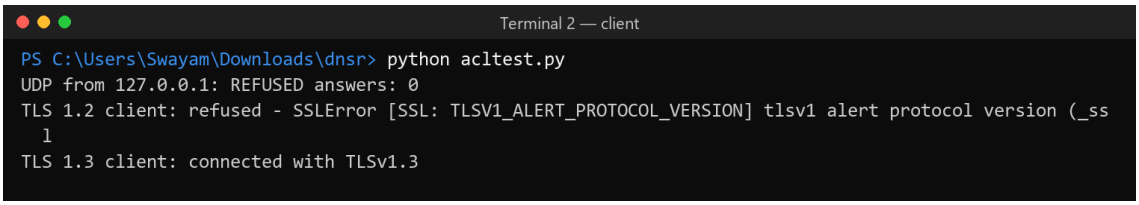


Fig. 6.18 Refused Client and TLS Version Enforcement

6.2.12 Fuzzing and benchmarks

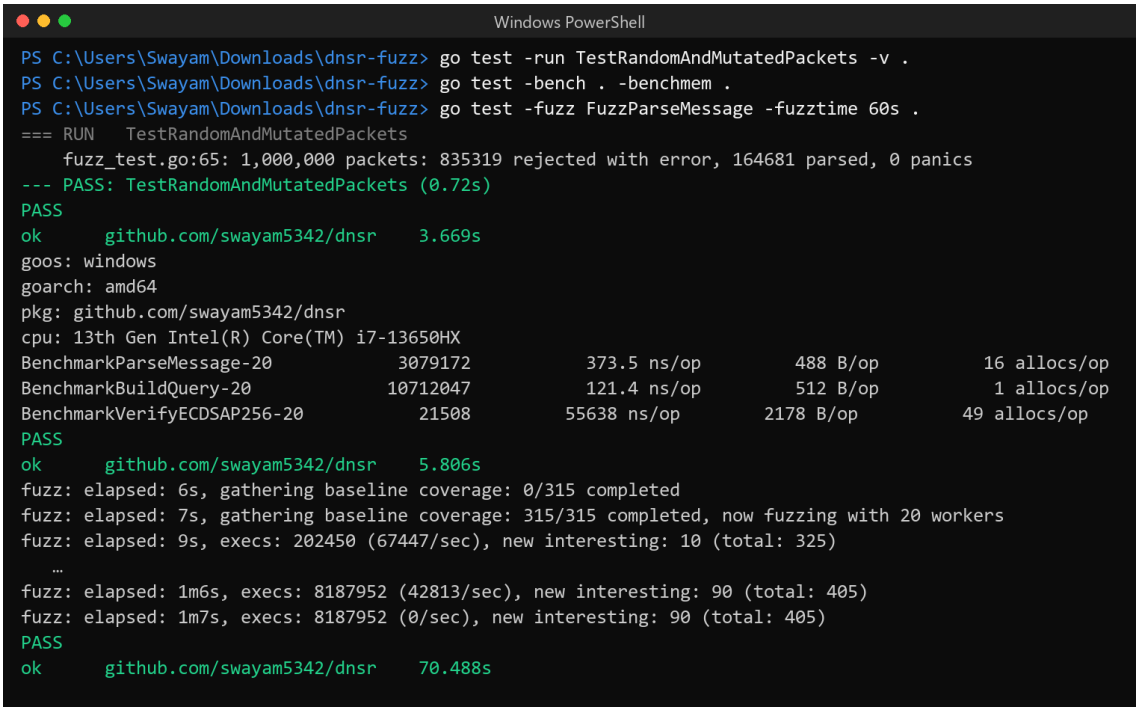


Fig. 6.19 Random-Packet Test, Benchmarks and Coverage-Guided Fuzzing

CHAPTER 7

RESULT AND DISCUSSION

This chapter evaluates DNSR against the goals of Section 2.4. It reports functional test results, correctness and latency of live resolution compared with a public resolver, the cost of QNAME minimisation, the effect of the caches, DNSSEC validation of real domains, the behaviour of server mode, the security and robustness evaluation, performance of the codec and of signature checks, and the status of the findings of a security review. It ends with a short cost analysis and a discussion.

7.1 Experimental Setup

All experiments were run on the machine described in Table 3.3, connected to the Internet over a home broadband link in India, on 20 October 2026. For the live experiments a measurement script started a fresh DNSR process for each run (so every run began with empty caches), parsed the summary line of the trace, and compared the answers with those of Google Public DNS (8.8.8.8) queried through dnspython. Fifteen popular domains were chosen to cover different TLDs (`.com`, `.org`, `.in`, `.io`, `.dev`), different DNS hosting providers and both direct answers and CNAME chains. Each domain was resolved three times. Because single runs are occasionally delayed by a server that does not answer within the timeout, results are summarised by the median of the runs, with the full range shown in the figures. The latency of one resolution is the wall-clock time reported by DNSR; the mean of a set of n resolutions, used for overall figures, is given by Eqn. 7.1.

$$L = \frac{1}{n}\sum_{i=1}^{n} L_i \tag{7.1}$$

7.2 Functional Verification

The offline suite of Section 4.1.13 was run with `go test -v` and again with the race detector (`go test -race`). All 32 tests passed in both runs and no data race was reported; the whole suite, including starting the six fake servers and generating fresh signing keys, completes in about three seconds. Table 7.1 groups the tests by the property they verify.

Table 7.1 Offline Test Results

Group	Tests	Result
Iterative resolution, CNAME, negative answers, MX, compression, TCP fallback	7	All PASS
Poisoning and redirection (poisoned glue, out-of-bailiwick referral)	2	All PASS
Reply matching (forged UDP reply, mismatching TCP reply)	2	All PASS
Randomness (cryptographic IDs, 0x20 case use, normalised answers, fall-back)	3	All PASS
Resource limits (TTL cap, bounded caches, expired delegation, glueless limit)	4	All PASS
QNAME minimisation (on and off)	2	All PASS
DNSSEC (secure, tampered record, wrong anchor, insecure delegation, RRSIG parsing)	5	All PASS
Server (AD and SERVFAIL, truncation, DNSSEC record stripping, rate limit, access list, DNS over TLS, TLS 1.3 only)	7	All PASS
Total (go test -v and go test -race)	32	32 PASS, 0 FAIL, 0 SKIP

7.3 Live Resolution: Correctness and Latency

Table 7.2 summarises the live runs. Of the 45 cold-cache runs, 44 completed successfully; the one that did not (for `google.com`) failed because none of the four root servers tried answered within the two-second timeout, a transient network loss, and the other runs for the same domain succeeded. For all 15 domains DNSR returned the same response code as Google Public DNS (15 of 15), and for 11 domains the returned addresses overlapped with those of 8.8.8.8. The exceptions, `google.com`, `microsoft.com`, `gmail.com`, `netflix.com`, are served by global content-delivery networks that return different addresses depending on where the question comes from; DNSR asks the authoritative servers from the user's own network, while 8.8.8.8 asks from Google's data centre, so different but equally valid addresses are expected.

Table 7.2 Live Cold-Cache Resolution of 15 Domains (Three Runs Each)

Domain	Runs OK	Hops	Median (ms)	Min to max (ms)	A records	Same as 8.8.8.8
google.com	2/3	3	7519	1829 to 13209	6	Code yes, IP differs (CDN)
www.github.com *	3/3	4	2208	1979 to 5072	1	Yes
www.wikipedia.org *	3/3	5	1176	943 to 4045	1	Yes
amazon.com	3/3	3	425	312 to 559	3	Yes

Domain	Runs OK	Hops	Median (ms)	Min to max (ms)	A records	Same as 8.8.8.8
vit.ac.in	3/3	4	1195	1151 to 1596	1	Yes
microsoft.com	3/3	3	338	258 to 398	1	Code yes, IP differs (CDN)
cloudflare.com	3/3	3	417	367 to 539	2	Yes
stackoverflow.com	3/3	3	555	467 to 2674	1	Yes
example.com	3/3	3	402	309 to 596	2	Yes
gmail.com	3/3	3	523	475 to 608	1	Code yes, IP differs (CDN)
netflix.com	3/3	3	374	344 to 400	3	Code yes, IP differs (CDN)
iana.org	3/3	3	398	372 to 753	1	Yes
python.org	3/3	3	244	160 to 460	4	Yes
go.dev	3/3	3	866	808 to 1156	4	Yes
github.io	3/3	3	927	832 to 931	4	Yes

* answer reached through a CNAME. Hops count the queries in the main resolution, including minimised probes; glueless sub-resolutions (for example for `github.io` and `vit.ac.in`) are additional.

Across all 44 successful runs the median cold-cache latency was 557 ms. Fig. 7.1 shows the median and range for each domain. The distribution is strongly skewed: about half of the runs finish in 250 to 700 ms, while five take more than two seconds. The slow runs coincide with the two-second timeout: when the randomly chosen server does not answer, DNSR waits the full timeout before trying the next one. This is the price of the sequential, polite server-selection strategy discussed in Section 4.3.3, and it is the main motivation for RTT-based server selection as future work.

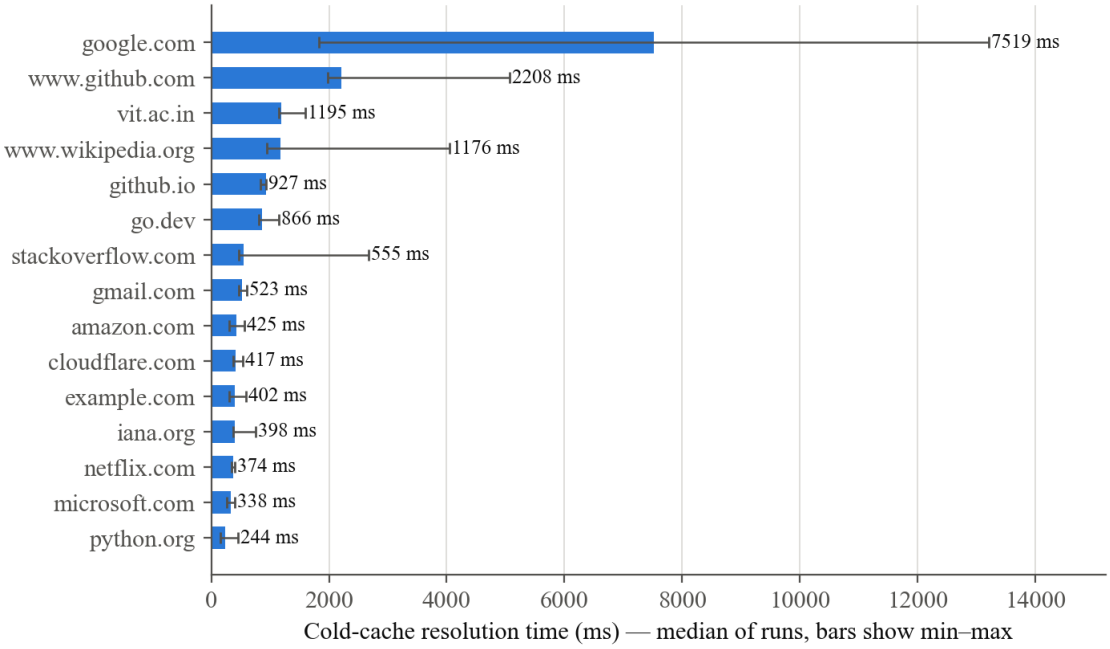


Fig. 7.1 Cold-Cache Resolution Time per Domain

7.4 Cost of QNAME Minimisation

To measure what minimisation costs, each domain was also resolved once with `-nomin`. Fig. 7.2 compares the number of queries in the main resolution. For 14 of the 15 domains the count is identical, because every probe landed on a real zone cut: the probe that the TLD server receives replaces the full query it would otherwise have received, so privacy improves at no cost. For `vit.ac.in`, minimisation needed 4 queries instead of 3, because `ac.in.` is not a separate zone, so the probe for it returns NODATA and one more label is added. Over all domains the mean was 3.3 queries per name.

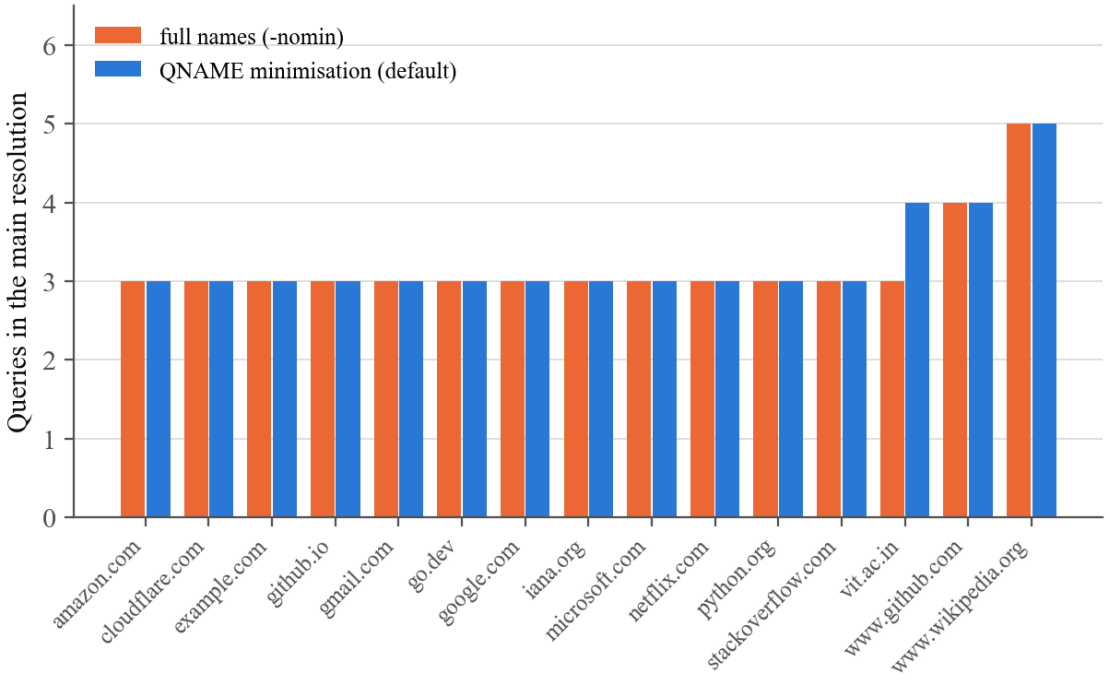


Fig. 7.2 Queries per Resolution With and Without QNAME Minimisation

7.5 Effect of Caching

To measure the two caches, five pairs of names were resolved within one process in the order: first name, a sibling name in the same domain, first name again. The first lookup starts with empty caches; the sibling lookup can use the delegation cache; the repeated lookup can use the answer cache. The relative reduction in latency is computed with Eqn. 7.2.

$$\rho = \frac{L_{cold} - L_{cached}}{L_{cold}} \times 100\% \tag{7.2}$$

Table 7.3 Latency with Empty, Delegation and Answer Caches (ms, hops in brackets)

First name / sibling	First (cold)	Sibling	ρ sibling	Repeat	ρ repeat
www.github.com / api.github.com	430 (4)	52 (1)	88%	98 (1)	77%
google.com / mail.google.com	721 (3)	134 (1)	81%	0 (0)	100%
www.wikipedia.org / en.wikipedia.org	769 (5)	128 (1)	83%	396 (1)	49%
vit.ac.in / www.vit.ac.in	1511 (4)	88 (1)	94%	0 (0)	100%
amazon.com / www.amazon.com	665 (3)	860 (4)	-29%	0 (0)	100%

The results, plotted in Fig. 7.3, show that the delegation cache removes the root and TLD steps and cuts the latency of a sibling name by 81 to 94 percent in four of the five cases. A repeated name that is not an alias is answered entirely from the answer

cache in under a millisecond (shown as 0 ms). Two observations reveal limitations of the current design. First, `www.amazon.com` took longer than the cold lookup of `amazon.com`, because it is a CNAME to a name in another provider's zone, which requires a new walk from the TLD. Second, a repeated lookup of an alias such as `www.github.com` still costs one query, because the CNAME record itself is not stored in the answer cache; only the final RRset of the target is cached. Caching CNAME records under the alias name would remove this last query.

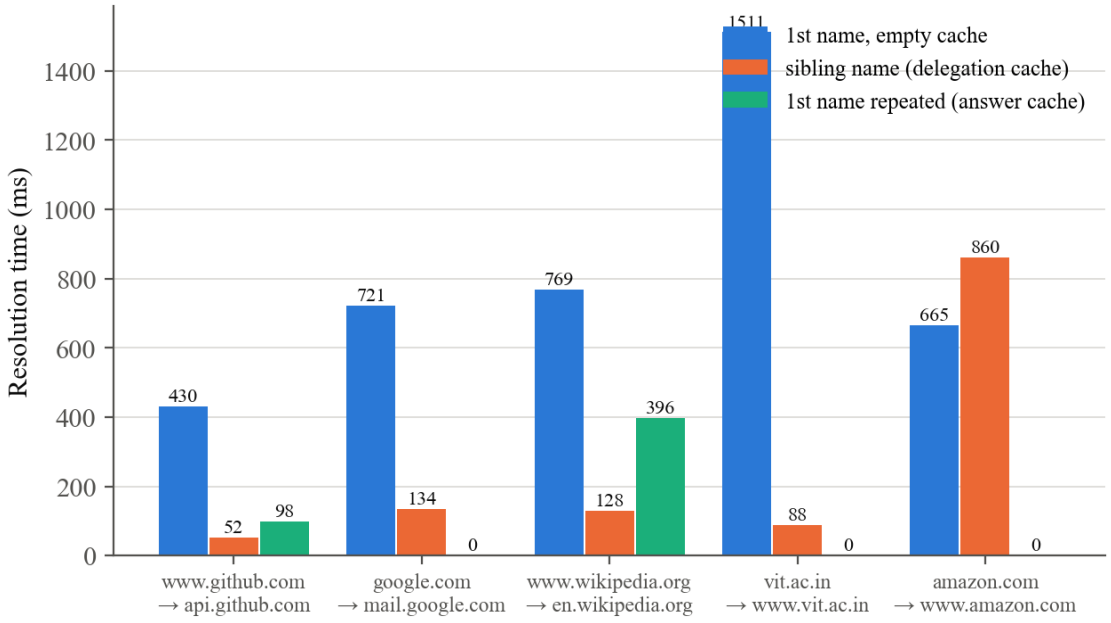


Fig. 7.3 Effect of the Delegation Cache and the Answer Cache

7.6 Record Types and Special Cases

Table 7.4 confirms that each supported record type, including the DNSSEC types DS and DNSKEY, is resolved and parsed from real servers. The PTR query for `8.8.8.8.in-addr.arpa` walks the reverse tree and returns `dns.google.`.

Table 7.4 Live Results for Each Supported Record Type

Type	Name queried	Records	Hops	Example record
A	example.com	2	3	172.66.147.243
AAAA	www.wikipedia.org	1	5	2001:df2:e500:ed1a::1
MX	gmail.com	5	3	5 gmail-smtp-in.l.google.com.
NS	vit.ac.in	4	4	ns-1067.awsdns-05.org.
TXT	google.com	17	3	"v=spf1 include:_spf.google.com ~all"
SOA	google.com	1	3	ns1.google.com. dns-admin.google.com. 100...
CNAME	www.github.com	1	3	github.com.

Type	Name queried	Records	Hops	Example record
PTR	8.8.8.8.in-addr.arpa	1	6	dns.google.
DS	cloudflare.com	1	2	2371 13 2 32996839A6D808AFE3EB4A795A0E6A7...
DNSKEY	iana.org	2	3	256 3 13 6IMTP24mO+zsJ7JYnvigaonVC8vJMUdD...

The special cases required by goal G3 were all observed live as well as offline: CNAME chains (`www.github.com`, `www.wikipedia.org`), NXDOMAIN (a non-existent `.com` name), NODATA (AAAA for `github.com`), glueless delegation (`github.io`, `vit.ac.in`) and TCP fallback (TXT for `apple.com`).

7.7 DNSSEC Validation of Real Domains

Eleven domains were resolved three times without and three times with `-dnssec`, each in a fresh process. Seven are signed with different algorithms and hosting providers, three are unsigned, and `dnssec-failed.org` is deliberately mis-signed by its operator to test validators. The experiment was run twice; the first run was disturbed by packet loss on the network, which inflated several timings, so the timings below come from the second run. The validation result was identical in all 66 validations of both runs. Table 7.5 lists the results and Fig. 7.4 compares the latencies; the extra time with validation is Δ = L_dnssec − L_plain (Eqn. 7.3).

$$\Delta = L_{dnssec} - L_{plain} \tag{7.3}$$

Table 7.5 DNSSEC Validation Results (Median of Three Cold-Cache Runs)

Domain	Result	Plain (ms)	With -dnssec (ms)	Δ (ms)	Reason
cloudflare.com	secure	251	855	604	chain verified from the root key
www.cloudflare.com	secure	208	1007	799	chain verified from the root key
iana.org	secure	356	1026	670	chain verified from the root key
isc.org	secure	406	981	575	chain verified from the root key
www.ietf.org	secure	567	999	432	chain verified from the root key
example.com	secure	281	1217	936	chain verified from the root key
nic.cz	secure	355	722	367	chain verified from the root key

Domain	Result	Plain (ms)	With -dnssec (ms)	Δ (ms)	Reason
github.com	insecure	280	477	197	no DS at the delegation
google.com	insecure	314	533	219	no DS at the delegation
vit.ac.in	insecure	3186	5102	1916	no DS at the delegation
dnssec-failed.org	bogus	2108	4230	2122	no DNSKEY matches the DS

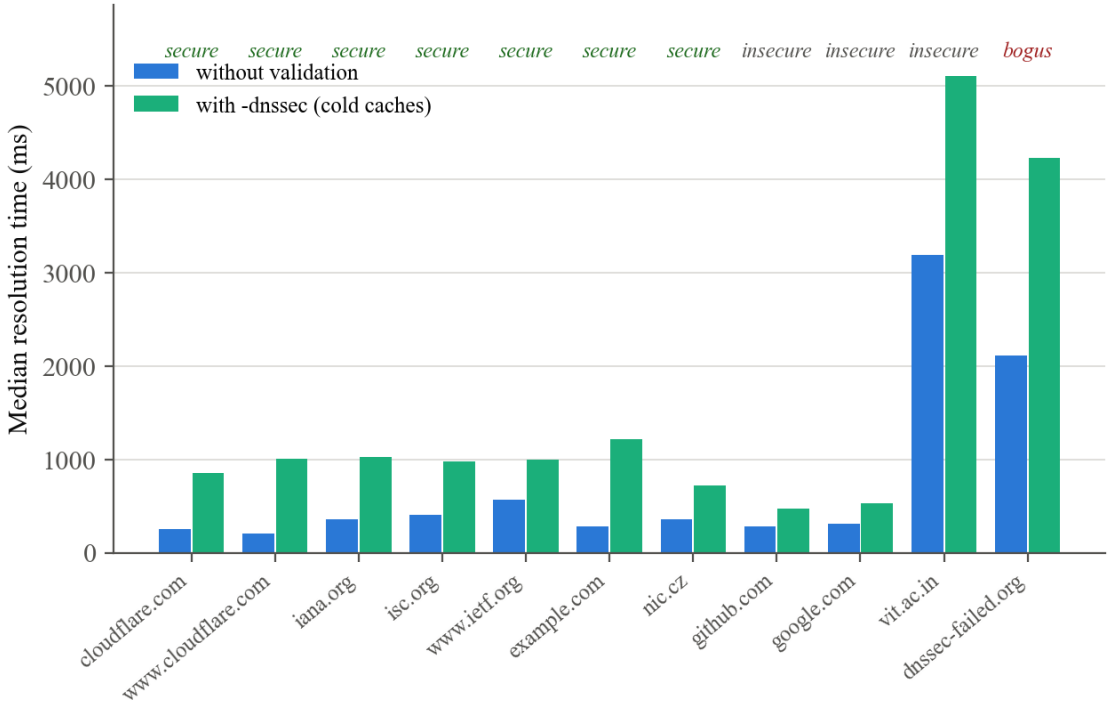


Fig. 7.4 Resolution Time Without and With DNSSEC Validation

Every signed domain validated as secure, the three unsigned domains were correctly reported as insecure, and the deliberately broken domain was rejected as bogus because no key in its DNSKEY set matches the DS record published in `.org`. This holds across RSA (root, `.com`, `.org`, `iana.org`) and ECDSA zones (`cloudflare.com`, `isc.org`), and across a zone cut at `www.cloudflare.com` that its operator signs as a separate zone. With empty caches, validation of a signed domain added a median of 604 ms, the cost of fetching the DS and DNSKEY records along the chain, each of which is a full lookup. Unsigned domains pay only for the DS query that shows the delegation is unsigned. Because validated keys are cached per zone, these costs are paid once: in server mode the second query for a signed name was answered in a few milliseconds (Section 7.8). Verifying the signatures themselves is negligible (Section 7.10).

7.8 Server-Mode Evaluation

DNSR was started with `dnsr serve -dnssec -addr 127.0.0.1:8053 -dot 127.0.0.1:8853 -selfsigned` and queried twice for each of eight questions over UDP by dnspython. Table 7.6 and Fig. 7.5 show the client-observed latency and flags. Every response was a valid DNS message that dnspython decoded without error.

Table 7.6 Server Mode over UDP: First and Repeated Query

Question	Status	Flags	First (ms)	Repeat (ms)	Answers
example.com A	NOERROR	QR RD RA AD	1031	4.7	2
www.github.com A	NOERROR	QR RD RA	608	37.9	2
gmail.com MX	NOERROR	QR RD RA	292	4.7	5
vit.ac.in NS	NOERROR	QR RD RA	1187	4.1	4
nosuchname-dnsr-vit.org A	NXDOMAIN	QR RD RA	219	29.5	0
www.wikipedia.org AAAA	NOERROR	QR RD RA	824	264.0	2
iana.org A	NOERROR	QR RD RA AD	689	2.8	1
dnssec-failed.org A	SERVFAIL	QR RD RA	983	2.3	0

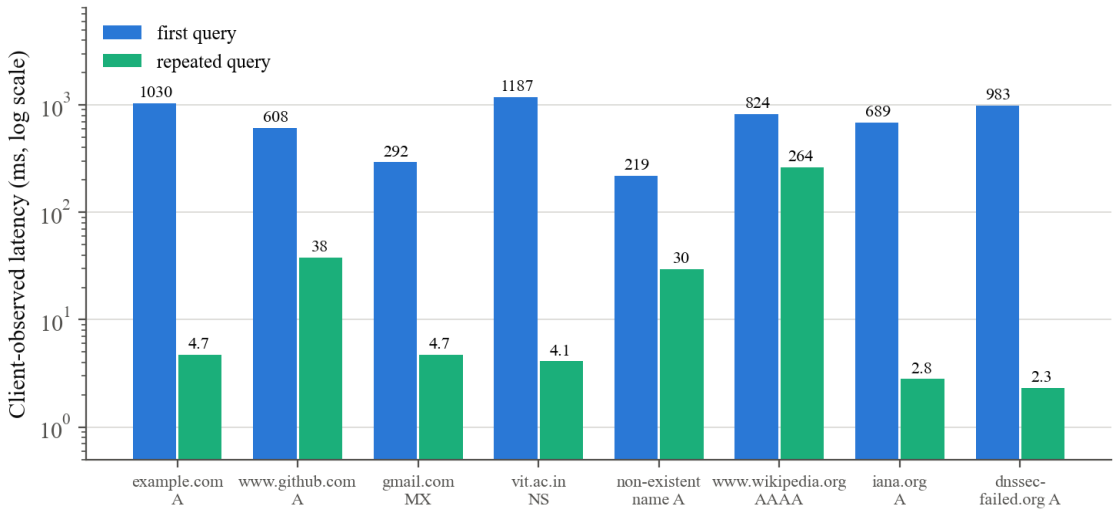


Fig. 7.5 Server Mode: Latency of First and Repeated Queries

Repeated queries for plain names were answered from the cache in 2 to 5 ms, a reduction of more than 99 percent. Secure answers (`example.com`, `iana.org`) carry the AD flag, unsigned ones do not, and `dnssec-failed.org` receives SERVFAIL both times. Repeated queries for aliases still needed one upstream query, and a repeated query for a non-existent name still went upstream because negative answers are not

cached. Over DNS over TLS (Table 7.7), first queries took 609, 228, 948 ms and repeated queries 10, 12, 10 ms; the repeats are a few milliseconds slower than over UDP because each query opens a new TLS connection. Finally, 100 back-to-back queries from one client were sent with a 0.3-second client timeout: 92 were answered and 8 were dropped by the rate limiter once the burst of 40 was exhausted and the refill rate of 20 per second could not keep up, with a single log line recording the drops. With `-allow 10.0.0.0/8`, a query from 127.0.0.1 was answered with REFUSED, and a client restricted to TLS 1.2 could not connect to the DNS over TLS port while a TLS 1.3 client could (Section 6.2.11).

Table 7.7 Server Mode over DNS over TLS

Question	Status	Flags	First (ms)	Repeat (ms)
isc.org A	NOERROR	QR RD RA AD	609	10.0
cloudflare.com A	NOERROR	QR RD RA AD	228	12.2
nic.cz A	NOERROR	QR RD RA AD	948	9.7

During this evaluation one defect was found and fixed. In validating mode, the authority section of an upstream NXDOMAIN reply carries NSEC3 and RRSIG records, and the first version of the server passed them to every client; the resulting response exceeded 512 bytes and was truncated for clients without EDNS. The server now returns DNSSEC records only to clients that set the DO bit, and a regression test (`TestServerStripsDNSSECRecordsForPlainClients`) covers the case.

7.9 Security and Robustness Evaluation

7.9.1 Poisoning, redirection and forgery

The adversarial tests passed: DNSR ignored the poisoned glue `ns.dnshost.org A 6.6.6.6` offered by the `com.` server and resolved the real address of the name server through `.org`; it rejected the referral from `com.` to `net.` with an out-of-bailiwick error; it ignored a forged reply with the wrong ID that arrived before the genuine one; it rejected a TCP reply for a different question; it still resolved names against a server that ignores letter case, by falling back from 0x20; and with DNSSEC enabled it reported a record altered after signing as bogus. In the live runs, every referral received passed the bailiwick check, showing that the check does not reject legitimate delegations.

7.9.2 Resistance to blind spoofing

Fig. 7.6 plots Eqn. 4.1 for the guess spaces of Eqns. 4.2 and 4.3. With the transaction ID alone, an attacker needs about 45,000 forged packets that arrive in time for a 50 percent chance of success. With the random ephemeral port that DNSR obtains by opening a fresh, connected UDP socket for every query, the same probability needs about 7.4×10^8 packets, which is impractical within a two-second query window on ordinary links. With 0x20 on a 13-letter name such as `www.example.com` it rises to about 6×10^{12}. In the live runs every one of the authoritative servers contacted for the 15 domains preserved letter case, so 0x20 was in effect for all of them and the fall-back was never needed. The answer filtering and bailiwick checks further reduce what even a successful forgery could inject, and for signed zones DNSSEC validation detects it outright.

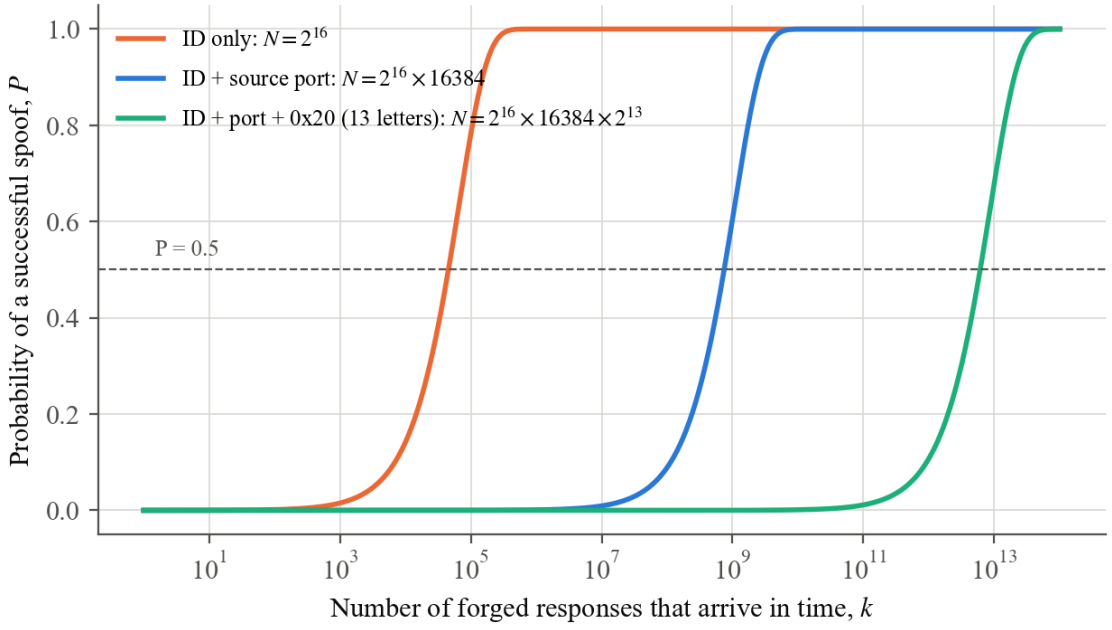


Fig. 7.6 Probability of a Successful Blind Spoof as a Function of Forged Packets

7.9.3 Resource limits

The limits added after the security review were verified by tests: a record served with a TTL of 4,000,000,000 seconds (about 127 years) was returned and cached with a TTL of at most one day; inserting 50 entries into caches limited to five left at most five entries; an expired delegation was not used; and a referral to eight non-existent glueless name servers caused at most three of them to be looked up before the resolution failed, instead of all eight.

7.9.4 Parser robustness under fuzzing

Two fuzzing experiments were run against `ParseMessage` and the DNSSEC record parsers (Section 6.2.12). In the first, one million packets were generated with a fixed seed: half were random byte strings of up to 600 bytes, and half were copies of a valid query, a valid reply or a signed reply with one to six bytes changed and, in a quarter of the cases, cut short. In the second, Go's native coverage-guided fuzzer [23] ran for 60 seconds on 20 workers. The results are summarised in Table 7.8 and Fig. 7.7.

Table 7.8 Fuzzing Results

Experiment	Inputs	Outcome
Random and mutated packets (fixed seed)	1,000,000 in 0.72 s	835,319 rejected with an error, 164,681 parsed, 0 panics, 0 hangs
Go coverage-guided fuzzing, 60 s, 20 workers	8,187,952 executions	90 new interesting inputs (corpus 405), no failure found

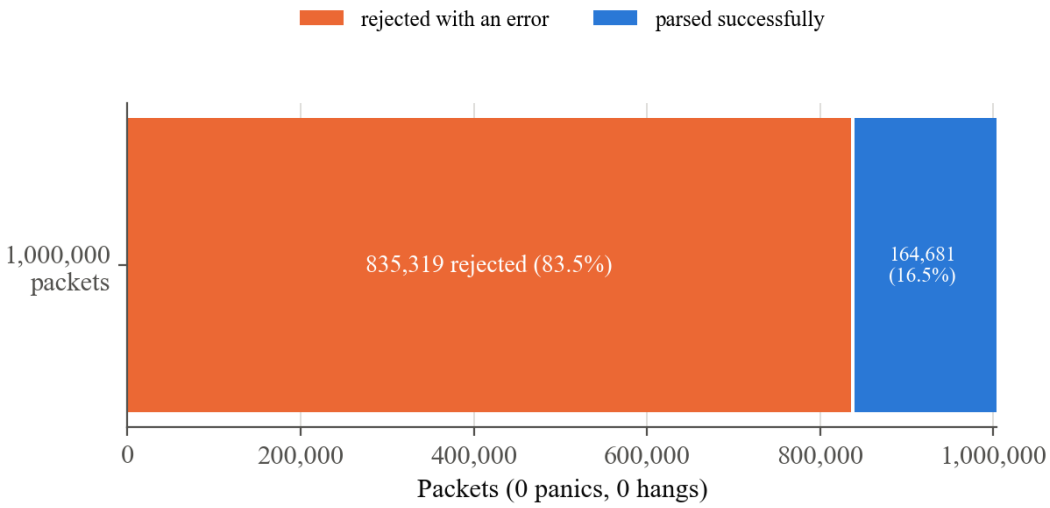


Fig. 7.7 Outcome of One Million Random and Mutated Packets

Not a single input caused a panic, an out-of-range access or a hang. The high proportion of rejected packets is the intended behaviour: inconsistent counts, truncated records, over-long names and invalid label types are reported as errors rather than being partially accepted. Pointer loops, which are specifically tested by `TestNameCompression`, are cut off by the 64-jump limit.

7.10 Codec and Signature Performance

The cost of the codec and of signature verification was measured with Go benchmarks (Table 7.9). Parsing a typical one-answer reply takes about 0.37 microseconds and building a query about 0.12 microseconds. Verifying an ECDSA P-256 signature over an RRset, including reconstructing the signed data, takes about 56 microseconds. A complete validation of a domain needs only a handful of such checks, a fraction of a millisecond in total, so both parsing and validation are negligible next to network round trips of tens to hundreds of milliseconds.

Table 7.9 Benchmark Results (Intel Core i7-13650HX)

Benchmark	Iterations	Time per operation	Memory per operation	Allocations
BenchmarkParseMessage	3,079,172	373.5 ns	488 B	16
BenchmarkBuildQuery	10,712,047	121.4 ns	512 B	1
BenchmarkVerifyECDSAP256	21,508	55,638 ns	2,178 B	49

7.11 Security Review Findings and Their Status

In addition to testing, the source code of the first version was reviewed for weaknesses that the tests did not cover. Table 7.10 lists the findings and their status in the final version. Eight of the nine were fixed and one was partly addressed. Three further measures were added after the review: query IDs from the cryptographic random source, a client access list and TLS 1.3 only for DNS over TLS.

Table 7.10 Findings of the Security Review

No.	Finding	Risk	Status in final version
1	No DNSSEC validation	An on-path attacker or compromised server can forge answers	Partly fixed: positive answers validated; NSEC/NSEC3 proofs future work
2	No upper limit on TTL	A poisoned record could stay cached for the life of the process	Fixed: TTLs capped at one day (TestTTLIsCapped)
3	Caches never shrink; delegations never expire	Memory exhaustion; stale delegations	Fixed: 10,000 entries per cache; delegations expire (two tests)
4	Every NS name of a glueless referral may be resolved	Query amplification (NXNSAttack [24])	Fixed: at most three lookups (TestGluelessLookupsAreLimited)
5	One goroutine per packet; no rate control; no access control	Resource exhaustion; open-resolver and reflection abuse	Fixed: 256 in flight, per-client rate limit, access list (TestServerRateLimit, TestServerAccessList)

No.	Finding	Risk	Status in final version
6	TCP reply checked only by ID	Weaker matching than on UDP	Fixed: shared matchesQuery check (TestTCPReplyMustMatchQuestion)
7	No 0x20 case randomisation	Less entropy against spoofing than possible	Fixed: 0x20 with per-server fall-back [28] (three tests)
8	Raw names printed in the trace	Terminal escape injection	Fixed: control characters escaped
9	Default server port 5353 clashes with mDNS on Windows	Server mode fails to start	Fixed: default port 8053

7.12 Achievement of Goals

Table 7.11 Goals and Results

ID	Goal	Result	Status
G1	Correct wire-format codec	All eleven record types parsed from live replies; compression test passes	Achieved
G2	Complete iterative resolution	15 of 15 domains give the same response code as 8.8.8.8; 44 of 45 runs succeeded	Achieved
G3	Correct special cases	CNAME, NXDOMAIN, NODATA, glueless and TCP fallback shown offline and live	Achieved
G4	Spoofing and poisoning defences	Poisoned glue ignored; out-of-bailiwick referral rejected; forged reply ignored; spoof space about 10^9, or 10^{13} with 0x20	Achieved
G5	Caching for performance	Sibling names 81 to 94% faster; repeated names under 1 ms (non-alias)	Achieved
G6	Two modes of operation	CLI trace and quiet mode; server answers an independent client over UDP, TCP and TLS	Achieved
G7	Robust parser	About 9.2 million fuzz inputs, no panic or hang	Achieved
G8	Small and dependency-free	2,205 lines of Go plus 900 lines of tests; standard library only	Achieved
G9	DNSSEC validation	7 signed domains secure, 3 unsigned insecure, 1 broken domain bogus, in all runs	Achieved for positive answers
G10	Privacy	Upstream servers see minimised names; DNS over TLS for clients	Achieved
G11	Resilient server	Access list, rate limit, concurrency cap, bounded caches and TTL cap verified; DoT limited to TLS 1.3	Achieved

7.13 Cost Analysis

The project has no direct financial cost. All software used is free and open source, and the hardware was the student's existing laptop. Table 7.12 summarises the resources.

Table 7.12 Cost of Resources

Resource	Item	Cost (INR)
Language and tools	Go tool chain, gofmt, go vet, go test, race detector	0
Libraries	Go standard library only (including crypto and TLS)	0
Validation tools	Python 3, dnspython, matplotlib	0
Infrastructure	Public root, TLD and authoritative DNS servers; Google Public DNS for comparison	0
Hardware	Existing laptop (Intel Core i7-13650HX)	0 (already owned)
Total		0

Running DNSR as a service would cost only the compute of a very small virtual machine, since the resolver uses a few megabytes of memory and negligible processor time; the network round trips, not computation, determine its performance. A publicly trusted certificate for DNS over TLS can be obtained free of charge from a certificate authority such as Let's Encrypt.

7.14 Discussion

The results show that a resolver of about two thousand lines can resolve real-world names correctly, including the awkward cases (glueless delegations across TLDs, CNAMEs into other zones, truncated replies, zone cuts that are not obvious from the name), while enforcing the checks that defeat classic cache-poisoning attacks and validating DNSSEC signatures against the root trust anchor. The trace output makes each of these checks visible, which was one of the main aims of the project. The evaluation also showed the value of testing against the real Internet: the root zone's change to a new key during the project would have broken validation completely had only the older trust anchor been built in.

The evaluation also exposed the limits of the design. Latency is dominated by the occasional unresponsive server, because servers are tried one at a time with a two-second timeout; production resolvers track the round-trip time of each server and prefer fast ones. The answer cache does not store CNAMEs or negative answers, so aliases and non-existent names are never fully served from cache. DNSSEC validation covers positive answers only: without NSEC and NSEC3 proofs, an attacker on the path could strip DS records to make a signed zone look unsigned, and negative answers are not authenticated. These points define the next steps, which are listed in Chapter 8.

CHAPTER 8

SUMMARY

This project designed, implemented and evaluated DNSR, a security-hardened iterative DNS resolver written from scratch in Go using only the standard library. DNSR encodes and decodes the RFC 1035 wire format by hand, resolves names starting from the IANA root hints with QNAME minimisation, follows referrals with glue, resolves glueless delegations, chases CNAMEs, distinguishes NXDOMAIN from NODATA and retries truncated replies over TCP. It validates answers with DNSSEC from the root trust anchors, keeps bounded caches of answers, delegations and keys, and can be used either as a command-line tool that prints a step-by-step trace or as a rate-limited recursive server over UDP, TCP and DNS over TLS.

8.1 Findings and Results

- ✓ **Correctness:** DNSR resolved all 15 test domains from the root with the same response code as Google Public DNS; 44 of 45 cold runs succeeded and the addresses agreed except where content-delivery networks return location-dependent answers.
- ✓ **Performance:** a cold-cache resolution took a median of 557 ms and 3.3 queries on average. The delegation cache made sibling names 81 to 94 percent faster, and repeated names were served in under a millisecond.
- ✓ **Privacy:** QNAME minimisation hid the full name from root and TLD servers at no extra cost for 14 of 15 domains and one extra query for the other; clients can reach the server over DNS over TLS.
- ✓ **DNSSEC:** all seven signed domains validated as secure, three unsigned domains as insecure and a deliberately broken domain as bogus, consistently across 66 validations; validation with empty caches added a median of about 0.6 seconds, paid once per zone.
- ✓ **Security:** 32 offline tests confirmed that poisoned glue, out-of-bailiwick referrals, forged replies, mismatching TCP replies and tampered records are rejected, that clients outside the access list are refused, and that the resource limits hold. Cryptographic random IDs, fresh source ports and 0x20 letter case

raise the blind-spoofing effort to about 6×10^{12} forged packets for an even chance on a typical name.

- ✓ **Robustness and efficiency:** about 9.2 million random, mutated and coverage-guided inputs caused no panic or hang; parsing a reply costs about 0.37 microseconds and verifying a signature about 56 microseconds.

8.2 Recommendations

- ✓ Before deploying DNSR on a network, run it with `-dnssec`, list the client networks with `-allow`, and use a certificate from a trusted authority for DNS over TLS rather than `-selfsigned`.
- ✓ Cache CNAME records and negative answers (RFC 2308) to remove the remaining upstream queries for aliases and non-existent names.
- ✓ Add authenticated denial of existence (NSEC and NSEC3) so that missing DS records and negative answers can be proven, closing the downgrade gap described in Section 7.14.
- ✓ Use the trace output and the offline hierarchy as teaching material: adding a new malicious server or a re-signed zone to the test hierarchy is a simple way to demonstrate a specific attack and its defence.

8.3 Future Work

- ✓ **Authenticated denial of existence:** validate NSEC and NSEC3 records (RFC 4035, RFC 5155) for NXDOMAIN, NODATA and unsigned delegations.
- ✓ **Negative caching and CNAME caching:** store negative answers with the SOA minimum TTL and cache aliases under their own names.
- ✓ **RTT-based server selection and parallel queries:** keep a smoothed round-trip time per server, as Unbound does, to avoid slow or dead servers.
- ✓ **DNS Cookies:** add a lightweight shared secret with each server for further protection against spoofing [29].
- ✓ **IPv6 transport:** use AAAA glue to reach name servers over IPv6.
- ✓ **DNS over HTTPS:** accept queries over HTTPS in addition to DNS over TLS.

In conclusion, DNSR meets all of its goals. It shows that the essential security logic of a recursive resolver, unpredictable queries, strict reply matching, bailiwick

rules and a DNSSEC chain of trust, can be expressed clearly in a small program, tested offline against hostile and signed servers and verified against the live Internet. It provides a solid, transparent base for further study of DNS security.

CHAPTER 9

REFERENCES

[1] P. Mockapetris, "Domain Names – Concepts and Facilities," RFC 1034, IETF, Nov. 1987.

[2] P. Mockapetris, "Domain Names – Implementation and Specification," RFC 1035, IETF, Nov. 1987.

[3] P. Mockapetris and K. J. Dunlap, "Development of the Domain Name System," in Proc. ACM SIGCOMM '88, Stanford, CA, USA, 1988, pp. 123–133.

[4] R. Elz and R. Bush, "Clarifications to the DNS Specification," RFC 2181, IETF, Jul. 1997.

[5] M. Andrews, "Negative Caching of DNS Queries (DNS NCACHE)," RFC 2308, IETF, Mar. 1998.

[6] J. Damas, M. Graff, and P. Vixie, "Extension Mechanisms for DNS (EDNS(0))," RFC 6891, IETF, Apr. 2013.

[7] J. Dickinson, S. Dickinson, R. Bellis, A. Mankin, and D. Wessels, "DNS Transport over TCP – Implementation Requirements," RFC 7766, IETF, Mar. 2016.

[8] A. Hubert and R. van Mook, "Measures for Making DNS More Resilient against Forged Answers," RFC 5452, IETF, Jan. 2009.

[9] D. Kaminsky, "Black Ops 2008: It's the End of the Cache as We Know It," presented at Black Hat USA, Las Vegas, NV, USA, Aug. 2008.

[10] S. Son and V. Shmatikov, "The Hitchhiker's Guide to DNS Cache Poisoning," in Proc. SecureComm 2010, LNICST, vol. 50, Springer, 2010, pp. 466–483.

[11] K. Man, Z. Qian, Z. Wang, X. Zheng, Y. Huang, and H. Duan, "DNS Cache Poisoning Attack Reloaded: Revolutions with Side Channels," in Proc. ACM CCS '20, 2020, pp. 1337–1350.

[12] J. Jung, E. Sit, H. Balakrishnan, and R. Morris, "DNS Performance and the Effectiveness of Caching," IEEE/ACM Trans. Networking, vol. 10, no. 5, pp. 589–603, Oct. 2002.

[13] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS Security Introduction and Requirements," RFC 4033, IETF, Mar. 2005.

[14] S. Bortzmeyer, R. Dolmans, and P. Hoffman, "DNS Query Name Minimisation to Improve Privacy," RFC 9156, IETF, Nov. 2021.

[15] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. Hoffman, "Specification for DNS over Transport Layer Security (TLS)," RFC 7858, IETF, May 2016.

[16] P. Hoffman and P. McManus, "DNS Queries over HTTPS (DoH)," RFC 8484, IETF, Oct. 2018.

[17] NLnet Labs, "Unbound – A Validating, Recursive, Caching DNS Resolver." [Online]. Available: https://nlnetlabs.nl/projects/unbound/

[18] Internet Systems Consortium, "BIND 9." [Online]. Available: https://www.isc.org/bind/

[19] M. Gieben, "dns – A DNS Library in Go." [Online]. Available: https://github.com/miekg/dns

[20] DNS Flag Day 2020. [Online]. Available: https://dnsflagday.net/2020/

[21] IANA, "Root Files (named.root)." [Online]. Available: https://www.iana.org/domains/root/files

[22] A. A. A. Donovan and B. W. Kernighan, The Go Programming Language. Boston, MA, USA: Addison-Wesley, 2015.

[23] The Go Authors, "Go Fuzzing." [Online]. Available: https://go.dev/doc/security/fuzz/

[24] Y. Afek, A. Bremler-Barr, and L. Shafir, "NXNSAttack: Recursive DNS Inefficiencies and Vulnerabilities," in Proc. 29th USENIX Security Symposium, 2020, pp. 631–648.

[25] D. Atkins and R. Austein, "Threat Analysis of the Domain Name System (DNS)," RFC 3833, IETF, Aug. 2004.

[26] P. Hoffman, A. Sullivan, and K. Fujiwara, "DNS Terminology," RFC 8499, IETF, Jan. 2019.

[27] S. Bortzmeyer and S. Huque, "NXDOMAIN: There Really Is Nothing Underneath," RFC 8020, IETF, Nov. 2016.

[28] D. Dagon, M. Antonakakis, P. Vixie, T. Jinmei, and W. Lee, "Increased DNS Forgery Resistance Through 0x20-Bit Encoding," in Proc. ACM CCS '08, 2008, pp. 211–222.

[29] D. Eastlake 3rd and M. Andrews, "Domain Name System (DNS) Cookies," RFC 7873, IETF, May 2016.

[30] S. Thomson, C. Huitema, V. Ksinant, and M. Souissi, "DNS Extensions to Support IP Version 6," RFC 3596, IETF, Oct. 2003.

[31] B. Halley et al., "dnspython – A DNS Toolkit for Python." [Online]. Available: https://www.dnspython.org/

[32] Microsoft, "The default dynamic port range for TCP/IP has changed since Windows Vista and in Windows Server 2008," Microsoft Learn, KB929851. [Online]. Available: https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/default-dynamic-port-range-tcpip-chang

[33] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "Resource Records for the DNS Security Extensions," RFC 4034, IETF, Mar. 2005.

[34] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "Protocol Modifications for the DNS Security Extensions," RFC 4035, IETF, Mar. 2005.

[35] IANA, "DNSSEC Trust Anchors (root-anchors.xml)." [Online]. Available: https://data.iana.org/root-anchors/root-anchors.xml

[36] J. Jansen, "Use of SHA-2 Algorithms with RSA in DNSKEY and RRSIG Resource Records for DNSSEC," RFC 5702, IETF, Oct. 2009.

[37] P. Hoffman and W. C. A. Wijngaards, "Elliptic Curve Digital Signature Algorithm (DSA) for DNSSEC," RFC 6605, IETF, Apr. 2012.

[38] O. Sury and R. Edmonds, "Edwards-Curve Digital Security Algorithm (EdDSA) for DNSSEC," RFC 8080, IETF, Feb. 2017.

[39] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, Aug. 2018.

APPENDIX A

COMPLETE SOURCE CODE

This appendix lists the complete source code of DNSR (module github.com/swayam5342/dnsr), followed by the fuzzing and benchmark harness used in Chapter 7.

A.1 wire.go

```
package main

// wire.go — RFC 1035 message format, hand-rolled. No net/dns helpers.
//
//   +---------------------+
//   |        Header       |  12 bytes: ID, flags, 4 counts
//   +---------------------+
//   |       Question      |  QNAME QTYPE QCLASS
//   +---------------------+
//   |        Answer       |  RRs
//   +---------------------+
//   |      Authority      |  RRs (NS records on a referral)
//   +---------------------+
//   |      Additional     |  RRs (glue A/AAAA for those NS)
//   +---------------------+

import (
    "encoding/base64"
    "encoding/binary"
    "errors"
    "fmt"
    "net"
    "strings"
    "time"
)

type RRType uint16

const (
    TypeA      RRType = 1
    TypeNS     RRType = 2
    TypeCNAME  RRType = 5
    TypeSOA    RRType = 6
    TypePTR    RRType = 12
    TypeMX     RRType = 15
    TypeTXT    RRType = 16
    TypeAAAA   RRType = 28
    TypeOPT    RRType = 41
    TypeDS     RRType = 43
    TypeRRSIG  RRType = 46
    TypeNSEC   RRType = 47
    TypeDNSKEY RRType = 48
    TypeNSEC3  RRType = 50
)

var typeNames = map[RRType]string{
    TypeA: "A", TypeNS: "NS", TypeCNAME: "CNAME", TypeSOA: "SOA", TypePTR: "PTR",
    TypeMX: "MX", TypeTXT: "TXT", TypeAAAA: "AAAA", TypeOPT: "OPT",
    TypeDS: "DS", TypeRRSIG: "RRSIG", TypeNSEC: "NSEC", TypeDNSKEY: "DNSKEY", TypeNSEC3: "NSEC3",
}

func (t RRType) String() string {
    if s, ok := typeNames[t]; ok {
        return s
    }
    return fmt.Sprintf("TYPE%d", uint16(t))
```

```
}

func ParseType(s string) (RRType, error) {
    s = strings.ToUpper(s)
    for t, n := range typeNames {
        if n == s {
            return t, nil
        }
    }
    return 0, fmt.Errorf("unknown record type %q", s)
}

const (
    RcodeNoError  = 0
    RcodeFormErr  = 1
    RcodeServFail = 2
    RcodeNXDomain = 3
    RcodeRefused  = 5
)

var rcodeNames = []string{"NOERROR", "FORMERR", "SERVFAIL", "NXDOMAIN", "NOTIMP", "REFUSED"}

func rcodeString(r int) string {
    if r < len(rcodeNames) {
        return rcodeNames[r]
    }
    return fmt.Sprintf("RCODE%d", r)
}

type Header struct {
    ID      uint16
    QR      bool
    Opcode  uint8
    AA      bool
    TC      bool
    RD      bool
    RA      bool
    AD      bool
    CD      bool
    Rcode   int
    QDCount uint16
    ANCount uint16
    NSCount uint16
    ARCount uint16
}

type Question struct {
    Name  string
    Type  RRType
    Class uint16
}

type RR struct {
    Name  string
    Type  RRType
    Class uint16
    TTL   uint32
    Data  string
    Raw   []byte
    Zone  string
}

func (r RR) String() string {
    return fmt.Sprintf("%-28s %6d IN %-5s %s", r.Name, r.TTL, r.Type, r.Data)
}

type EDNS struct {
    UDPSize uint16
    DO      bool
}

type Message struct {
    Header     Header
```

```
    Questions  []Question
    Answers    []RR
    Authority  []RR
    Additional []RR
    EDNS       *EDNS
}

func encodeName(buf []byte, name string) ([]byte, error) {
    name = strings.TrimSuffix(name, ".")
    if name != "" {
        for label := range strings.SplitSeq(name, ".") {
            if len(label) == 0 || len(label) > 63 {
                return nil, fmt.Errorf("bad label %q in %q", label, name)
            }
            buf = append(buf, byte(len(label)))
            buf = append(buf, label...)
        }
    }
    return append(buf, 0), nil
}

// BuildQuery makes a non-recursive query (RD=0) with an EDNS0 OPT record
// advertising a 1232-byte UDP buffer (the DNS Flag Day 2020 value).
// With dnssec set, the DO bit asks servers to include RRSIG records.
func BuildQuery(id uint16, name string, qtype RRType, dnssec bool) ([]byte, error) {
    b := make([]byte, 12, 512)
    binary.BigEndian.PutUint16(b[0:], id)
    binary.BigEndian.PutUint16(b[2:], 0)
    binary.BigEndian.PutUint16(b[4:], 1)
    binary.BigEndian.PutUint16(b[10:], 1)


    var err error
    if b, err = encodeName(b, name); err != nil {
        return nil, err
    }
    b = binary.BigEndian.AppendUint16(b, uint16(qtype))
    b = binary.BigEndian.AppendUint16(b, 1)


    b = append(b, 0)
    b = binary.BigEndian.AppendUint16(b, uint16(TypeOPT))
    b = binary.BigEndian.AppendUint16(b, 1232)
    var ext uint32
    if dnssec {
        ext = 0x8000
    }
    b = binary.BigEndian.AppendUint32(b, ext)
    b = binary.BigEndian.AppendUint16(b, 0)
    return b, nil
}

var errShort = errors.New("message truncated")


// readName decodes a possibly-compressed name starting at off.
// Returns the name and the offset just past it in the *original* stream.
func readName(msg []byte, off int) (string, int, error) {
    var labels []string
    end, total := -1, 1
    for jumps := 0; ; {
        if off >= len(msg) {
            return "", 0, errShort
        }
        l := int(msg[off])
        switch {
        case l == 0:
            if end < 0 {
                end = off + 1
            }
            return strings.Join(labels, ".") + ".", end, nil
        case l&0xC0 == 0xC0:
            if off+1 >= len(msg) {
                return "", 0, errShort
            }
            if end < 0 {
                end = off + 2
            }
            jumps++
```

```
            if jumps > 64 {
                return "", 0, errors.New("compression pointer loop")
            }
            off = int(binary.BigEndian.Uint16(msg[off:]) & 0x3FFF)
        case l&0xC0 != 0:
            return "", 0, fmt.Errorf("unsupported label type 0x%x", l)
        default:
            off++
            if off+l > len(msg) {
                return "", 0, errShort
            }
            if total += l + 1; total > 255 {
                return "", 0, errors.New("name longer than 255 bytes")
            }
            labels = append(labels, string(msg[off:off+l]))
            off += l
        }
    }
}

func readRR(msg []byte, off int) (RR, int, error) {
    var rr RR
    var err error
    if rr.Name, off, err = readName(msg, off); err != nil {
        return rr, 0, err
    }
    if off+10 > len(msg) {
        return rr, 0, errShort
    }
    rr.Type = RRType(binary.BigEndian.Uint16(msg[off:]))
    rr.Class = binary.BigEndian.Uint16(msg[off+2:])
    rr.TTL = binary.BigEndian.Uint32(msg[off+4:])
    rdlen := int(binary.BigEndian.Uint16(msg[off+8:]))
    off += 10
    if off+rdlen > len(msg) {
        return rr, 0, errShort
    }
    rd := msg[off : off+rdlen]
    rr.Raw = rd


    switch rr.Type {
    case TypeA:
        if rdlen == 4 {
            rr.Data = net.IP(rd).String()
        }
    case TypeAAAA:
        if rdlen == 16 {
            rr.Data = net.IP(rd).String()
        }
    case TypeNS, TypeCNAME, TypePTR:
        rr.Data, _, err = readName(msg, off)
    case TypeMX:
        if rdlen >= 3 {
            var host string
            host, _, err = readName(msg, off+2)
            rr.Data = fmt.Sprintf("%d %s", binary.BigEndian.Uint16(rd), host)
        }
    case TypeSOA:
        var mname, rname string
        var p int
        if mname, p, err = readName(msg, off); err == nil {
            if rname, p, err = readName(msg, p); err == nil && p+20 <= off+rdlen {
                rr.Data = fmt.Sprintf("%s %s %d %d %d %d %d", mname, rname,
                    binary.BigEndian.Uint32(msg[p:]), binary.BigEndian.Uint32(msg[p+4:]),
                    binary.BigEndian.Uint32(msg[p+8:]), binary.BigEndian.Uint32(msg[p+12:]),
                    binary.BigEndian.Uint32(msg[p+16:]))
            }
        }
    case TypeTXT:
        var parts []string
        for i := 0; i < len(rd); {
            n := int(rd[i])
            i++
            if i+n > len(rd) {
                break
            }
            parts = append(parts, fmt.Sprintf("%q", rd[i:i+n]))
            i += n
```

```
        }
        rr.Data = strings.Join(parts, " ")
    case TypeDS:
        if rdlen > 4 {
            rr.Data = fmt.Sprintf("%d %d %d %X", binary.BigEndian.Uint16(rd), rd[2], rd[3],
rd[4:])
        }
    case TypeDNSKEY:
        if rdlen > 4 {
            rr.Data = fmt.Sprintf("%d %d %d %s", binary.BigEndian.Uint16(rd), rd[2], rd[3],
base64.StdEncoding.EncodeToString(rd[4:]))
        }
    case TypeRRSIG:
        if s, perr := parseRRSIG(rd); perr == nil {
            ts := func(v uint32) string { return time.Unix(int64(v),
0).UTC().Format("20060102150405") }
            rr.Data = fmt.Sprintf("%s %d %d %d %s %s %d %s %s", s.TypeCovered, s.Algorithm,
s.Labels, s.OrigTTL,
                ts(s.Expiration), ts(s.Inception), s.KeyTag, s.Signer,
base64.StdEncoding.EncodeToString(s.Signature))
        }
    default:
        rr.Data = fmt.Sprintf("\\# %d %x", rdlen, rd)
    }
    if err != nil {
        return rr, 0, err
    }
    return rr, off + rdlen, nil
}

func ParseMessage(msg []byte) (*Message, error) {
    if len(msg) < 12 {
        return nil, errShort
    }
    m := &Message{}
    f := binary.BigEndian.Uint16(msg[2:])
    m.Header = Header{
        ID:      binary.BigEndian.Uint16(msg[0:]),
        QR:      f&0x8000 != 0,
        Opcode:  uint8(f>>11) & 0xF,
        AA:      f&0x0400 != 0,
        TC:      f&0x0200 != 0,
        RD:      f&0x0100 != 0,
        RA:      f&0x0080 != 0,
        AD:      f&0x0020 != 0,
        CD:      f&0x0010 != 0,
        Rcode:   int(f & 0xF),
        QDCount: binary.BigEndian.Uint16(msg[4:]),
        ANCount: binary.BigEndian.Uint16(msg[6:]),
        NSCount: binary.BigEndian.Uint16(msg[8:]),
        ARCount: binary.BigEndian.Uint16(msg[10:]),
    }
    off := 12
    for i := 0; i < int(m.Header.QDCount); i++ {
        name, p, err := readName(msg, off)
        if err != nil {
            return nil, err
        }
        if p+4 > len(msg) {
            return nil, errShort
        }
        m.Questions = append(m.Questions, Question{name,
            RRType(binary.BigEndian.Uint16(msg[p:])), binary.BigEndian.Uint16(msg[p+2:])})
        off = p + 4
    }
    sections := []struct {
        n   uint16
        dst *[]RR
    }{{m.Header.ANCount, &m.Answers}, {m.Header.NSCount, &m.Authority}, {m.Header.ARCount,
&m.Additional}}
    for _, s := range sections {
        for i := 0; i < int(s.n); i++ {
            rr, p, err := readRR(msg, off)
            if err != nil {
                return nil, err
            }
            if rr.Type == TypeOPT {
                m.EDNS = &EDNS{UDPSize: rr.Class, DO: rr.TTL&0x8000 != 0}
```

```
            } else {
                *s.dst = append(*s.dst, rr)
            }
            off = p
        }
    }
    return m, nil
}
```

A.2 resolver.go

```
package main

// resolver.go — iterative resolution, the way a real recursor does it:
//
//   root (.)  ──referral──▶  TLD (com.)  ──referral──▶  example.com. NS  ──▶ answer
//
// We start from hard-coded root hints, send non-recursive queries, and follow
// referrals (NS in Authority + glue in Additional) until someone answers
// authoritatively. CNAMEs are chased, glueless delegations are resolved
// with a nested lookup, and truncated UDP responses fall back to TCP.
// Queries are QNAME-minimised (RFC 9156): each server only sees one label
// more than the zone it serves, until the final server gets the full name.

import (
    crand "crypto/rand"
    "encoding/binary"
    "errors"
    "fmt"
    "io"
    "math/rand/v2"
    "net"
    "strings"
    "sync"
    "time"
)

var rootHints = []nameserver{
    {"a.root-servers.net.", "198.41.0.4"},
    {"b.root-servers.net.", "170.247.170.2"},
    {"c.root-servers.net.", "192.33.4.12"},
    {"d.root-servers.net.", "199.7.91.13"},
    {"e.root-servers.net.", "192.203.230.10"},
    {"f.root-servers.net.", "192.5.5.241"},
    {"g.root-servers.net.", "192.112.36.4"},
    {"h.root-servers.net.", "198.97.190.53"},
    {"i.root-servers.net.", "192.36.148.17"},
    {"j.root-servers.net.", "192.58.128.30"},
    {"k.root-servers.net.", "193.0.14.129"},
    {"l.root-servers.net.", "199.7.83.42"},
    {"m.root-servers.net.", "202.12.27.33"},
}

const (
    defaultMaxTTL      = 86400
    defaultCacheSize   = 10000
    maxGluelessLookups = 3
    maxMinimiseLabels  = 10
)

type nameserver struct {
    Name string
    IP   string
}

type Result struct {
    Rcode        int
    Answers      []RR
    Authority    []RR
    Sigs         []RR
    Hops         int
```

```
    Elapsed      time.Duration
    Security     Security
    SecurityNote string
}

type cacheEntry struct {
    rrs     []RR
    sigs    []RR
    expires time.Time
}

type zoneEntry struct {
    ns      []nameserver
    expires time.Time
}

type Resolver struct {
    Timeout           time.Duration
    MaxHops           int
    Trace             io.Writer
    Port              string
    Roots             []nameserver
    Minimise          bool
    DNSSEC            bool
    Anchors           []DSRecord
    MaxTTL            uint32
    MaxCacheEntries   int
    CaseRandomisation bool
    color             bool
    mu                sync.Mutex
    cache             map[string]cacheEntry
    zoneNS            map[string]zoneEntry
    keys              map[string]keyEntry
    no0x20            map[string]time.Time
}

func NewResolver() *Resolver {
    return &Resolver{
        Timeout:           2 * time.Second,
        MaxHops:           40,
        Port:              "53",
        Roots:             rootHints,
        Minimise:          true,
        Anchors:           rootAnchors,
        MaxTTL:            defaultMaxTTL,
        MaxCacheEntries:   defaultCacheSize,
        CaseRandomisation: true,
        cache:             map[string]cacheEntry{},
        zoneNS:            map[string]zoneEntry{},
        keys:              map[string]keyEntry{},
        no0x20:            map[string]time.Time{},
    }
}

var (
    ErrLoop     = errors.New("resolution loop / too deep")
    ErrNoServer = errors.New("all nameservers failed")
)

func fqdn(s string) string {
    s = strings.ToLower(strings.TrimSpace(s))
    if !strings.HasSuffix(s, ".") {
        s += "."
    }
    return s
}

// isSubdomain reports whether child is at or below parent ("a.b.c." under "b.c.").
func isSubdomain(child, parent string) bool {
    if parent == "." {
        return true
    }
    return child == parent || strings.HasSuffix(child, "."+parent)
}
```

```
// splitLabels returns the labels of a name, without the root ("www.example.com." → www, example,
com).
func splitLabels(name string) []string {
    name = strings.TrimSuffix(name, ".")
    if name == "" {
        return nil
    }
    return strings.Split(name, ".")
}


// parentName drops the leftmost label ("www.example.com." → "example.com.").
func parentName(name string) string {
    ls := splitLabels(name)
    if len(ls) <= 1 {
        return "."
    }
    return strings.Join(ls[1:], ".") + "."
}


// childToward returns the name one label below zone on the way to name
// ("www.example.com." from "com." → "example.com.").
func childToward(name, zone string) string {
    if name == zone || !isSubdomain(name, zone) {
        return name
    }
    nl, zl := splitLabels(name), len(splitLabels(zone))
    return strings.Join(nl[len(nl)-zl-1:], ".") + "."
}


// Resolve resolves name/qtype from the root and, when DNSSEC is enabled,
// validates the answer against the root trust anchor.
func (r *Resolver) Resolve(name string, qtype RRType) (*Result, error) {
    start := time.Now()
    res, err := r.resolve(fqdn(name), qtype, 0, 0)
    if err == nil && r.DNSSEC {
        res.Security, res.SecurityNote = r.validateResult(res)
        r.traceSecurity(res)
    }
    if res != nil {
        res.Elapsed = time.Since(start)
    }
    return res, err
}


const (
    cDim, cCyan, cGreen, cYellow, cRed, cBold, cReset = "\033[2m", "\033[36m", "\033[32m",
"\033[33m", "\033[31m", "\033[1m", "\033[0m"
)


func (r *Resolver) c(code, s string) string {
    if !r.color {
        return s
    }
    return code + printable(s) + cReset
}


// tracef prints one trace line; control characters from untrusted names are escaped.
func (r *Resolver) tracef(depth int, format string, a ...any) {
    if r.Trace == nil {
        return
    }
    for i, v := range a {
        if s, ok := v.(string); ok && !strings.HasPrefix(s, "\033") {
            a[i] = printable(s)
        }
    }
    fmt.Fprintf(r.Trace, "%s%s\n", strings.Repeat("  │ ", depth), fmt.Sprintf(format, a...))
}


// printable escapes bytes that could act as terminal control sequences.
func printable(s string) string {
    for i := 0; i < len(s); i++ {
        if s[i] < 0x20 || s[i] == 0x7f {
```

```
            var b strings.Builder
            for j := 0; j < len(s); j++ {
                if c := s[j]; c < 0x20 || c == 0x7f {
                    fmt.Fprintf(&b, "\\%03d", c)
                } else {
                    b.WriteByte(c)
                }
            }
            return b.String()
        }
    }
    return s
}

// clampTTLs caps every TTL at MaxTTL so no record can stay cached for longer.
func (r *Resolver) clampTTLs(rrs []RR) []RR {
    out := make([]RR, len(rrs))
    for i, rr := range rrs {
        if rr.TTL > r.MaxTTL {
            rr.TTL = r.MaxTTL
        }
        out[i] = rr
    }
    return out
}

func minTTL(rrs []RR) uint32 {
    m := rrs[0].TTL
    for _, rr := range rrs {
        if rr.TTL < m {
            m = rr.TTL
        }
    }
    return m
}

// evict makes room in a bounded cache: expired entries go first, then arbitrary ones.
func evict[V any](m map[string]V, max int, expired func(V) bool) {
    if len(m) < max {
        return
    }
    for k, v := range m {
        if expired(v) {
            delete(m, k)
        }
    }
    for k := range m {
        if len(m) < max {
            break
        }
        delete(m, k)
    }
}

func (r *Resolver) cacheGet(name string, t RRType) ([]RR, []RR, bool) {
    r.mu.Lock()
    defer r.mu.Unlock()
    e, ok := r.cache[name+"|"+t.String()]
    if !ok || time.Now().After(e.expires) {
        return nil, nil, false
    }
    left := uint32(time.Until(e.expires).Seconds())
    out := make([]RR, len(e.rrs))
    for i, rr := range e.rrs {
        rr.TTL = left
        out[i] = rr
    }
    return out, e.sigs, true
}

func (r *Resolver) cachePut(name string, t RRType, rrs, sigs []RR) {
    if len(rrs) == 0 {
        return
    }
    ttl := min(minTTL(rrs), r.MaxTTL)
    now := time.Now()
```

```
    r.mu.Lock()
    defer r.mu.Unlock()
    evict(r.cache, r.MaxCacheEntries, func(e cacheEntry) bool { return now.After(e.expires) })
    r.cache[name+"|"+t.String()] = cacheEntry{rrs, sigs, now.Add(time.Duration(ttl) *
time.Second)}
}

func (r *Resolver) putZone(zone string, ns []nameserver, ttl uint32) {
    ttl = min(ttl, r.MaxTTL)
    now := time.Now()
    r.mu.Lock()
    defer r.mu.Unlock()
    evict(r.zoneNS, r.MaxCacheEntries, func(e zoneEntry) bool { return now.After(e.expires) })
    r.zoneNS[zone] = zoneEntry{ns, now.Add(time.Duration(ttl) * time.Second)}
}

func (r *Resolver) closestKnownZone(name string) (string, []nameserver) {
    r.mu.Lock()
    defer r.mu.Unlock()
    now := time.Now()
    for z := name; ; {
        if e, ok := r.zoneNS[z]; ok && len(e.ns) > 0 && now.Before(e.expires) {
            return z, e.ns
        }
        if z == "." {
            return ".", r.Roots
        }
        z = parentName(z)
    }
}

// coveredSigs returns the RRSIGs that cover records of type t.
func coveredSigs(sigs []RR, t RRType) []RR {
    var out []RR
    for _, s := range sigs {
        if len(s.Raw) >= 2 && RRType(binary.BigEndian.Uint16(s.Raw)) == t {
            out = append(out, s)
        }
    }
    return out
}

// referralNS extracts the delegation from a reply: NS records in the authority
// section, or, for a minimised NS query, NS records for qname in the answer.
func referralNS(msg *Message, qname string, probing bool) ([]RR, string) {
    var out []RR
    var child string
    if probing {
        for _, rr := range msg.Answers {
            if rr.Type == TypeNS && strings.EqualFold(rr.Name, qname) {
                out, child = append(out, rr), strings.ToLower(rr.Name)
            }
        }
        if len(out) > 0 {
            return out, child
        }
    }
    for _, rr := range msg.Authority {
        if rr.Type == TypeNS {
            out, child = append(out, rr), strings.ToLower(rr.Name)
        }
    }
    return out, child
}

// unexpectedAnswer reports whether a reply to a minimised NS query contains
// anything other than NS records for the probed name (a CNAME, for example).
func unexpectedAnswer(msg *Message, qname string) bool {
    for _, rr := range msg.Answers {
        if rr.Type == TypeRRSIG {
            continue
        }
        if rr.Type != TypeNS || !strings.EqualFold(rr.Name, qname) {
            return true
        }
    }
```

```
    return false
}

func (r *Resolver) resolve(name string, qtype RRType, depth, hops int) (*Result, error) {
    if depth > 8 {
        return nil, ErrLoop
    }
    if rrs, sigs, ok := r.cacheGet(name, qtype); ok {
        r.tracef(depth, "%s %s %s", r.c(cGreen, "✓ cache hit"), name, qtype)
        return &Result{Rcode: RcodeNoError, Answers: rrs, Sigs: sigs, Hops: hops}, nil
    }

    start := name
    if qtype == TypeDS && name != "." {
        start = parentName(name)
    }
    zone, servers := r.closestKnownZone(start)
    if zone != "." {
        r.tracef(depth, "%s starting at cached zone %s", r.c(cDim, "↷"), r.c(cBold, zone))
    }
    minimise := r.Minimise && len(splitLabels(name)) <= maxMinimiseLabels
    probe := childToward(name, zone)

    for ; hops < r.MaxHops; hops++ {
        qname, qt := name, qtype
        if minimise && probe != name {
            qname, qt = probe, TypeNS
        }
        probing := qname != name
        msg, ns, rtt, err := r.queryAny(servers, qname, qt, depth)
        if err != nil {
            return nil, fmt.Errorf("querying %s servers for %s: %w", zone, qname, err)
        }

        if probing && (msg.Header.Rcode != RcodeNoError || unexpectedAnswer(msg, qname)) {
            r.tracef(depth, "%s %s gave %s to the minimised query, asking for the full name",
r.c(cYellow, "↺"), ns.Name, rcodeString(msg.Header.Rcode))
            minimise = false
            continue
        }

        if !probing {
            if msg.Header.Rcode == RcodeNXDomain {
                r.tracef(depth, "%s %s says %s (%s)", r.c(cRed, "✗"), ns.Name, r.c(cRed,
"NXDOMAIN"), rtt)
                return &Result{Rcode: RcodeNXDomain, Authority: r.clampTTLs(msg.Authority), Hops:
hops + 1}, nil
            }
            if msg.Header.Rcode != RcodeNoError {
                r.tracef(depth, "%s %s returned %s, trying next server", r.c(cYellow, "!"),
ns.Name, rcodeString(msg.Header.Rcode))
                servers = without(servers, ns)
                if len(servers) == 0 {
                    return &Result{Rcode: msg.Header.Rcode, Hops: hops + 1}, nil
                }
                continue
            }

            var direct, cnames, sigs []RR
            for _, rr := range msg.Answers {
                if !strings.EqualFold(rr.Name, name) {
                    continue
                }
                rr.Zone = zone
                switch {
                case rr.Type == qtype:
                    direct = append(direct, rr)
                case rr.Type == TypeCNAME:
                    cnames = append(cnames, rr)
                case rr.Type == TypeRRSIG:
                    sigs = append(sigs, rr)
                }
            }
            if len(direct) > 0 {
                r.tracef(depth, "%s %s answered %s (%s)", r.c(cGreen, "✓"), ns.Name,
```

```
                    r.c(cGreen, fmt.Sprintf("%d %s record(s)", len(direct), qtype)), rtt)
                direct, sigs = r.clampTTLs(direct), coveredSigs(sigs, qtype)
                r.cachePut(name, qtype, direct, sigs)
                return &Result{Rcode: RcodeNoError, Answers: direct, Sigs: sigs, Hops: hops + 1},
nil
            }
            if len(cnames) > 0 {
                target := fqdn(cnames[0].Data)
                r.tracef(depth, "%s %s is an alias → %s, restarting from the top", r.c(cCyan, "↪
CNAME"), name, r.c(cBold, target))
                sub, err := r.resolve(target, qtype, depth+1, hops+1)
                if err != nil {
                    return nil, err
                }
                sub.Answers = append(r.clampTTLs(cnames[:1]), sub.Answers...)
                sub.Sigs = append(coveredSigs(sigs, TypeCNAME), sub.Sigs...)
                return sub, nil
            }
            if msg.Header.AA {
                r.tracef(depth, "%s %s: name exists but has no %s records (NODATA)", r.c(cYellow,
"Ø"), ns.Name, qtype)
                return &Result{Rcode: RcodeNoError, Authority: r.clampTTLs(msg.Authority), Hops:
hops + 1}, nil
            }
        }

        nsRRs, child := referralNS(msg, qname, probing)
        if len(nsRRs) == 0 {
            if probing && msg.Header.AA {
                r.tracef(depth, "%s %s: no zone cut at %s (%s)", r.c(cDim, "·"), ns.Name, qname,
rtt)
                probe = childToward(name, probe)
                continue
            }
            return nil, fmt.Errorf("lame response from %s: no answer, no referral", ns.Name)
        }

        if !isSubdomain(name, child) || !isSubdomain(child, zone) || child == zone {
            return nil, fmt.Errorf("out-of-bailiwick referral from %s: %s is not under %s",
ns.Name, child, zone)
        }

        next := r.glueFor(nsRRs, msg.Additional, zone)
        names := make([]string, 0, len(nsRRs))
        for _, rr := range nsRRs {
            names = append(names, strings.TrimSuffix(rr.Data, "."))
        }
        r.tracef(depth, "%s %s → referral to %s %s (%s)", r.c(cCyan, "↓"), ns.Name, r.c(cBold,
child),
            r.c(cDim, fmt.Sprintf("[%s]", summarize(names, 3))), rtt)

        if len(next) == 0 {
            r.tracef(depth, "%s no glue for %s — resolving a nameserver's address first",
r.c(cYellow, "…"), child)
            for i, nsrr := range shuffled(nsRRs) {
                if i >= maxGluelessLookups {
                    r.tracef(depth, "%s giving up after %d nameserver lookups for %s",
r.c(cYellow, "!"), maxGluelessLookups, child)
                    break
                }
                sub, err := r.resolve(fqdn(nsrr.Data), TypeA, depth+1, 0)
                if err != nil || sub.Rcode != RcodeNoError {
                    continue
                }
                for _, a := range sub.Answers {
                    if a.Type == TypeA {
                        next = append(next, nameserver{fqdn(nsrr.Data), a.Data})
                    }
                }
                if len(next) > 0 {
                    break
                }
            }
            if len(next) == 0 {
                return nil, fmt.Errorf("could not find an address for any %s nameserver", child)
            }
        }
```

```
        r.putZone(child, next, minTTL(nsRRs))
        zone, servers = child, next
        probe = childToward(name, zone)
    }
    return nil, ErrLoop
}

// glueFor pairs NS names with A records from the Additional section.
// Glue is only trusted if it lies inside the zone of the server that sent it
// (askedZone) — a .com server may vouch for ns1.example.com, not for ns.evil.org.
func (r *Resolver) glueFor(nsRRs, additional []RR, askedZone string) []nameserver {
    want := map[string]bool{}
    for _, rr := range nsRRs {
        want[fqdn(rr.Data)] = true
    }
    var out []nameserver
    for _, rr := range additional {
        n := fqdn(rr.Name)
        if rr.Type == TypeA && want[n] && isSubdomain(n, askedZone) {
            out = append(out, nameserver{n, rr.Data})
        }
    }
    return out
}

// queryAny tries servers in random order until one gives a usable reply.
func (r *Resolver) queryAny(servers []nameserver, name string, qtype RRType, depth int)
(*Message, nameserver, time.Duration, error) {
    var lastErr error = ErrNoServer
    for i, ns := range shuffled(servers) {
        if i >= 4 {
            break
        }
        r.tracef(depth, "%s asking %s %s for %s %s", r.c(cDim, "→"), ns.Name, r.c(cDim,
"("+ns.IP+")"), name, qtype)
        t0 := time.Now()
        msg, err := r.exchange(ns.IP, name, qtype)
        rtt := time.Since(t0).Round(time.Millisecond)
        if err != nil {
            r.tracef(depth, "%s %s: %v", r.c(cYellow, "!"), ns.Name, err)
            lastErr = err
            continue
        }
        return msg, ns, rtt, nil
    }
    return nil, nameserver{}, 0, lastErr
}

// matchesQuery is the anti-spoofing check: a reply is only accepted if it
// carries our ID and repeats exactly the question we asked.
func matchesQuery(m *Message, id uint16, name string, qtype RRType) bool {
    return m.Header.ID == id && m.Header.QR && len(m.Questions) == 1 &&
        strings.EqualFold(m.Questions[0].Name, name) && m.Questions[0].Type == qtype
}

var errCaseMismatch = errors.New("reply does not repeat the 0x20 letter case of the query")

// randomID draws a query ID from the operating system's cryptographic random source.
func randomID() uint16 {
    var b [2]byte
    crand.Read(b[:])
    return binary.BigEndian.Uint16(b[:])
}

// randomCase flips each letter of name to upper or lower case at random
// (the 0x20 technique), adding one bit of entropy per letter to the query.
func randomCase(name string) string {
    bits := make([]byte, len(name))
    crand.Read(bits)
    b := []byte(name)
    for i, c := range b {
        if 'a' <= c|0x20 && c|0x20 <= 'z' {
            b[i] = c&^0x20 | (bits[i]&1)<<5
        }
```

```
    }
    return string(b)
}

// lowerNames normalises the names in an accepted reply, so that the random
// letter case of the query never reaches the caches or the output.
func lowerNames(m *Message) {
    for i := range m.Questions {
        m.Questions[i].Name = strings.ToLower(m.Questions[i].Name)
    }
    for _, sec := range [][]RR{m.Answers, m.Authority, m.Additional} {
        for i := range sec {
            sec[i].Name = strings.ToLower(sec[i].Name)
            switch sec[i].Type {
            case TypeNS, TypeCNAME, TypePTR, TypeMX, TypeSOA:
                sec[i].Data = strings.ToLower(sec[i].Data)
            }
        }
    }
}

func (r *Resolver) caseUnsafe(ip string) bool {
    r.mu.Lock()
    defer r.mu.Unlock()
    t, ok := r.no0x20[ip]
    return ok && time.Now().Before(t)
}

func (r *Resolver) markCaseUnsafe(ip string) {
    now := time.Now()
    r.mu.Lock()
    defer r.mu.Unlock()
    evict(r.no0x20, r.MaxCacheEntries, func(t time.Time) bool { return now.After(t) })
    r.no0x20[ip] = now.Add(time.Hour)
}

// exchange sends one query, with 0x20 case randomisation unless the server is
// known not to preserve case. A server that answers with different case is
// remembered for an hour and asked again without 0x20.
func (r *Resolver) exchange(ip, name string, qtype RRType) (*Message, error) {
    mixed := r.CaseRandomisation && !r.caseUnsafe(ip)
    m, err := r.exchangeOnce(ip, name, qtype, mixed)
    if errors.Is(err, errCaseMismatch) {
        r.tracef(0, "%s %s does not preserve letter case, retrying without 0x20", r.c(cYellow,
"!"), ip)
        r.markCaseUnsafe(ip)
        m, err = r.exchangeOnce(ip, name, qtype, false)
    }
    return m, err
}

// exchangeOnce sends one query over UDP, retrying over TCP if the answer is truncated.
func (r *Resolver) exchangeOnce(ip, name string, qtype RRType, mixed bool) (*Message, error) {
    sent := name
    if mixed {
        sent = randomCase(name)
    }
    id := randomID()
    q, err := BuildQuery(id, sent, qtype, r.DNSSEC)
    if err != nil {
        return nil, err
    }
    addr := net.JoinHostPort(ip, r.Port)

    conn, err := net.DialTimeout("udp", addr, r.Timeout)
    if err != nil {
        return nil, err
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(r.Timeout))
    if _, err := conn.Write(q); err != nil {
        return nil, err
    }
    buf := make([]byte, 4096)
    for {
        n, err := conn.Read(buf)
```

```
        if err != nil {
            return nil, err
        }
        m, err := ParseMessage(buf[:n])
        if err != nil || !matchesQuery(m, id, sent, qtype) {
            continue
        }
        if mixed && m.Questions[0].Name != sent {
            return nil, errCaseMismatch
        }
        if m.Header.TC {
            r.tracef(0, "%s response truncated, retrying over TCP", r.c(cYellow, "⇄"))
            return exchangeTCP(addr, q, id, sent, qtype, r.Timeout, mixed)
        }
        lowerNames(m)
        return m, nil
    }
}

// DNS over TCP: same message, prefixed with a 2-byte length (RFC 1035 §4.2.2).
func exchangeTCP(addr string, q []byte, id uint16, name string, qtype RRType, timeout
time.Duration, mixed bool) (*Message, error) {
    conn, err := net.DialTimeout("tcp", addr, timeout)
    if err != nil {
        return nil, err
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(timeout))
    frame := binary.BigEndian.AppendUint16(nil, uint16(len(q)))
    if _, err := conn.Write(append(frame, q...)); err != nil {
        return nil, err
    }
    var lb [2]byte
    if _, err := io.ReadFull(conn, lb[:]); err != nil {
        return nil, err
    }
    buf := make([]byte, binary.BigEndian.Uint16(lb[:]))
    if _, err := io.ReadFull(conn, buf); err != nil {
        return nil, err
    }
    m, err := ParseMessage(buf)
    if err != nil {
        return nil, err
    }
    if !matchesQuery(m, id, name, qtype) {
        return nil, errors.New("TCP reply does not match the query (ID or question)")
    }
    if mixed && m.Questions[0].Name != name {
        return nil, errCaseMismatch
    }
    lowerNames(m)
    return m, nil
}

func shuffled[T any](in []T) []T {
    out := append([]T(nil), in...)
    rand.Shuffle(len(out), func(i, j int) { out[i], out[j] = out[j], out[i] })
    return out
}

func without(in []nameserver, x nameserver) []nameserver {
    var out []nameserver
    for _, s := range in {
        if s != x {
            out = append(out, s)
        }
    }
    return out
}

func summarize(s []string, n int) string {
    if len(s) <= n {
        return strings.Join(s, ", ")
    }
    return fmt.Sprintf("%s, +%d more", strings.Join(s[:n], ", "), len(s)-n)
}
```

A.3 dnssec.go

```
package main

// dnssec.go — DNSSEC validation of positive answers (RFC 4033–4035):
//
//   root anchor —▶ . DNSKEY —▶ com. DS —▶ com. DNSKEY —▶ example.com. DS —▶ example.com.
DNSKEY —▶ RRSIG on the answer
//
// Every answer RRset must carry an RRSIG made by a key of the zone that signed
// it. Those keys are trusted only if one of them matches the DS record that the
// parent zone publishes (and signs), and so on up to the root, whose key must
// match a built-in trust anchor. Proofs of non-existence (NSEC/NSEC3) are not
// validated, so a missing DS is reported as insecure without a proof.

import (
    "bytes"
    "crypto"
    "crypto/ecdsa"
    "crypto/ed25519"
    "crypto/elliptic"
    "crypto/rsa"
    "crypto/sha1"
    "crypto/sha256"
    "crypto/sha512"
    "encoding/binary"
    "encoding/hex"
    "errors"
    "fmt"
    "hash"
    "math/big"
    "sort"
    "strings"
    "time"
)

type Security int

const (
    SecUnchecked Security = iota
    SecInsecure
    SecSecure
    SecBogus
)

func (s Security) String() string {
    return [...]string{"unchecked", "insecure", "secure", "bogus"}[s]
}

type DSRecord struct {
    Owner      string
    KeyTag     uint16
    Algorithm  uint8
    DigestType uint8
    Digest     []byte
}

func mustHex(s string) []byte {
    b, err := hex.DecodeString(s)
    if err != nil {
        panic(err)
    }
    return b
}

// IANA root trust anchors (root-anchors.xml): KSK-2017 and KSK-2024.
var rootAnchors = []DSRecord{
    {".", 20326, 8, 2,
mustHex("E06D44B80B8F1D39A95C0B0D7C65D08458E880409BBC683457104237C7F8EC8D")},
```

```
    {".", 38696, 8, 2,
mustHex("683D2D0ACB8C9B712A1948B27F741219298D0A450D612C483AF444A4C0FB2B16")},
}

type rrsig struct {
    TypeCovered RRType
    Algorithm   uint8
    Labels      uint8
    OrigTTL     uint32
    Expiration  uint32
    Inception   uint32
    KeyTag      uint16
    Signer      string
    Signature   []byte
}

// parseRRSIG decodes RRSIG RDATA; the signer name is never compressed (RFC 4034 §3.1.7).
func parseRRSIG(rd []byte) (*rrsig, error) {
    if len(rd) < 19 {
        return nil, errShort
    }
    be16, be32 := binary.BigEndian.Uint16, binary.BigEndian.Uint32
    s := &rrsig{TypeCovered: RRType(be16(rd)), Algorithm: rd[2], Labels: rd[3], OrigTTL:
be32(rd[4:]),
        Expiration: be32(rd[8:]), Inception: be32(rd[12:]), KeyTag: be16(rd[16:])}
    name, end, err := readName(rd, 18)
    if err != nil {
        return nil, err
    }
    s.Signer, s.Signature = strings.ToLower(name), rd[end:]
    return s, nil
}

// prefix is the RRSIG RDATA without the signature, the first part of the signed data.
func (s *rrsig) prefix() []byte {
    b := binary.BigEndian.AppendUint16(nil, uint16(s.TypeCovered))
    b = append(b, s.Algorithm, s.Labels)
    b = binary.BigEndian.AppendUint32(b, s.OrigTTL)
    b = binary.BigEndian.AppendUint32(b, s.Expiration)
    b = binary.BigEndian.AppendUint32(b, s.Inception)
    b = binary.BigEndian.AppendUint16(b, s.KeyTag)
    b, _ = encodeName(b, strings.ToLower(s.Signer))
    return b
}

type dnskey struct {
    Owner     string
    Flags     uint16
    Algorithm uint8
    Key       []byte
    Raw       []byte
    Tag       uint16
}

func parseDNSKEY(rr RR) (*dnskey, error) {
    if len(rr.Raw) < 5 || rr.Raw[2] != 3 {
        return nil, errors.New("malformed DNSKEY")
    }
    return &dnskey{Owner: strings.ToLower(rr.Name), Flags: binary.BigEndian.Uint16(rr.Raw),
Algorithm: rr.Raw[3],
        Key: rr.Raw[4:], Raw: rr.Raw, Tag: keyTag(rr.Raw)}, nil
}

func parseDS(rr RR) (DSRecord, error) {
    if len(rr.Raw) < 5 {
        return DSRecord{}, errors.New("malformed DS")
    }
    return DSRecord{strings.ToLower(rr.Name), binary.BigEndian.Uint16(rr.Raw), rr.Raw[2],
rr.Raw[3], rr.Raw[4:]}, nil
}

// keyTag is the 16-bit checksum of DNSKEY RDATA (RFC 4034 Appendix B).
func keyTag(rd []byte) uint16 {
    var ac uint32
    for i, b := range rd {
```

```
        if i&1 == 0 {
            ac += uint32(b) << 8
        } else {
            ac += uint32(b)
        }
    }
    ac += ac >> 16 & 0xFFFF
    return uint16(ac)
}

func supportedAlgorithm(a uint8) bool { return a == 8 || a == 10 || a == 13 || a == 14 || a == 15
}

func digester(t uint8) hash.Hash {
    switch t {
    case 1:
        return sha1.New()
    case 2:
        return sha256.New()
    case 4:
        return sha512.New384()
    }
    return nil
}

// dsMatches checks that a DS record is the digest of this DNSKEY (RFC 4034 §5.1.4).
func dsMatches(ds DSRecord, k *dnskey) bool {
    if ds.KeyTag != k.Tag || ds.Algorithm != k.Algorithm || !strings.EqualFold(ds.Owner, k.Owner)
{
        return false
    }
    h := digester(ds.DigestType)
    if h == nil {
        return false
    }
    owner, _ := encodeName(nil, k.Owner)
    h.Write(owner)
    h.Write(k.Raw)
    return bytes.Equal(h.Sum(nil), ds.Digest)
}

// canonicalRData is the RDATA in canonical form: names uncompressed and lower-case (RFC 4034
§6.2).
func canonicalRData(rr RR) ([]byte, error) {
    switch rr.Type {
    case TypeNS, TypeCNAME, TypePTR, TypeMX, TypeSOA:
        rr.Data = strings.ToLower(rr.Data)
        return encodeRData(rr)
    }
    return rr.Raw, nil
}

// labelCount counts labels for the RRSIG Labels field, ignoring a leading "*".
func labelCount(name string) int {
    ls := splitLabels(name)
    if len(ls) > 0 && ls[0] == "*" {
        return len(ls) - 1
    }
    return len(ls)
}

// signedData builds the bytes an RRSIG signs: its own fields followed by the
// RRset in canonical form and order (RFC 4034 §3.1.8.1, §6.3).
func signedData(s *rrsig, set []RR) ([]byte, error) {
    owner := strings.ToLower(set[0].Name)
    if n := labelCount(owner); int(s.Labels) < n {
        ls := splitLabels(owner)
        owner = "*." + strings.Join(ls[len(ls)-int(s.Labels):], ".") + "."
        if s.Labels == 0 {
            owner = "*."
        }
    }
    ow, err := encodeName(nil, owner)
    if err != nil {
        return nil, err
    }
```

```
    var rds [][]byte
    for _, rr := range set {
        rd, err := canonicalRData(rr)
        if err != nil {
            return nil, err
        }
        rds = append(rds, rd)
    }
    sort.Slice(rds, func(i, j int) bool { return bytes.Compare(rds[i], rds[j]) < 0 })
    out := s.prefix()
    for i, rd := range rds {
        if i > 0 && bytes.Equal(rd, rds[i-1]) {
            continue
        }
        out = append(out, ow...)
        out = binary.BigEndian.AppendUint16(out, uint16(set[0].Type))
        out = binary.BigEndian.AppendUint16(out, set[0].Class)
        out = binary.BigEndian.AppendUint32(out, s.OrigTTL)
        out = binary.BigEndian.AppendUint16(out, uint16(len(rd)))
        out = append(out, rd...)
    }
    return out, nil
}

// rsaPublicKey decodes the RFC 3110 key format: exponent length, exponent, modulus.
func rsaPublicKey(k []byte) (*rsa.PublicKey, error) {
    if len(k) < 3 {
        return nil, errors.New("short RSA key")
    }
    elen, off := int(k[0]), 1
    if elen == 0 {
        elen, off = int(binary.BigEndian.Uint16(k[1:])), 3
    }
    if off+elen >= len(k) || elen > 4 {
        return nil, errors.New("bad RSA key")
    }
    e := new(big.Int).SetBytes(k[off : off+elen])
    return &rsa.PublicKey{N: new(big.Int).SetBytes(k[off+elen:]), E: int(e.Int64())}, nil
}

var errBadSignature = errors.New("signature does not verify")

// verifySignature checks one RRSIG against one DNSKEY for the supported algorithms.
func verifySignature(s *rrsig, k *dnskey, data []byte) error {
    switch k.Algorithm {
    case 8, 10:
        pub, err := rsaPublicKey(k.Key)
        if err != nil {
            return err
        }
        h, sum := crypto.SHA256, sha256.Sum256(data)
        digest := sum[:]
        if k.Algorithm == 10 {
            s512 := sha512.Sum512(data)
            h, digest = crypto.SHA512, s512[:]
        }
        if rsa.VerifyPKCS1v15(pub, h, digest, s.Signature) != nil {
            return errBadSignature
        }
        return nil
    case 13, 14:
        curve, size := elliptic.P256(), 32
        var digest []byte
        if k.Algorithm == 13 {
            d := sha256.Sum256(data)
            digest = d[:]
        } else {
            d := sha512.Sum384(data)
            curve, size, digest = elliptic.P384(), 48, d[:]
        }
        if len(k.Key) != 2*size || len(s.Signature) != 2*size {
            return errors.New("bad ECDSA key or signature length")
        }
        pub, err := ecdsa.ParseUncompressedPublicKey(curve, append([]byte{4}, k.Key...))
        if err != nil {
            return err
        }
```

```
        r, ss := new(big.Int).SetBytes(s.Signature[:size]),
new(big.Int).SetBytes(s.Signature[size:])
        if !ecdsa.Verify(pub, digest, r, ss) {
            return errBadSignature
        }
        return nil
    case 15:
        if len(k.Key) != ed25519.PublicKeySize || !ed25519.Verify(ed25519.PublicKey(k.Key), data,
s.Signature) {
            return errBadSignature
        }
        return nil
    }
    return fmt.Errorf("unsupported algorithm %d", k.Algorithm)
}

// checkValidity applies the inception/expiration window using serial-number arithmetic (RFC
1982).
func checkValidity(s *rrsig, now time.Time) error {
    t := uint32(now.Unix())
    if int32(t-s.Inception) < 0 {
        return errors.New("signature not yet valid")
    }
    if int32(s.Expiration-t) < 0 {
        return errors.New("signature expired")
    }
    return nil
}

// verifyRRset succeeds if any RRSIG made by zone with one of keys verifies the RRset.
func verifyRRset(set, sigs []RR, keys []*dnskey, zone string) error {
    t := set[0].Type
    lastErr := fmt.Errorf("no RRSIG from %s for %s %s", zone, set[0].Name, t)
    for _, sr := range sigs {
        s, err := parseRRSIG(sr.Raw)
        if err != nil {
            lastErr = err
            continue
        }
        if s.TypeCovered != t || !strings.EqualFold(sr.Name, set[0].Name) || s.Signer != zone ||
            !isSubdomain(strings.ToLower(set[0].Name), s.Signer) {
            continue
        }
        if int(s.Labels) > labelCount(set[0].Name) {
            lastErr = errors.New("RRSIG labels field larger than the owner name")
            continue
        }
        if err := checkValidity(s, time.Now()); err != nil {
            lastErr = err
            continue
        }
        data, err := signedData(s, set)
        if err != nil {
            return err
        }
        for _, k := range keys {
            if k.Tag != s.KeyTag || k.Algorithm != s.Algorithm {
                continue
            }
            if err := verifySignature(s, k, data); err != nil {
                lastErr = fmt.Errorf("%s %s: %w", set[0].Name, t, err)
                continue
            }
            return nil
        }
    }
    return lastErr
}

type keyEntry struct {
    keys    []*dnskey
    sec     Security
    why     string
    expires time.Time
}

// zoneKeys returns the validated zone-signing keys of zone, walking the chain
```

```
// of DS and DNSKEY records up to the root trust anchor.
func (r *Resolver) zoneKeys(zone string, depth int) ([]*dnskey, Security, string) {
    if depth > 16 {
        return nil, SecBogus, "chain of trust too deep"
    }
    now := time.Now()
    r.mu.Lock()
    e, ok := r.keys[zone]
    r.mu.Unlock()
    if ok && now.Before(e.expires) {
        return e.keys, e.sec, e.why
    }
    keys, sec, why, ttl := r.fetchZoneKeys(zone, depth)
    if sec == SecBogus {
        ttl = 60
    }
    r.mu.Lock()
    evict(r.keys, r.MaxCacheEntries, func(e keyEntry) bool { return now.After(e.expires) })
    r.keys[zone] = keyEntry{keys, sec, why, now.Add(time.Duration(min(ttl, r.MaxTTL)) *
time.Second)}
    r.mu.Unlock()
    return keys, sec, why
}

func (r *Resolver) fetchZoneKeys(zone string, depth int) ([]*dnskey, Security, string, uint32) {
    var dss []DSRecord
    if zone == "." {
        dss = r.Anchors
    } else {
        res, err := r.resolve(zone, TypeDS, 1, 0)
        if err != nil {
            return nil, SecBogus, fmt.Sprintf("cannot fetch DS for %s: %v", zone, err), 60
        }
        dsSet := filterType(res.Answers, TypeDS)
        if res.Rcode != RcodeNoError || len(dsSet) == 0 {
            r.tracef(1, "%s no DS for %s at its parent — unsigned delegation", r.c(cYellow, "»"),
zone)
            return nil, SecInsecure, fmt.Sprintf("no DS record for %s (unsigned delegation)",
zone), 300
        }
        parent := dsSet[0].Zone
        for _, sr := range coveredSigs(res.Sigs, TypeDS) {
            if s, err := parseRRSIG(sr.Raw); err == nil && s.Signer != zone && isSubdomain(zone,
s.Signer) {
                parent = s.Signer
                break
            }
        }
        pkeys, psec, pwhy := r.zoneKeys(parent, depth+1)
        if psec != SecSecure {
            return nil, psec, pwhy, 300
        }
        if err := verifyRRset(dsSet, res.Sigs, pkeys, parent); err != nil {
            return nil, SecBogus, fmt.Sprintf("DS for %s: %v", zone, err), 60
        }
        for _, rr := range dsSet {
            if ds, err := parseDS(rr); err == nil && supportedAlgorithm(ds.Algorithm) &&
digester(ds.DigestType) != nil {
                dss = append(dss, ds)
            }
        }
        if len(dss) == 0 {
            return nil, SecInsecure, fmt.Sprintf("%s uses only unsupported DNSSEC algorithms",
zone), 300
        }
        r.tracef(1, "%s DS for %s verified with the keys of %s", r.c(cGreen, "»"), zone, parent)
    }

    res, err := r.resolve(zone, TypeDNSKEY, 1, 0)
    if err != nil {
        return nil, SecBogus, fmt.Sprintf("cannot fetch DNSKEY for %s: %v", zone, err), 60
    }
    keyRRs := filterType(res.Answers, TypeDNSKEY)
    if len(keyRRs) == 0 {
        return nil, SecBogus, fmt.Sprintf("%s has a DS record but no DNSKEY", zone), 60
    }
    var all, trusted, zsks []*dnskey
    for _, rr := range keyRRs {
```

```
        k, err := parseDNSKEY(rr)
        if err != nil || k.Flags&0x0100 == 0 {
            continue
        }
        all = append(all, k)
        for _, ds := range dss {
            if dsMatches(ds, k) {
                trusted = append(trusted, k)
                break
            }
        }
    }
    if len(trusted) == 0 {
        return nil, SecBogus, fmt.Sprintf("no DNSKEY of %s matches its DS / trust anchor", zone),
60
    }
    if err := verifyRRset(keyRRs, res.Sigs, trusted, zone); err != nil {
        return nil, SecBogus, fmt.Sprintf("DNSKEY set of %s: %v", zone, err), 60
    }
    zsks = all
    r.tracef(1, "%s DNSKEY set of %s verified (%d keys, signed by key %d)", r.c(cGreen, "»"),
zone, len(zsks), trusted[0].Tag)
    return zsks, SecSecure, "", minTTL(keyRRs)
}

func filterType(rrs []RR, t RRType) []RR {
    var out []RR
    for _, rr := range rrs {
        if rr.Type == t {
            out = append(out, rr)
        }
    }
    return out
}

// validateResult checks every RRset of a positive answer; the weakest result wins.
func (r *Resolver) validateResult(res *Result) (Security, string) {
    if res.Rcode != RcodeNoError || len(res.Answers) == 0 {
        return SecUnchecked, "negative answers are not validated (NSEC/NSEC3 not implemented)"
    }
    type key struct {
        owner string
        t     RRType
    }
    var order []key
    sets := map[key][]RR{}
    for _, rr := range res.Answers {
        k := key{strings.ToLower(rr.Name), rr.Type}
        if _, ok := sets[k]; !ok {
            order = append(order, k)
        }
        sets[k] = append(sets[k], rr)
    }
    overall, note := SecSecure, "chain of trust verified from the root key"
    for _, k := range order {
        set := sets[k]
        var sigs []RR
        for _, s := range coveredSigs(res.Sigs, k.t) {
            if strings.EqualFold(s.Name, k.owner) {
                sigs = append(sigs, s)
            }
        }
        zone := set[0].Zone
        for _, sr := range sigs {
            if s, err := parseRRSIG(sr.Raw); err == nil && isSubdomain(k.owner, s.Signer) {
                zone = s.Signer
                break
            }
        }
        if zone == "" {
            zone = "."
        }
        keys, sec, why := r.zoneKeys(zone, 1)
        switch sec {
        case SecBogus:
            return SecBogus, why
        case SecInsecure:
            overall, note = SecInsecure, why
```

```
        case SecSecure:
            if err := verifyRRset(set, sigs, keys, zone); err != nil {
                return SecBogus, err.Error()
            }
        }
    }
    return overall, note
}

func (r *Resolver) traceSecurity(res *Result) {
    switch res.Security {
    case SecSecure:
        r.tracef(0, "%s %s", r.c(cGreen, "» DNSSEC: secure"), res.SecurityNote)
    case SecBogus:
        r.tracef(0, "%s %s", r.c(cRed, "» DNSSEC: BOGUS"), res.SecurityNote)
    default:
        r.tracef(0, "%s %s", r.c(cYellow, "» DNSSEC: "+res.Security.String()), res.SecurityNote)
    }
}
```

A.4 server.go

```
package main

import (
    "crypto/ecdsa"
    "crypto/elliptic"
    "crypto/rand"
    "crypto/sha256"
    "crypto/tls"
    "crypto/x509"
    "crypto/x509/pkix"
    "encoding/binary"
    "fmt"
    "io"
    "log"
    "math/big"
    "net"
    "net/netip"
    "strings"
    "sync"
    "time"
)

func encodeRData(rr RR) ([]byte, error) {
    switch rr.Type {
    case TypeNS, TypeCNAME, TypePTR:
        return encodeName(nil, rr.Data)
    case TypeMX:
        var pref uint16
        var host string
        if _, err := fmt.Sscanf(rr.Data, "%d %s", &pref, &host); err != nil {
            return nil, err
        }
        return encodeName(binary.BigEndian.AppendUint16(nil, pref), host)
    case TypeSOA:
        f := strings.Fields(rr.Data)
        if len(f) != 7 {
            return nil, fmt.Errorf("bad SOA %q", rr.Data)
        }
        b, err := encodeName(nil, f[0])
        if err != nil {
            return nil, err
        }
        if b, err = encodeName(b, f[1]); err != nil {
            return nil, err
        }
        for _, s := range f[2:] {
            var v uint32
            fmt.Sscan(s, &v)
            b = binary.BigEndian.AppendUint32(b, v)
        }
        return b, nil
```

```
    default:
        return rr.Raw, nil
    }
}

func appendRR(b []byte, rr RR) ([]byte, error) {
    b, err := encodeName(b, rr.Name)
    if err != nil {
        return nil, err
    }
    rd, err := encodeRData(rr)
    if err != nil {
        return nil, err
    }
    b = binary.BigEndian.AppendUint16(b, uint16(rr.Type))
    b = binary.BigEndian.AppendUint16(b, 1)
    b = binary.BigEndian.AppendUint32(b, rr.TTL)
    b = binary.BigEndian.AppendUint16(b, uint16(len(rd)))
    return append(b, rd...), nil
}

func buildResponse(id uint16, q Question, rd bool, rcode int, ans, auth, add []RR) []byte {
    flags := uint16(0x8000 | 0x0080)
    if rd {
        flags |= 0x0100
    }
    flags |= uint16(rcode & 0xF)

    b := make([]byte, 12)
    binary.BigEndian.PutUint16(b[0:], id)
    binary.BigEndian.PutUint16(b[2:], flags)
    binary.BigEndian.PutUint16(b[4:], 1)
    b, _ = encodeName(b, q.Name)
    b = binary.BigEndian.AppendUint16(b, uint16(q.Type))
    b = binary.BigEndian.AppendUint16(b, q.Class)

    var na, nn uint16
    for _, rr := range ans {
        if nb, err := appendRR(b, rr); err == nil {
            b, na = nb, na+1
        }
    }
    for _, rr := range auth {
        if nb, err := appendRR(b, rr); err == nil {
            b, nn = nb, nn+1
        }
    }
    var nr uint16
    for _, rr := range add {
        if nb, err := appendRR(b, rr); err == nil {
            b, nr = nb, nr+1
        }
    }
    binary.BigEndian.PutUint16(b[6:], na)
    binary.BigEndian.PutUint16(b[8:], nn)
    binary.BigEndian.PutUint16(b[10:], nr)
    return b
}

// Server answers DNS queries from clients using a shared Resolver. It limits
// the number of queries in flight and the query rate of each client address,
// and only serves clients whose address is in Allow (loopback when empty).
type Server struct {
    R           *Resolver
    MaxInflight int
    Rate        float64
    Burst       float64
    Allow       []netip.Prefix
    sem         chan struct{}
    mu          sync.Mutex
    buckets     map[string]*bucket
    refused     map[string]time.Time
}

var loopbackOnly = []netip.Prefix{netip.MustParsePrefix("127.0.0.0/8"),
netip.MustParsePrefix("::1/128")}
```

```
// ParseAllow reads a comma-separated list of CIDR ranges or single addresses.
func ParseAllow(list string) ([]netip.Prefix, error) {
    var out []netip.Prefix
    for _, item := range strings.Split(list, ",") {
        item = strings.TrimSpace(item)
        if item == "" {
            continue
        }
        if strings.Contains(item, "/") {
            p, err := netip.ParsePrefix(item)
            if err != nil {
                return nil, err
            }
            out = append(out, p.Masked())
            continue
        }
        a, err := netip.ParseAddr(item)
        if err != nil {
            return nil, err
        }
        out = append(out, netip.PrefixFrom(a.Unmap(), a.Unmap().BitLen()))
    }
    return out, nil
}

// permitted reports whether a client address falls inside the access list.
func (s *Server) permitted(ip string) bool {
    a, err := netip.ParseAddr(ip)
    if err != nil {
        return false
    }
    a = a.Unmap().WithZone("")
    allow := s.Allow
    if len(allow) == 0 {
        allow = loopbackOnly
    }
    for _, p := range allow {
        if p.Contains(a) {
            return true
        }
    }
    return false
}

// noteRefused logs a refused client at most once per minute.
func (s *Server) noteRefused(ip string) {
    now := time.Now()
    s.mu.Lock()
    defer s.mu.Unlock()
    if now.Sub(s.refused[ip]) < time.Minute {
        return
    }
    if len(s.refused) > 10000 {
        clear(s.refused)
    }
    s.refused[ip] = now
    log.Printf("access list: refusing queries from %s", ip)
}

type bucket struct {
    tokens  float64
    last    time.Time
    logged  time.Time
    dropped int
}

func NewServer(r *Resolver) *Server {
    return &Server{R: r, MaxInflight: 256, Rate: 20, Burst: 40}
}

func (s *Server) init() {
    if s.sem == nil {
        s.sem = make(chan struct{}, s.MaxInflight)
        s.buckets = map[string]*bucket{}
        s.refused = map[string]time.Time{}
```

```
    }
}

// allow is a per-client token bucket: Rate queries per second, bursts up to Burst.
func (s *Server) allow(ip string) bool {
    now := time.Now()
    s.mu.Lock()
    defer s.mu.Unlock()
    if len(s.buckets) > 10000 {
        for k, b := range s.buckets {
            if now.Sub(b.last) > time.Minute {
                delete(s.buckets, k)
            }
        }
    }
    b, ok := s.buckets[ip]
    if !ok {
        b = &bucket{tokens: s.Burst, last: now}
        s.buckets[ip] = b
    }
    b.tokens = min(s.Burst, b.tokens+now.Sub(b.last).Seconds()*s.Rate)
    b.last = now
    if b.tokens < 1 {
        if b.dropped++; now.Sub(b.logged) > time.Minute {
            log.Printf("rate limit: dropping queries from %s (%d dropped)", ip, b.dropped)
            b.logged = now
        }
        return false
    }
    b.tokens--
    return true
}

// handle turns one query into one response; nil means drop the packet.
// For UDP, a response larger than the client's buffer is truncated (TC=1).
func (s *Server) handle(pkt []byte, client string, udp bool) []byte {
    m, err := ParseMessage(pkt)
    if err != nil || len(m.Questions) != 1 || m.Header.QR {
        return nil
    }
    q := m.Questions[0]
    if !s.permitted(client) {
        s.noteRefused(client)
        return buildResponse(m.Header.ID, q, m.Header.RD, RcodeRefused, nil, nil, nil)
    }
    if !s.allow(client) {
        return nil
    }
    res, err := s.R.Resolve(q.Name, q.Type)
    var out []byte
    switch {
    case err != nil:
        log.Printf("%-30s %-5s SERVFAIL (%v)", q.Name, q.Type, err)
        out = buildResponse(m.Header.ID, q, m.Header.RD, RcodeServFail, nil, nil, nil)
    case res.Security == SecBogus:
        log.Printf("%-30s %-5s SERVFAIL (DNSSEC bogus: %s)", q.Name, q.Type, res.SecurityNote)
        out = buildResponse(m.Header.ID, q, m.Header.RD, RcodeServFail, nil, nil, nil)
    default:
        log.Printf("%-30s %-5s %-8s %d answer(s), %d hops, %s%s", q.Name, q.Type,
rcodeString(res.Rcode),
            len(res.Answers), res.Hops, res.Elapsed.Round(1e6), securityTag(res))
        ans, auth := res.Answers, res.Authority
        if m.EDNS != nil && m.EDNS.DO {
            ans = append(append([]RR(nil), ans...), res.Sigs...)
        } else {
            auth = withoutDNSSEC(auth)
        }
        out = buildResponse(m.Header.ID, q, m.Header.RD, res.Rcode, ans, auth, nil)
        if res.Security == SecSecure {
            out[3] |= 0x20
        }
    }
    if limit := udpLimit(m); udp && len(out) > limit {
        out = buildResponse(m.Header.ID, q, m.Header.RD, int(out[3]&0xF), nil, nil, nil)
        out[2] |= 0x02
    }
    return out
}
```

```
// withoutDNSSEC drops RRSIG, NSEC and NSEC3 records, which only clients that set DO asked for.
func withoutDNSSEC(rrs []RR) []RR {
    var out []RR
    for _, rr := range rrs {
        if rr.Type != TypeRRSIG && rr.Type != TypeNSEC && rr.Type != TypeNSEC3 {
            out = append(out, rr)
        }
    }
    return out
}

func securityTag(res *Result) string {
    if res.Security == SecUnchecked {
        return ""
    }
    return ", dnssec " + res.Security.String()
}

// udpLimit is the largest UDP response the client accepts: 512 bytes, or its EDNS size up to
1232.
func udpLimit(m *Message) int {
    if m.EDNS == nil || m.EDNS.UDPSize < 512 {
        return 512
    }
    return min(int(m.EDNS.UDPSize), 1232)
}

func hostOf(addr net.Addr) string {
    h, _, err := net.SplitHostPort(addr.String())
    if err != nil {
        return addr.String()
    }
    return h
}

// ServeUDP answers queries on a UDP socket; when MaxInflight queries are
// already being resolved, new packets are dropped instead of queued.
func (s *Server) ServeUDP(pc net.PacketConn) error {
    s.init()
    buf := make([]byte, 1500)
    for {
        n, client, err := pc.ReadFrom(buf)
        if err != nil {
            return err
        }
        pkt := append([]byte(nil), buf[:n]...)
        select {
        case s.sem <- struct{}{}:
        default:
            continue
        }
        go func() {
            defer func() { <-s.sem }()
            if out := s.handle(pkt, hostOf(client), true); out != nil {
                pc.WriteTo(out, client)
            }
        }()
    }
}

// ServeStream answers length-prefixed queries on TCP or TLS connections (RFC 7766, RFC 7858).
func (s *Server) ServeStream(ln net.Listener) error {
    s.init()
    for {
        c, err := ln.Accept()
        if err != nil {
            return err
        }
        select {
        case s.sem <- struct{}{}:
        default:
            c.Close()
            continue
        }
        go func() {
```

```
            defer func() { <-s.sem }()
            defer c.Close()
            client := hostOf(c.RemoteAddr())
            for {
                c.SetDeadline(time.Now().Add(10 * time.Second))
                var lb [2]byte
                if _, err := io.ReadFull(c, lb[:]); err != nil {
                    return
                }
                pkt := make([]byte, binary.BigEndian.Uint16(lb[:]))
                if _, err := io.ReadFull(c, pkt); err != nil {
                    return
                }
                out := s.handle(pkt, client, false)
                if out == nil {
                    return
                }
                if _, err := c.Write(append(binary.BigEndian.AppendUint16(nil, uint16(len(out))),
out...)); err != nil {
                    return
                }
            }
        }()
    }
}

// dotConfig is the TLS configuration for DNS over TLS: TLS 1.3 only.
func dotConfig(cert tls.Certificate) *tls.Config {
    return &tls.Config{Certificates: []tls.Certificate{cert}, MinVersion: tls.VersionTLS13}
}

// selfSignedCert creates an in-memory ECDSA certificate for localhost, for testing DoT.
func selfSignedCert() (tls.Certificate, string, error) {
    key, err := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
    if err != nil {
        return tls.Certificate{}, "", err
    }
    serial, _ := rand.Int(rand.Reader, big.NewInt(1<<62))
    tmpl := &x509.Certificate{
        SerialNumber: serial, Subject: pkix.Name{CommonName: "dnsr"},
        NotBefore: time.Now().Add(-time.Hour), NotAfter: time.Now().Add(365 * 24 * time.Hour),
        DNSNames: []string{"localhost"}, IPAddresses: []net.IP{net.IPv4(127, 0, 0, 1),
net.IPv6loopback},
        KeyUsage: x509.KeyUsageDigitalSignature, ExtKeyUsage:
[]x509.ExtKeyUsage{x509.ExtKeyUsageServerAuth},
    }
    der, err := x509.CreateCertificate(rand.Reader, tmpl, tmpl, &key.PublicKey, key)
    if err != nil {
        return tls.Certificate{}, "", err
    }
    sum := sha256.Sum256(der)
    return tls.Certificate{Certificate: [][]byte{der}, PrivateKey: key}, fmt.Sprintf("%X", sum),
nil
}

type ServeConfig struct {
    Addr       string
    DoTAddr    string
    CertFile   string
    KeyFile    string
    SelfSigned bool
}

// serve listens on UDP and TCP at cfg.Addr and, if cfg.DoTAddr is set, on DNS over TLS.
func serve(cfg ServeConfig, s *Server) error {
    pc, err := net.ListenPacket("udp", cfg.Addr)
    if err != nil {
        return err
    }
    ln, err := net.Listen("tcp", cfg.Addr)
    if err != nil {
        return err
    }
    host, port, _ := net.SplitHostPort(pc.LocalAddr().String())
    log.Printf("dnsr listening on udp+tcp://%s — try: dig @%s -p %s example.com", pc.LocalAddr(),
host, port)
    allowed := s.Allow
```

```
    if len(allowed) == 0 {
        allowed = loopbackOnly
    }
    log.Printf("accepting clients from %v (change with -allow)", allowed)
    errs := make(chan error, 3)
    go func() { errs <- s.ServeUDP(pc) }()
    go func() { errs <- s.ServeStream(ln) }()

    if cfg.DoTAddr != "" {
        var cert tls.Certificate
        switch {
        case cfg.CertFile != "":
            if cert, err = tls.LoadX509KeyPair(cfg.CertFile, cfg.KeyFile); err != nil {
                return err
            }
        case cfg.SelfSigned:
            var fp string
            if cert, fp, err = selfSignedCert(); err != nil {
                return err
            }
            log.Printf("self-signed certificate, SHA-256 fingerprint %s", fp)
        default:
            return fmt.Errorf("DoT needs -cert and -key, or -selfsigned")
        }
        tl, err := tls.Listen("tcp", cfg.DoTAddr, dotConfig(cert))
        if err != nil {
            return err
        }
        log.Printf("dnsr listening on DNS over TLS %s", tl.Addr())
        go func() { errs <- s.ServeStream(tl) }()
    }
    return <-errs
}
```

A.5 main.go

```
package main

import (
    "flag"
    "fmt"
    "os"
)

const usage = `dnsr — an iterative DNS resolver built from scratch

usage:
  dnsr [-t TYPE] [-q] [-dnssec] <name> [name...]   resolve from the root servers down
  dnsr serve [-addr 127.0.0.1:8053] [flags]        run as a recursive DNS server

flags:
  -t TYPE   record type: A, AAAA, NS, MX, TXT, CNAME, SOA, PTR, DS, DNSKEY, RRSIG (default A)
  -q        quiet: print only the final answer, no trace
  -dnssec   validate answers with DNSSEC from the root trust anchor
  -nomin    send the full name to every server (disable QNAME minimisation)
  -no0x20   do not randomise the letter case of queries
  -nocolor  disable ANSI colours

serve flags:
  -addr HOST:PORT      UDP and TCP listen address (default 127.0.0.1:8053)
  -dot HOST:PORT       also serve DNS over TLS on this address (port 853 is standard)
  -cert FILE -key FILE TLS certificate and key for -dot
  -selfsigned          use a generated self-signed certificate for -dot
  -allow LIST          clients allowed to query, e.g. 127.0.0.0/8,192.168.0.0/16
                       (default: loopback only; others get REFUSED)
  -rate N -burst N     per-client query rate limit (default 20/s, burst 40)
  -dnssec, -nomin, -no0x20, -trace

examples:
  dnsr github.com
  dnsr -t MX gmail.com
```

```
  dnsr -dnssec cloudflare.com
  dnsr serve -dnssec -dot 127.0.0.1:8853 -selfsigned
  dnsr serve -addr 0.0.0.0:8053 -allow 127.0.0.0/8,192.168.0.0/16
`


func main() {
    if len(os.Args) > 1 && os.Args[1] == "serve" {
        fs := flag.NewFlagSet("serve", flag.ExitOnError)
        fs.Usage = func() { fmt.Fprint(os.Stderr, usage) }
        addr := fs.String("addr", "127.0.0.1:8053", "UDP and TCP listen address")
        dot := fs.String("dot", "", "DNS over TLS listen address")
        cert := fs.String("cert", "", "TLS certificate file")
        key := fs.String("key", "", "TLS key file")
        selfSigned := fs.Bool("selfsigned", false, "generate a self-signed certificate")
        rate := fs.Float64("rate", 20, "queries per second per client")
        burst := fs.Float64("burst", 40, "burst size per client")
        dnssec := fs.Bool("dnssec", false, "validate answers with DNSSEC")
        nomin := fs.Bool("nomin", false, "disable QNAME minimisation")
        no0x20 := fs.Bool("no0x20", false, "disable 0x20 case randomisation")
        allow := fs.String("allow", "", "comma-separated client networks allowed to query")
        trace := fs.Bool("trace", false, "print full resolution trace for each query")
        fs.Parse(os.Args[2:])
        acl, err := ParseAllow(*allow)
        if err != nil {
            fmt.Fprintln(os.Stderr, "serve: bad -allow list:", err)
            os.Exit(2)
        }
        r := NewResolver()
        r.DNSSEC, r.Minimise, r.CaseRandomisation = *dnssec, !*nomin, !*no0x20
        if *trace {
            r.Trace = os.Stderr
        }
        s := NewServer(r)
        s.Rate, s.Burst, s.Allow = *rate, *burst, acl
        cfg := ServeConfig{Addr: *addr, DoTAddr: *dot, CertFile: *cert, KeyFile: *key,
SelfSigned: *selfSigned}
        if err := serve(cfg, s); err != nil {
            fmt.Fprintln(os.Stderr, "serve:", err)
            os.Exit(1)
        }
        return
    }

    qt := flag.String("t", "A", "record type")
    quiet := flag.Bool("q", false, "quiet")
    dnssec := flag.Bool("dnssec", false, "validate with DNSSEC")
    nomin := flag.Bool("nomin", false, "disable QNAME minimisation")
    no0x20 := flag.Bool("no0x20", false, "disable 0x20 case randomisation")
    nocolor := flag.Bool("nocolor", false, "disable colour")
    flag.Usage = func() { fmt.Fprint(os.Stderr, usage) }
    flag.Parse()
    if flag.NArg() == 0 {
        flag.Usage()
        os.Exit(2)
    }
    qtype, err := ParseType(*qt)
    if err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(2)
    }

    r := NewResolver()
    r.DNSSEC, r.Minimise, r.CaseRandomisation = *dnssec, !*nomin, !*no0x20
    if !*quiet {
        r.Trace = os.Stdout
    }
    if fi, _ := os.Stdout.Stat(); !*nocolor && fi != nil && fi.Mode()&os.ModeCharDevice != 0 {
        r.color = true
    }

    exit := 0
    for i, name := range flag.Args() {
        if i > 0 {
            fmt.Println()
        }
        fmt.Println(r.c(cBold, fmt.Sprintf(";; resolving %s %s", printable(fqdn(name)), qtype)))
        res, err := r.Resolve(name, qtype)
```

```
        if err != nil {
            fmt.Println(r.c(cRed, ";; error: "+err.Error()))
            exit = 1
            continue
        }
        if res.Security == SecBogus {
            fmt.Println(r.c(cRed, ";; SERVFAIL · DNSSEC validation failed: "+res.SecurityNote))
            exit = 1
            continue
        }
        sec := ""
        if res.Security != SecUnchecked {
            sec = " · " + res.Security.String()
        }
        fmt.Printf("\n;; %s%s · %d hops · %s\n", rcodeString(res.Rcode), sec, res.Hops,
res.Elapsed.Round(1e6))
        for _, rr := range res.Answers {
            fmt.Println(printable(rr.String()))
        }
        if len(res.Answers) == 0 {
            for _, rr := range res.Authority {
                fmt.Println(r.c(cDim, rr.String()))
            }
        }
    }
    os.Exit(exit)
}
```

A.6 resolver_test.go

```
package main

// resolver_test.go — a fake DNS hierarchy on loopback (127.0.0.10–15) so the
// full iterative walk can be tested offline and deterministically:
//
//   .            127.0.0.10  → delegates com., org.
//   com.         127.0.0.11  → delegates example.com. (with glue), glueless.com. (no glue)
//   example.com. 127.0.0.12  → authoritative: www, alias (CNAME), mail (MX), big (TC→TCP)
//   org.         127.0.0.13  → delegates dnshost.org.
//   dnshost.org. 127.0.0.14  → ns.dnshost.org A 127.0.0.15
//   glueless.com 127.0.0.15  → authoritative, its NS lives under .org

import (
    "bytes"
    "crypto/ecdsa"
    "crypto/elliptic"
    "crypto/rand"
    "crypto/sha256"
    "encoding/asn1"
    "encoding/binary"
    "io"
    "math/big"
    "net"
    "strconv"
    "strings"
    "sync"
    "testing"
    "time"
)

const testPort = "15353"

type fakeZone struct {
    origin  string
    records []RR
    deleg   map[string][]RR
    glue    []RR
    tcOnUDP map[string]bool
    signer  *testSigner
    tamper  map[string]string
    forge   map[string]bool
    badTCP  map[string]bool
    noCase  bool
```

```
    mu      sync.Mutex
    seen    []Question
    raw     []string
}

func a(name, ip string) RR {
    return RR{Name: name, Type: TypeA, Class: 1, TTL: 300, Data: ip, Raw: net.ParseIP(ip).To4()}
}
func ns(zone, host string) RR { return RR{Name: zone, Type: TypeNS, Class: 1, TTL: 3600, Data:
host} }

type testSigner struct {
    zone string
    key  *ecdsa.PrivateKey
    rr   RR
    tag  uint16
}

func newSigner(zone string) *testSigner {
    k, _ := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
    pub, _ := k.PublicKey.Bytes()
    rd := append([]byte{0x01, 0x01, 3, 13}, pub[1:]...)
    return &testSigner{zone, k, RR{Name: zone, Type: TypeDNSKEY, Class: 1, TTL: 3600, Raw: rd},
keyTag(rd)}
}

func (s *testSigner) ds() RR {
    owner, _ := encodeName(nil, s.zone)
    sum := sha256.Sum256(append(owner, s.rr.Raw...))
    rd := append(binary.BigEndian.AppendUint16(nil, s.tag), 13, 2)
    return RR{Name: s.zone, Type: TypeDS, Class: 1, TTL: 3600, Raw: append(rd, sum[:]...)}
}

func (s *testSigner) anchor() DSRecord {
    d, _ := parseDS(s.ds())
    return d
}

func (s *testSigner) sign(set []RR) RR {
    now := uint32(time.Now().Unix())
    sig := &rrsig{TypeCovered: set[0].Type, Algorithm: 13, Labels:
uint8(labelCount(set[0].Name)), OrigTTL: set[0].TTL,
        Expiration: now + 3600, Inception: now - 3600, KeyTag: s.tag, Signer: s.zone}
    data, _ := signedData(sig, set)
    h := sha256.Sum256(data)
    der, _ := ecdsa.SignASN1(rand.Reader, s.key, h[:])
    sb := rawECDSASignature(der, 32)
    return RR{Name: set[0].Name, Type: TypeRRSIG, Class: 1, TTL: set[0].TTL, Raw:
append(sig.prefix(), sb...)}
}

// rawECDSASignature converts an ASN.1 ECDSA signature to the fixed-size r||s form DNSSEC uses.
func rawECDSASignature(der []byte, size int) []byte {
    var sig struct{ R, S *big.Int }
    asn1.Unmarshal(der, &sig)
    return append(sig.R.FillBytes(make([]byte, size)), sig.S.FillBytes(make([]byte, size))...)
}

// withSigs signs each RRset of an answer, then applies any tampering, so a
// tampered record carries a signature over the original data.
func (z *fakeZone) withSigs(an []RR) []RR {
    if z.signer != nil && len(an) > 0 {
        var order []RRType
        sets := map[RRType][]RR{}
        for _, rr := range an {
            if _, ok := sets[rr.Type]; !ok {
                order = append(order, rr.Type)
            }
            sets[rr.Type] = append(sets[rr.Type], rr)
        }
        for _, t := range order {
            an = append(an, z.signer.sign(sets[t]))
        }
    }
```

```
    for i := range an {
        if v, ok := z.tamper[an[i].Name]; ok && an[i].Type == TypeA {
            an[i].Data, an[i].Raw = v, net.ParseIP(v).To4()
        }
    }
    return an
}

func (z *fakeZone) rawNames() []string {
    z.mu.Lock()
    defer z.mu.Unlock()
    return append([]string(nil), z.raw...)
}

func (z *fakeZone) questions() []Question {
    z.mu.Lock()
    defer z.mu.Unlock()
    return append([]Question(nil), z.seen...)
}

func (z *fakeZone) answer(q Question, udp bool) []byte {
    z.mu.Lock()
    z.seen = append(z.seen, q)
    z.mu.Unlock()
    resp := func(rcode int, aa, tc bool, an, au, ad []RR) []byte {
        b := buildResponse(0, q, false, rcode, an, au, ad)
        if aa {
            b[2] |= 0x04
        }
        if tc {
            b[2] |= 0x02
        }
        return b
    }
    soa := RR{Name: z.origin, Type: TypeSOA, Class: 1, TTL: 60,
        Data: "ns." + z.origin + " admin." + z.origin + " 1 7200 900 1209600 60"}

    if _, cut := z.deleg[q.Name]; cut && q.Type == TypeDS {
        var ds []RR
        for _, rr := range z.records {
            if rr.Name == q.Name && rr.Type == TypeDS {
                ds = append(ds, rr)
            }
        }
        if len(ds) == 0 {
            return resp(0, true, false, nil, []RR{soa}, nil)
        }
        return resp(0, true, false, z.withSigs(ds), nil, nil)
    }
    for child, nsrrs := range z.deleg {
        if isSubdomain(q.Name, child) {
            return resp(0, false, false, nil, nsrrs, z.glue)
        }
    }
    var exists bool
    var an []RR
    for _, rr := range z.records {
        if rr.Name != q.Name {
            continue
        }
        exists = true
        if rr.Type == q.Type || rr.Type == TypeCNAME {
            an = append(an, rr)
        }
    }
    if !exists {
        return resp(RcodeNXDomain, true, false, nil, z.withSigs([]RR{soa}), nil)
    }
    if udp && z.tcOnUDP[q.Name] {
        return resp(0, true, true, nil, nil, nil)
    }
    if len(an) == 0 {
        return resp(0, true, false, nil, []RR{soa}, nil)
    }
    return resp(0, true, false, z.withSigs(an), nil, nil)
}
```

```
func startFake(t *testing.T, ip string, z *fakeZone) {
    t.Helper()
    addr := net.JoinHostPort(ip, testPort)
    pc, err := net.ListenPacket("udp", addr)
    if err != nil {
        t.Skipf("cannot bind %s: %v", addr, err)
    }
    ln, err := net.Listen("tcp", addr)
    if err != nil {
        t.Skipf("cannot bind tcp %s: %v", addr, err)
    }
    t.Cleanup(func() { pc.Close(); ln.Close() })

    reply := func(pkt []byte, udp bool) []byte {
        m, err := ParseMessage(pkt)
        if err != nil || len(m.Questions) != 1 {
            return nil
        }
        q := m.Questions[0]
        asked := q.Name
        q.Name = strings.ToLower(asked)
        z.mu.Lock()
        z.raw = append(z.raw, asked)
        z.mu.Unlock()
        if !udp && z.badTCP[q.Name] {
            q.Name = "evil." + q.Name
        }
        out := z.answer(q, udp)
        if !z.noCase && q.Name == strings.ToLower(asked) {
            enc, _ := encodeName(nil, asked)
            copy(out[12:], enc)
        }
        binary.BigEndian.PutUint16(out, m.Header.ID)
        return out
    }
    go func() {
        buf := make([]byte, 1500)
        for {
            n, from, err := pc.ReadFrom(buf)
            if err != nil {
                return
            }
            if m, err := ParseMessage(buf[:n]); err == nil && len(m.Questions) == 1 &&
z.forge[strings.ToLower(m.Questions[0].Name)] {
                q := m.Questions[0]
                forged := buildResponse(m.Header.ID^0xFFFF, q, false, 0, []RR{a(q.Name,
"6.6.6.6")}, nil, nil)
                pc.WriteTo(forged, from)
            }
            if out := reply(buf[:n], true); out != nil {
                pc.WriteTo(out, from)
            }
        }
    }()
    go func() {
        for {
            c, err := ln.Accept()
            if err != nil {
                return
            }
            go func(c net.Conn) {
                defer c.Close()
                var lb [2]byte
                if _, err := io.ReadFull(c, lb[:]); err != nil {
                    return
                }
                pkt := make([]byte, binary.BigEndian.Uint16(lb[:]))
                io.ReadFull(c, pkt)
                out := reply(pkt, false)
                c.Write(append(binary.BigEndian.AppendUint16(nil, uint16(len(out))), out...))
            }(c)
        }
    }()
}

type testNet struct {
```

```
    r     *Resolver
    trace *bytes.Buffer
    zones map[string]*fakeZone
    root  *testSigner
}

func newTestNet(t *testing.T) *testNet {
    rootKey, comKey, exKey := newSigner("."), newSigner("com."), newSigner("example.com.")
    many := []RR{}
    for i := range 40 {
        i++
        many = append(many, a("many.example.com.", "10.1.2."+strconv.Itoa(i)))
    }
    var fanout []RR
    for i := range 8 {
        i++
        fanout = append(fanout, ns("fanout.com.", "ns"+strconv.Itoa(i)+".nowhere.org."))
    }
    huge := a("huge.example.com.", "10.7.7.7")
    huge.TTL = 4000000000


    zones := map[string]*fakeZone{
        ".": {origin: ".",
            records: []RR{rootKey.rr, comKey.ds()},
            deleg: map[string][]RR{
                "com.": {ns("com.", "ns.gtld.com.")},
                "org.": {ns("org.", "ns.org.")},
            },
            glue:   []RR{a("ns.gtld.com.", "127.0.0.11"), a("ns.org.", "127.0.0.13")},
            signer: rootKey,
        },
        "com.": {origin: "com.",
            records: []RR{comKey.rr, exKey.ds()},
            deleg: map[string][]RR{
                "example.com.":  {ns("example.com.", "ns1.example.com.")},
                "glueless.com.": {ns("glueless.com.", "ns.dnshost.org.")},
                "trap.com.":     {ns("net.", "ns.evil.net.")},
                "fanout.com.":   fanout,
            },
            glue: []RR{
                a("ns1.example.com.", "127.0.0.12"),
                a("ns.dnshost.org.", "6.6.6.6"),
            },
            signer: comKey,
        },
        "example.com.": {origin: "example.com.",
            records: append([]RR{
                exKey.rr,
                a("www.example.com.", "93.184.216.34"),
                {Name: "alias.example.com.", Type: TypeCNAME, Class: 1, TTL: 300, Data:
"www.example.com."},
                {Name: "example.com.", Type: TypeMX, Class: 1, TTL: 300, Data: "10
mail.example.com."},
                a("mail.example.com.", "93.184.216.35"),
                a("big.example.com.", "10.9.9.9"),
                a("badtcp.example.com.", "10.9.9.8"),
                a("forged.example.com.", "10.1.1.1"),
                a("tampered.example.com.", "10.2.2.2"),
                huge,
            }, many...),
            tcOnUDP: map[string]bool{"big.example.com.": true, "badtcp.example.com.": true},
            badTCP:  map[string]bool{"badtcp.example.com.": true},
            forge:   map[string]bool{"forged.example.com.": true},
            tamper:  map[string]string{"tampered.example.com.": "6.6.6.6"},
            signer:  exKey,
        },
        "org.": {origin: "org.",
            deleg: map[string][]RR{"dnshost.org.": {ns("dnshost.org.", "ns.dnshost.org.")}},
            glue:  []RR{a("ns.dnshost.org.", "127.0.0.14")},
        },
        "dnshost.org.": {origin: "dnshost.org.",
            records: []RR{a("ns.dnshost.org.", "127.0.0.15")},
        },
        "glueless.com.": {origin: "glueless.com.",
            records: []RR{a("glueless.com.", "10.0.0.1")},
        },
    }
```

```
    for ip, z := range map[string]string{"127.0.0.10": ".", "127.0.0.11": "com.", "127.0.0.12":
"example.com.",
        "127.0.0.13": "org.", "127.0.0.14": "dnshost.org.", "127.0.0.15": "glueless.com."} {
        startFake(t, ip, zones[z])
    }

    r := NewResolver()
    r.Port = testPort
    r.Timeout = 500 * time.Millisecond
    r.Roots = []nameserver{{"fake-root.", "127.0.0.10"}}
    r.Anchors = []DSRecord{rootKey.anchor()}
    var trace bytes.Buffer
    r.Trace = &trace
    return &testNet{r, &trace, zones, rootKey}
}

func newTestResolver(t *testing.T) (*Resolver, *bytes.Buffer) {
    n := newTestNet(t)
    return n.r, n.trace
}

func TestReferralChain(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("www.example.com", TypeA)
    if err != nil {
        t.Fatal(err)
    }
    if len(res.Answers) != 1 || res.Answers[0].Data != "93.184.216.34" || res.Hops != 3 {
        t.Fatalf("got %+v\n%s", res, trace)
    }
    for _, want := range []string{"referral to com.", "referral to example.com.", "answered 1 A"}
{
        if !strings.Contains(trace.String(), want) {
            t.Errorf("trace missing %q:\n%s", want, trace)
        }
    }
}

func TestDelegationCache(t *testing.T) {
    r, trace := newTestResolver(t)
    r.Resolve("www.example.com", TypeA)
    trace.Reset()
    res, err := r.Resolve("mail.example.com", TypeA)
    if err != nil || res.Hops != 1 {
        t.Fatalf("expected 1 hop via cached example.com. delegation, got %+v err=%v\n%s", res,
err, trace)
    }
    if strings.Contains(trace.String(), "fake-root") {
        t.Errorf("should not have touched the root:\n%s", trace)
    }
}

func TestCNAME(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("alias.example.com", TypeA)
    if err != nil {
        t.Fatal(err)
    }
    if len(res.Answers) != 2 || res.Answers[0].Type != TypeCNAME || res.Answers[1].Data !=
"93.184.216.34" {
        t.Fatalf("got %+v\n%s", res.Answers, trace)
    }
}

func TestNXDomainAndNoData(t *testing.T) {
    r, _ := newTestResolver(t)
    res, err := r.Resolve("nope.example.com", TypeA)
    if err != nil || res.Rcode != RcodeNXDomain || len(res.Authority) == 0 ||
res.Authority[0].Type != TypeSOA {
        t.Fatalf("NXDOMAIN: %+v %v", res, err)
    }
    res, err = r.Resolve("www.example.com", TypeAAAA)
    if err != nil || res.Rcode != RcodeNoError || len(res.Answers) != 0 {
        t.Fatalf("NODATA: %+v %v", res, err)
    }
}
```

```
func TestMX(t *testing.T) {
    r, _ := newTestResolver(t)
    res, err := r.Resolve("example.com", TypeMX)
    if err != nil || len(res.Answers) != 1 || res.Answers[0].Data != "10 mail.example.com." {
        t.Fatalf("%+v %v", res, err)
    }
}

func TestGluelessDelegationIgnoresPoisonedGlue(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("glueless.com", TypeA)
    if err != nil || len(res.Answers) != 1 || res.Answers[0].Data != "10.0.0.1" {
        t.Fatalf("%+v %v\n%s", res, err, trace)
    }
    if strings.Contains(trace.String(), "6.6.6.6") {
        t.Errorf("used out-of-bailiwick glue:\n%s", trace)
    }
    if !strings.Contains(trace.String(), "no glue") {
        t.Errorf("expected glueless path:\n%s", trace)
    }
}

func TestOutOfBailiwickReferralRejected(t *testing.T) {
    r, _ := newTestResolver(t)
    _, err := r.Resolve("www.trap.com", TypeA)
    if err == nil || !strings.Contains(err.Error(), "bailiwick") {
        t.Fatalf("expected bailiwick error, got %v", err)
    }
}

func TestTruncationFallsBackToTCP(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("big.example.com", TypeA)
    if err != nil || len(res.Answers) != 1 || res.Answers[0].Data != "10.9.9.9" {
        t.Fatalf("%+v %v\n%s", res, err, trace)
    }
    if !strings.Contains(trace.String(), "TCP") {
        t.Errorf("expected TCP retry in trace:\n%s", trace)
    }
}

func TestNameCompression(t *testing.T) {
    pkt := []byte{
        0x12, 0x34, 0x81, 0x80, 0, 1, 0, 1, 0, 0, 0, 0,
        3, 'w', 'w', 'w', 7, 'e', 'x', 'a', 'm', 'p', 'l', 'e', 3, 'c', 'o', 'm', 0,
        0, 1, 0, 1,
        0xC0, 0x0C, 0, 1, 0, 1, 0, 0, 0x0E, 0x10, 0, 4, 1, 2, 3, 4,
    }
    m, err := ParseMessage(pkt)
    if err != nil || m.Answers[0].Name != "www.example.com." || m.Answers[0].Data != "1.2.3.4" {
        t.Fatalf("%+v %v", m, err)
    }
    loop := append(append([]byte{}, pkt[:12]...), 0xC0, 0x0C)
    if _, err := ParseMessage(loop); err == nil {
        t.Fatal("expected loop error")
    }
}
```

A.7 security_test.go

```
package main


// security_test.go — tests for the hardening features, QNAME minimisation,
// DNSSEC validation and the server, all against the fake hierarchy.


import (
    "crypto/tls"
    "crypto/x509"
    "encoding/binary"
    "io"
```

```
    "net"
    "strings"
    "testing"
    "time"
)

func TestTTLIsCapped(t *testing.T) {
    r, _ := newTestResolver(t)
    res, err := r.Resolve("huge.example.com", TypeA)
    if err != nil || len(res.Answers) != 1 {
        t.Fatalf("%+v %v", res, err)
    }
    if res.Answers[0].TTL > defaultMaxTTL {
        t.Fatalf("TTL %d not capped at %d", res.Answers[0].TTL, defaultMaxTTL)
    }
    r.mu.Lock()
    e := r.cache["huge.example.com.|A"]
    r.mu.Unlock()
    if time.Until(e.expires) > defaultMaxTTL*time.Second {
        t.Fatalf("cache entry expires in %v", time.Until(e.expires))
    }
}

func TestCachesAreBounded(t *testing.T) {
    r := NewResolver()
    r.MaxCacheEntries = 5
    for i := range 50 {
        name := "n" + string(rune('a'+i%26)) + string(rune('a'+i/26)) + ".example."
        r.cachePut(name, TypeA, []RR{a(name, "10.0.0.1")}, nil)
        r.putZone(name, []nameserver{{"ns.", "10.0.0.1"}}, 300)
    }
    if len(r.cache) > 5 || len(r.zoneNS) > 5 {
        t.Fatalf("cache %d entries, zoneNS %d entries; limit 5", len(r.cache), len(r.zoneNS))
    }
}

func TestExpiredDelegationIsIgnored(t *testing.T) {
    r := NewResolver()
    r.zoneNS["example.com."] = zoneEntry{[]nameserver{{"ns1.example.com.", "10.0.0.1"}},
time.Now().Add(-time.Second)}
    if zone, _ := r.closestKnownZone("www.example.com."); zone != "." {
        t.Fatalf("used expired delegation %s", zone)
    }
}

func TestGluelessLookupsAreLimited(t *testing.T) {
    n := newTestNet(t)
    _, err := n.r.Resolve("www.fanout.com", TypeA)
    if err == nil || !strings.Contains(err.Error(), "could not find an address") {
        t.Fatalf("expected failure, got %v", err)
    }
    asked := map[string]bool{}
    for _, q := range n.zones["org."].questions() {
        if strings.HasSuffix(q.Name, ".nowhere.org.") && strings.HasPrefix(q.Name, "ns") {
            asked[q.Name] = true
        }
    }
    if len(asked) == 0 || len(asked) > maxGluelessLookups {
        t.Fatalf("looked up %d of 8 nameservers, want 1..%d: %v", len(asked), maxGluelessLookups,
asked)
    }
}

func TestForgedReplyIsIgnored(t *testing.T) {
    r, trace := newTestResolver(t)
    res, err := r.Resolve("forged.example.com", TypeA)
    if err != nil || len(res.Answers) != 1 || res.Answers[0].Data != "10.1.1.1" {
        t.Fatalf("accepted a forged reply: %+v %v\n%s", res, err, trace)
    }
}

func TestTCPReplyMustMatchQuestion(t *testing.T) {
    r, trace := newTestResolver(t)
    _, err := r.Resolve("badtcp.example.com", TypeA)
    if err == nil || !strings.Contains(err.Error(), "does not match") {
        t.Fatalf("expected TCP mismatch error, got %v\n%s", err, trace)
```

```
    }
}

func TestQNAMEMinimisation(t *testing.T) {
    n := newTestNet(t)
    if _, err := n.r.Resolve("www.example.com", TypeA); err != nil {
        t.Fatal(err)
    }
    for zone, want := range map[string]string{".": "com.", "com.": "example.com."} {
        for _, q := range n.zones[zone].questions() {
            if q.Name != want || q.Type != TypeNS {
                t.Errorf("%s server saw %s %s, want only %s NS", zone, q.Name, q.Type, want)
            }
        }
    }
}

func TestQNAMEMinimisationCanBeDisabled(t *testing.T) {
    n := newTestNet(t)
    n.r.Minimise = false
    n.r.Resolve("www.example.com", TypeA)
    if qs := n.zones["."].questions(); len(qs) == 0 || qs[0].Name != "www.example.com." {
        t.Errorf("with minimisation off the root should see the full name, saw %v", qs)
    }
}

func TestDNSSECSecure(t *testing.T) {
    n := newTestNet(t)
    n.r.DNSSEC = true
    for _, name := range []string{"www.example.com", "alias.example.com"} {
        res, err := n.r.Resolve(name, TypeA)
        if err != nil || res.Security != SecSecure {
            t.Fatalf("%s: security %v (%s) err %v\n%s", name, res.Security, res.SecurityNote,
err, n.trace)
        }
    }
    res, _ := n.r.Resolve("example.com", TypeMX)
    if res.Security != SecSecure {
        t.Fatalf("MX: %v %s", res.Security, res.SecurityNote)
    }
}

func TestDNSSECBogusWhenRecordIsTampered(t *testing.T) {
    n := newTestNet(t)
    n.r.DNSSEC = true
    res, err := n.r.Resolve("tampered.example.com", TypeA)
    if err != nil || res.Security != SecBogus {
        t.Fatalf("security %v (%s) err %v", res.Security, res.SecurityNote, err)
    }
}

func TestDNSSECBogusWithWrongTrustAnchor(t *testing.T) {
    n := newTestNet(t)
    n.r.DNSSEC = true
    n.r.Anchors = []DSRecord{newSigner(".").anchor()}
    res, err := n.r.Resolve("www.example.com", TypeA)
    if err != nil || res.Security != SecBogus {
        t.Fatalf("security %v (%s) err %v", res.Security, res.SecurityNote, err)
    }
}

func TestDNSSECInsecureDelegation(t *testing.T) {
    n := newTestNet(t)
    n.r.DNSSEC = true
    res, err := n.r.Resolve("glueless.com", TypeA)
    if err != nil || res.Security != SecInsecure {
        t.Fatalf("security %v (%s) err %v", res.Security, res.SecurityNote, err)
    }
}

func TestRRSIGParsesAndPrints(t *testing.T) {
    s := newSigner("example.com.")
    sig := s.sign([]RR{a("www.example.com.", "10.0.0.1")})
    p, err := parseRRSIG(sig.Raw)
    if err != nil || p.Signer != "example.com." || p.TypeCovered != TypeA || p.KeyTag != s.tag {
```

```
        t.Fatalf("%+v %v", p, err)
    }
}

func plainQuery(id uint16, name string, qtype RRType) []byte {
    b := make([]byte, 12)
    binary.BigEndian.PutUint16(b, id)
    binary.BigEndian.PutUint16(b[2:], 0x0100)
    binary.BigEndian.PutUint16(b[4:], 1)
    b, _ = encodeName(b, name)
    b = binary.BigEndian.AppendUint16(b, uint16(qtype))
    return binary.BigEndian.AppendUint16(b, 1)
}

func startTestServer(t *testing.T, s *Server) net.Addr {
    pc, err := net.ListenPacket("udp", "127.0.0.1:0")
    if err != nil {
        t.Skip(err)
    }
    t.Cleanup(func() { pc.Close() })
    go s.ServeUDP(pc)
    return pc.LocalAddr()
}

func udpAsk(t *testing.T, addr net.Addr, q []byte) *Message {
    c, err := net.Dial("udp", addr.String())
    if err != nil {
        t.Fatal(err)
    }
    defer c.Close()
    c.SetDeadline(time.Now().Add(5 * time.Second))
    c.Write(q)
    buf := make([]byte, 4096)
    n, err := c.Read(buf)
    if err != nil {
        t.Fatal(err)
    }
    m, err := ParseMessage(buf[:n])
    if err != nil {
        t.Fatal(err)
    }
    return m
}

func TestServerSetsADForSecureAnswers(t *testing.T) {
    n := newTestNet(t)
    n.r.DNSSEC = true
    addr := startTestServer(t, NewServer(n.r))
    m := udpAsk(t, addr, plainQuery(7, "www.example.com.", TypeA))
    if !m.Header.AD || len(m.Answers) != 1 || m.Answers[0].Data != "93.184.216.34" {
        t.Fatalf("AD=%v answers=%v", m.Header.AD, m.Answers)
    }
    m = udpAsk(t, addr, plainQuery(8, "tampered.example.com.", TypeA))
    if m.Header.Rcode != RcodeServFail {
        t.Fatalf("bogus answer should be SERVFAIL, got %s", rcodeString(m.Header.Rcode))
    }
}

func TestServerTruncatesLargeUDPResponses(t *testing.T) {
    n := newTestNet(t)
    addr := startTestServer(t, NewServer(n.r))
    m := udpAsk(t, addr, plainQuery(9, "many.example.com.", TypeA))
    if !m.Header.TC || len(m.Answers) != 0 {
        t.Fatalf("expected TC=1 with no answers for a >512-byte response, got TC=%v and %d
answers", m.Header.TC, len(m.Answers))
    }
}

func TestServerRateLimit(t *testing.T) {
    s := NewServer(NewResolver())
    s.init()
    s.Rate, s.Burst = 1, 5
    allowed := 0
    for range 20 {
        if s.allow("192.0.2.1") {
            allowed++
```

```
        }
    }
    if allowed != 5 {
        t.Fatalf("allowed %d of 20 back-to-back queries, want burst of 5", allowed)
    }
    if !s.allow("192.0.2.2") {
        t.Fatal("another client should not be limited")
    }
}

func TestDNSOverTLS(t *testing.T) {
    n := newTestNet(t)
    cert, _, err := selfSignedCert()
    if err != nil {
        t.Fatal(err)
    }
    ln, err := tls.Listen("tcp", "127.0.0.1:0", dotConfig(cert))
    if err != nil {
        t.Skip(err)
    }
    t.Cleanup(func() { ln.Close() })
    go NewServer(n.r).ServeStream(ln)

    leaf, _ := x509.ParseCertificate(cert.Certificate[0])
    pool := x509.NewCertPool()
    pool.AddCert(leaf)
    c, err := tls.Dial("tcp", ln.Addr().String(), &tls.Config{RootCAs: pool, ServerName:
"localhost"})
    if err != nil {
        t.Fatal(err)
    }
    defer c.Close()
    c.SetDeadline(time.Now().Add(5 * time.Second))
    q, _ := BuildQuery(42, "www.example.com.", TypeA, false)
    c.Write(append(binary.BigEndian.AppendUint16(nil, uint16(len(q))), q...))
    var lb [2]byte
    if _, err := io.ReadFull(c, lb[:]); err != nil {
        t.Fatal(err)
    }
    buf := make([]byte, binary.BigEndian.Uint16(lb[:]))
    io.ReadFull(c, buf)
    m, err := ParseMessage(buf)
    if err != nil || m.Header.ID != 42 || len(m.Answers) != 1 || m.Answers[0].Data !=
"93.184.216.34" {
        t.Fatalf("%+v %v", m, err)
    }
}

func TestServerStripsDNSSECRecordsForPlainClients(t *testing.T) {
    n := newTestNet(t)
    addr := startTestServer(t, NewServer(n.r))
    m := udpAsk(t, addr, plainQuery(10, "nope.example.com.", TypeA))
    if m.Header.Rcode != RcodeNXDomain || m.Header.TC {
        t.Fatalf("rcode %s TC=%v", rcodeString(m.Header.Rcode), m.Header.TC)
    }
    for _, rr := range m.Authority {
        if rr.Type == TypeRRSIG {
            t.Fatalf("plain client received %s", rr.Type)
        }
    }
    q, _ := BuildQuery(11, "nope.example.com.", TypeA, true)
    m = udpAsk(t, addr, q)
    sigs := 0
    for _, rr := range m.Authority {
        if rr.Type == TypeRRSIG {
            sigs++
        }
    }
    if sigs == 0 {
        t.Fatal("client with DO=1 should receive the RRSIG")
    }
}

func TestDNSOverTLSRequiresTLS13(t *testing.T) {
    cert, _, err := selfSignedCert()
    if err != nil {
        t.Fatal(err)
```

```
    }
    ln, err := tls.Listen("tcp", "127.0.0.1:0", dotConfig(cert))
    if err != nil {
        t.Skip(err)
    }
    t.Cleanup(func() { ln.Close() })
    go func() {
        for {
            c, err := ln.Accept()
            if err != nil {
                return
            }
            c.(*tls.Conn).Handshake()
            c.Close()
        }
    }()
    if c, err := tls.Dial("tcp", ln.Addr().String(), &tls.Config{InsecureSkipVerify: true,
MaxVersion: tls.VersionTLS12}); err == nil {
        c.Close()
        t.Fatal("a TLS 1.2 client was accepted")
    }
    c, err := tls.Dial("tcp", ln.Addr().String(), &tls.Config{InsecureSkipVerify: true})
    if err != nil {
        t.Fatal(err)
    }
    defer c.Close()
    if v := c.ConnectionState().Version; v != tls.VersionTLS13 {
        t.Fatalf("negotiated TLS version %x, want 1.3", v)
    }
}

func TestQueryIDsAndCaseAreRandom(t *testing.T) {
    ids := map[uint16]bool{}
    for range 2000 {
        ids[randomID()] = true
    }
    if len(ids) < 1900 {
        t.Fatalf("only %d distinct IDs in 2000 draws", len(ids))
    }
    name := "www.example-domain-with-many-letters.com."
    variants := map[string]bool{}
    for range 20 {
        m := randomCase(name)
        if !strings.EqualFold(m, name) || len(m) != len(name) {
            t.Fatalf("randomCase changed the name: %s", m)
        }
        variants[m] = true
    }
    if len(variants) < 19 {
        t.Fatalf("only %d distinct case patterns in 20 draws", len(variants))
    }
}

func TestQueriesUseRandomCaseAndAnswersAreLowercase(t *testing.T) {
    n := newTestNet(t)
    for _, name := range []string{"www.example.com", "mail.example.com", "alias.example.com"} {
        res, err := n.r.Resolve(name, TypeA)
        if err != nil || len(res.Answers) == 0 {
            t.Fatalf("%s: %+v %v", name, res, err)
        }
        for _, rr := range res.Answers {
            if rr.Name != strings.ToLower(rr.Name) {
                t.Fatalf("answer name not normalised: %s", rr.Name)
            }
        }
    }
    mixed := 0
    for _, q := range n.zones["example.com."].rawNames() {
        if q != strings.ToLower(q) {
            mixed++
        }
    }
    if mixed == 0 {
        t.Fatalf("no query used mixed case: %v", n.zones["example.com."].rawNames())
    }
}
```

```
func TestCaseRandomisationFallsBackForServersThatDoNotPreserveCase(t *testing.T) {
    n := newTestNet(t)
    n.zones["example.com."].noCase = true
    for _, name := range []string{"www.example.com", "mail.example.com", "alias.example.com",
"big.example.com", "forged.example.com"} {
        res, err := n.r.Resolve(name, TypeA)
        if err != nil || len(res.Answers) == 0 {
            t.Fatalf("%s failed against a case-insensitive server: %+v %v\n%s", name, res, err,
n.trace)
        }
        if n.r.caseUnsafe("127.0.0.12") {
            break
        }
    }
    if !n.r.caseUnsafe("127.0.0.12") || !strings.Contains(n.trace.String(), "does not preserve
letter case") {
        t.Fatalf("fallback not recorded:\n%s", n.trace)
    }
}

func TestServerAccessList(t *testing.T) {
    n := newTestNet(t)
    s := NewServer(n.r)
    acl, err := ParseAllow("10.0.0.0/8, 192.168.1.5")
    if err != nil {
        t.Fatal(err)
    }
    s.Allow = acl
    addr := startTestServer(t, s)
    m := udpAsk(t, addr, plainQuery(12, "www.example.com.", TypeA))
    if m.Header.Rcode != RcodeRefused || len(m.Answers) != 0 {
        t.Fatalf("client outside the list got %s with %d answers", rcodeString(m.Header.Rcode),
len(m.Answers))
    }
    for ip, want := range map[string]bool{"10.1.2.3": true, "192.168.1.5": true, "192.168.1.6":
false,
        "127.0.0.1": false, "::ffff:10.0.0.1": true, "garbage": false} {
        if got := s.permitted(ip); got != want {
            t.Errorf("custom list: permitted(%s) = %v, want %v", ip, got, want)
        }
    }
    def := NewServer(n.r)
    for ip, want := range map[string]bool{"127.0.0.1": true, "127.5.5.5": true, "::1": true,
"192.168.1.1": false} {
        if got := def.permitted(ip); got != want {
            t.Errorf("default list: permitted(%s) = %v, want %v", ip, got, want)
        }
    }
    for _, bad := range []string{"10.0.0.0/33", "not-an-ip"} {
        if _, err := ParseAllow(bad); err == nil {
            t.Errorf("ParseAllow(%q) should fail", bad)
        }
    }
}
```

A.8 go.mod

```
module github.com/swayam5342/dnsr

go 1.25
```

A.9 fuzz_test.go (evaluation harness, Chapter 7)

```
package main

import (
    "math/rand"
    "testing"
)
```

```
func seedPackets() [][]byte {
    q, _ := BuildQuery(0x1234, "www.example.com.", TypeA, true)
    resp := []byte{
        0x12, 0x34, 0x81, 0x80, 0, 1, 0, 1, 0, 0, 0, 0,
        3, 'w', 'w', 'w', 7, 'e', 'x', 'a', 'm', 'p', 'l', 'e', 3, 'c', 'o', 'm', 0,
        0, 1, 0, 1,
        0xC0, 0x0C, 0, 1, 0, 1, 0, 0, 0x0E, 0x10, 0, 4, 1, 2, 3, 4,
    }
    s := newSigner("example.com.")
    signed := buildResponse(0x1234, Question{"www.example.com.", TypeA, 1}, false, 0,
        []RR{a("www.example.com.", "1.2.3.4"), s.sign([]RR{a("www.example.com.", "1.2.3.4")}),
s.rr, s.ds()}, nil, nil)
    return [][]byte{q, resp, signed}
}

func FuzzParseMessage(f *testing.F) {
    for _, s := range seedPackets() {
        f.Add(s)
    }
    f.Fuzz(func(t *testing.T, b []byte) {
        if m, err := ParseMessage(b); err == nil {
            for _, rr := range m.Answers {
                if rr.Type == TypeRRSIG {
                    parseRRSIG(rr.Raw)
                }
                if rr.Type == TypeDNSKEY {
                    parseDNSKEY(rr)
                }
            }
        }
    })
}

func TestRandomAndMutatedPackets(t *testing.T) {
    rng := rand.New(rand.NewSource(42))
    seeds := seedPackets()
    errs, oks := 0, 0
    for i := range 1_000_000 {
        var b []byte
        if i%2 == 0 {
            b = make([]byte, rng.Intn(600))
            rng.Read(b)
        } else {
            s := seeds[rng.Intn(len(seeds))]
            b = append([]byte(nil), s...)
            for range 1 + rng.Intn(6) {
                b[rng.Intn(len(b))] = byte(rng.Intn(256))
            }
            if rng.Intn(4) == 0 {
                b = b[:rng.Intn(len(b))]
            }
        }
        if _, err := ParseMessage(b); err != nil {
            errs++
        } else {
            oks++
        }
    }
    t.Logf("1,000,000 packets: %d rejected with error, %d parsed, 0 panics", errs, oks)
}

func BenchmarkParseMessage(b *testing.B) {
    p := seedPackets()[1]
    for b.Loop() {
        ParseMessage(p)
    }
}

func BenchmarkBuildQuery(b *testing.B) {
    for i := 0; b.Loop(); i++ {
        BuildQuery(uint16(i), "www.example.com.", TypeA, false)
    }
}

func BenchmarkVerifyECDSAP256(b *testing.B) {
```

```
    s := newSigner("example.com.")
    k, _ := parseDNSKEY(s.rr)
    set := []RR{a("www.example.com.", "1.2.3.4")}
    sig := s.sign(set)
    for b.Loop() {
        if err := verifyRRset(set, []RR{sig}, []*dnskey{k}, "example.com."); err != nil {
            b.Fatal(err)
        }
    }
}
```

APPENDIX B

USER MANUAL

B.1 Building and testing (Go 1.25 or later)

```
go build -o dnsr .        # Linux / macOS
go build -o dnsr.exe .    # Windows
go test -v ./...          # run the offline test suite
go test -race ./...       # with the race detector
```

B.2 Resolving names

```
./dnsr github.com
./dnsr -t MX gmail.com
./dnsr -t AAAA www.wikipedia.org vit.ac.in   # several names share one cache
./dnsr -q -t TXT google.com                  # answer only, no trace
./dnsr -dnssec cloudflare.com                # validate from the root trust anchor
./dnsr -nomin www.github.com                 # send the full name to every server
./dnsr -t DNSKEY iana.org                    # DNSSEC record types can be queried too
```

B.3 Running the recursive server

```
./dnsr serve -addr 0.0.0.0:8053 -allow 127.0.0.0/8,192.168.0.0/16   # serve a LAN
./dnsr serve                                  # UDP and TCP on 127.0.0.1:8053
./dnsr serve -dnssec -trace                   # validate, print a trace per query
./dnsr serve -dot 127.0.0.1:853 -cert c.pem -key k.pem   # DNS over TLS
./dnsr serve -dot 127.0.0.1:8853 -selfsigned   # test certificate, fingerprint logged
./dnsr serve -rate 50 -burst 100              # per-client rate limit
dig @127.0.0.1 -p 8053 example.com            # query it from another terminal
```

B.4 Requirements and troubleshooting

- ✓ Outbound UDP and TCP port 53 must be allowed. Some campus and corporate networks block it and force their own resolver; if every root server times out, try another network such as a phone hotspot.
- ✓ The default server port is 8053 because port 5353 is used by the multicast DNS service on Windows.
- ✓ The Windows `nslookup` tool does not honour a custom port; use `dig` or a script based on dnspython to query the server.
- ✓ With `-dnssec`, a bogus answer is printed as SERVFAIL with the reason; in server mode it is returned as SERVFAIL and secure answers carry the AD flag.
- ✓ Exit status 0 means every name resolved; 1 means at least one resolution failed or was bogus; 2 means a usage error.